

Teoretická informatika

Obor C, 3. ročník

David Weber

SPŠE JEČNÁ

Poslední aktualizace: 30. července 2026

Předmluva

Obsah

Předmluva	1
1 Grafové algoritmy	7
1.1 Grafy a jejich reprezentace	7
1.2 Stromy	9
1.3 Prohledávání do šířky	10
1.4 Prohledávání do hloubky	12
1.5 Binární halda	13
1.5.1 Vkládání nového vrcholu	15
1.5.2 Odstranění minima	15
1.5.3 Zvýšení a snížení hodnoty klíče ve vrcholu	16
1.5.4 Časové složitosti operací	16
1.6 Dijkstrův algoritmus	18
1.6.1 Správnost Dijkstrova algoritmu	19
1.6.2 Časová složitost Dijkstrova algoritmu	20
1.7 Algoritmus A*	21
1.7.1 Správnost algoritmu A*	22
1.7.2 Ukázky heuristických funkcí	23
2 Algoritmicky těžké problémy	26
2.1 Problém splnitelnosti	26
2.1.1 Konjunktivní normální forma	27
2.1.2 Převod do CNF pomocí tabulky	28

2.1.3	Tseitinova transformace	29
2.2	Problém SAT	31
2.2.1	Algoritmus DPLL	31
2.3	Problém 3-SAT	35
2.3.1	Převod SAT na 3-SAT	35
2.4	Problém nezávislé množiny v grafu	38
2.4.1	Převod 3-SAT na NzMna	39
2.4.2	Aplikace NzMna – barvení intervalového grafu	40
2.5	Problém kliky v grafu	41
2.5.1	Převod Kliky na NzMna	42
2.6	Třídy problémů P a NP	43
2.6.1	Rozbor předchozích problémů	43
2.6.2	Ověřování problémů	44
2.6.3	Zavedení tříd složitosti	45
2.6.4	Další NP-úplné problémy	48
3	Konečné automaty	50
3.1	Úvod	50
3.2	Formální jazyky	51
3.2.1	Rozhodovací problémy jako jazyky	52
3.3	Deterministické konečné automaty	52
3.4	Nedeterministické konečné automaty	54
3.4.1	Vztah DFA a NFA	55
3.4.2	Přechody bez čtení znaku	56
3.4.3	Regulární jazyky	56
3.5	Regulární výrazy	57
3.5.1	Kleeneho věta	58
3.6	Gramatiky	60
3.6.1	Derivační stromy	61

3.6.2	Regulární gramatiky	62
4	Kompilátory	65
4.1	Úvodem	65
4.1.1	Překladový řetězec	65
4.1.2	Fáze překladače	66
4.2	Lexikální analýza	67
4.2.1	Lexémy a tokeny	67
4.2.2	Od regulárního výrazu k lexeru	68
4.3	Syntaktická analýza	71
4.3.1	Bezkontextové gramatiky	71
4.3.2	Aritmetické výrazy	72
4.3.3	Analýza shora dolů a zdola nahoru	72
4.4	Sémantická analýza	76
4.4.1	Tabulka symbolů	76
4.4.2	Typová kontrola	76
4.5	Optimalizace	78
4.5.1	Lokální optimalizace	78
4.5.2	Globální optimalizace a graf toku řízení	80
4.5.3	Generování bajtkódu	80
4.6	Interpretované jazyky	82
4.6.1	Virtuální stroj	82
4.6.2	JIT kompilace	84
5	Hardware	86
5.1	Základní části počítače	86
5.2	Uložení programu a dat	87
5.2.1	Von Neumannova architektura	87
5.2.2	Harvardská architektura	88
5.3	Procesor	89

5.3.1	Sběrnice	90
5.3.2	Instrukční cyklus	90
5.3.3	Registry	91
5.4	Vnitřní paměti	92
5.4.1	Cache	92
5.4.2	Hlavní paměť RAM	93
5.5	Externí paměti a úložiště	94
5.5.1	Pevný disk HDD	95
5.5.2	Disk SSD	96
5.6	Paměťová hierarchie	97
6	Kryptografie	99
6.1	Co je to kryptografie?	99
6.1.1	Šifrování a dešifrování	100
6.2	Odbočka k pravděpodobnosti	100
6.2.1	Náhodný pokus, výsledky a jevy	101
6.2.2	Podmíněná pravděpodobnost a nezávislost	102
6.3	Historické metody	103
6.3.1	Substituční šifra s posunem	103
6.3.2	Obecná substituční šifra	104
6.4	Symetrická kryptografie	104
6.4.1	Operace XOR	105
6.4.2	Jednorázová šifra	105
6.4.3	Proč se klíč nesmí opakovat?	106
6.5	Asymetrická kryptografie	107
6.5.1	Výpočetní bezpečnost	108
6.6	Algoritmus RSA	108
6.6.1	Vytvoření klíčů	108
6.6.2	Šifrování a dešifrování	109

6.6.3	Malá Fermatova věta a čínská věta o zbytcích	109
6.6.4	Korektnost RSA	109
6.6.5	Bezpečnost a faktorizace	111
6.7	Pseudonáhodné generátory a jednosměrné funkce	111
6.7.1	Zanedbatelná funkce	112
6.7.2	Pseudonáhodný generátor	112
6.7.3	Žádné pseudonáhodné generátory při $P = NP$	113
6.7.4	Jednosměrná funkce	114
6.7.5	Souvislost OWF, PRG a šifrování	115
6.8	Hešovací funkce	116
6.8.1	Kolize jsou nevyhnutelné	116
6.8.2	Narozeninový paradox	116
6.8.3	Bezpečnostní vlastnosti	118
6.9	Secure Hash Algorithm	119
6.9.1	Sponge function	119
6.9.2	SHA-3 jako konkrétní sponge function	120
6.10	Elektronický podpis	121
6.10.1	Podpis otisku	122
6.10.2	Certifikát a ověření veřejného klíče	122
6.10.3	Elektronický podpis v právním rámci	123

7 Umělá inteligence

124

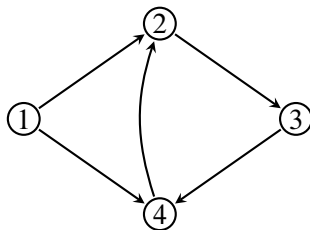
Grafové algoritmy

1.1 Grafy a jejich reprezentace

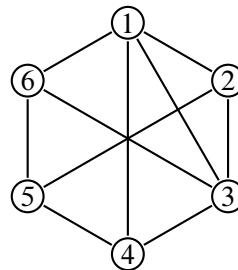
Grafy jsou základním stavebním kamenem mnoha úloh v informatice. Zachycují objekty a jejich vzájemné vztahy. Lze jimi reprezentovat třeba propojení zařízení v počítačové síti, schéma elektrického obvodu, mapu města nebo vazby mezi atomy. Jejich matematická podstata je však velice jednoduchá – *vrcholy* propojené *hranami*.

Definice 1.1.1 (Graf). Grafem G nazveme uspořádanou dvojici (V, E) , kde V je množina *vrcholů* (nebo také *uzlů*) a E množina *hran*, přičemž pokud

- $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$, pak G nazýváme *neorientovaným* grafem;
- $E \subseteq \{(u, v) \mid u, v \in V, u \neq v\}$, pak G nazýváme *orientovaným* grafem.



(a) Příklad orientovaného grafu.

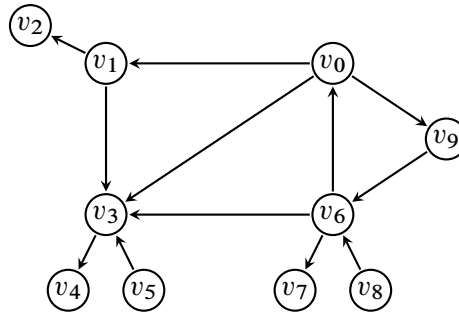


(b) Příklad neorientovaného grafu.

Podstatnou otázkou u grafů jsou způsoby jejich reprezentace, neboť při programování jsou různé způsoby různě efektivní a je dobré si mezi nimi umět vybrat. Pracujme tedy s grafem $G = (V, E)$, přičemž vrcholy si pro jednoduchost označíme přirozenými čísly $1, 2, \dots, n$, tzn. $V = \{1, 2, \dots, n\}$.

- **Matice sousednosti.** První způsob, jak graf reprezentovat, je pomocí matice. V tomto případě graf G popisuje matice $n \times n$, kde na pozici ij je 1, pokud z vrcholu i do vrcholu j vede hrana, jinak 0.

Příklad 1.1.2 (Prásátkový graf). Orientovaný graf G (viz obrázek 1.2) a jeho matice sousednosti A .

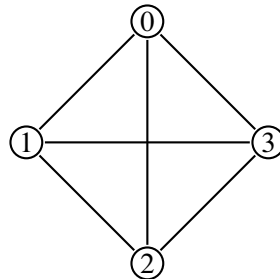


Obrázek 1.2: Prasátkový graf.

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- **Matice incidence.** Řádky jsou indexovány vrcholy a sloupce jednotlivými hranami. Na místě ij je 1, pokud hrana vede *do* vrcholu i , nebo -1 , pokud z něj vychází, jinak 0. Pro neorientované grafy píšeme vždy 1, pokud daná hrana vede *do/z* vrcholu.

Příklad 1.1.3 (Úplný graf K_4). Graf G má 4 vrcholy a celkem 6 hran (viz obrázek 1.3); jeho matici incidence označíme $\mathbf{I}$. Řádky představují (shora dolů) vrcholy 0, 1, 2, 3 a sloupce (zleva doprava) jednotlivé hrany $e_1, \dots, e_6$.

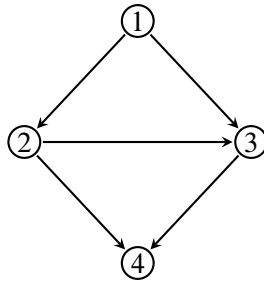
Obrázek 1.3: Úplný graf K_4 .

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

- **Seznam sousedů.** Jedná se pravděpodobně o nejpoužívanější variantu reprezentace grafů. Pro každý vrchol $v \in V$ si jednoduše pamatujeme seznam $S[v]$ všech vrcholů, do kterých přímo vede hrana z vrcholu v .¹

Příklad 1.1.4. Graf G s vrcholy $1, \dots, 4$ a jejich seznamy sousedů S (viz obrázek 1.4).

⁽¹⁾Název *seznam* zde odkazuje na použitou datovou strukturu. Z matematického pohledu popisuje $S[v]$ množinu sousedů.



$$\begin{array}{ll} S[1] = \{2, 3\} & S[3] = \{4\} \\ S[2] = \{3, 4\} & S[4] = \emptyset \end{array}$$

Obrázek 1.4: Graf G a seznamy sousedů pro jeho každý vrchol.

Tím jsme si ukázali základní možnosti reprezentace grafů, avšak zatím jsme si je mezi sebou nijak neporovnali. Pojďme se tedy podívat na paměťové nároky daných variant. Označme si počet vrcholů grafu $|V| = n$ a celkový počet hran $|E| = m$.

- *Matice sousednosti* spotřebuje paměť $\mathcal{O}(n^2)$, neboť potřebujeme uložit matici velikosti $n \times n$. Je tak vhodná především pro husté grafy, tedy grafy s velkým počtem hran.
- *Matice incidence* využije $\mathcal{O}(nm)$ paměťových buněk. Tento způsob reprezentace je vhodný především pro matematickou práci s grafy, avšak při programování se moc nevyužívá, neboť obsahuje spoustu nadbytečných informací.
- *Reprezentace pomocí seznamů sousedů* je obecně hodně využívána. Spotřebuje $\mathcal{O}(n + m)$ paměti.² Je proto vhodná zejména pro řídké grafy, které neobsahují příliš mnoho hran (např. stromy).

1.2 Stromy

Stromy tvoří speciální oblast teorie grafů. V informatice jde o jeden z nejčastěji používaných typů grafů, a proto jim věnujeme samostatnou sekci, v níž si připomeneme některé základní pojmy.

Definice 1.2.1 (Strom). Neorientovaný graf $G = (V, E)$ je strom, pokud

- je souvislý³
- a neobsahuje kružnici.

Je dobré si uvědomit, že mezi každou dvojicí vrcholů ve stromě vede **právě jedna** cesta. Zároveň z definice stromu plyne, že odebráním libovolné hrany se nutně poruší souvislost: přestane existovat cesta mezi koncovými vrcholy odebrané hrany.

Poslední příklad 1.5c sice není z definice strom, avšak čtenář by mohl namítnout, že jednotlivé “části” daného grafu tvoří stromy. Tomuto typu grafů se říká (celkem pochopitelně) *lesy*, neboť jejich *komponenty souvislosti*⁴ tvoří stromy (viz obrázek 1.6).

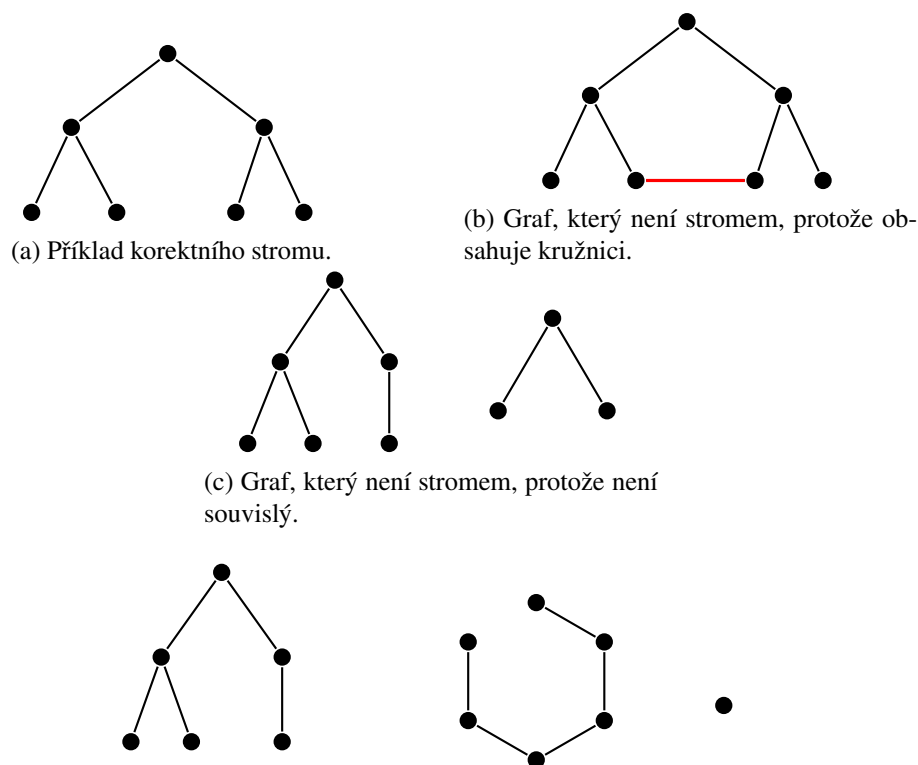
Definice 1.2.2 (Les). Graf $G = (V, E)$ je les, pokud každá jeho komponenta souvislosti je strom.

Ekvivalentně můžeme les definovat jako *disjunktní sjednocení stromů*. Jako poslední si uvedme speciální typ stromu, se kterým budeme dále nejčastěji pracovat, a to tzv. *binární strom*. Blíže se s ním setkáme v sekci 1.5, kdy budeme rozebírat tzv. *binární haldu*.

⁽²⁾U orientovaného grafu uložíme každou hranu jednou, u neorientovaného dvakrát, jednou u každého koncového vrcholu. Konstantní násobek však v $\mathcal{O}$ -notaci nehraje roli. Blíže se na tento argument podíváme v odvození časové složitosti algoritmu BFS, viz 1.3.1.

⁽³⁾Mezi každou dvojicí vrcholů vede cesta.

⁽⁴⁾Intuitivně se jedná o část grafu, kde mezi každými dvěma vrcholy vede cesta. Formální zavedení termínu se provádí přes *binární relaci*. Pro zájemce: [https://en.wikipedia.org/wiki/Component_\(graph_theory\)](https://en.wikipedia.org/wiki/Component_(graph_theory)).

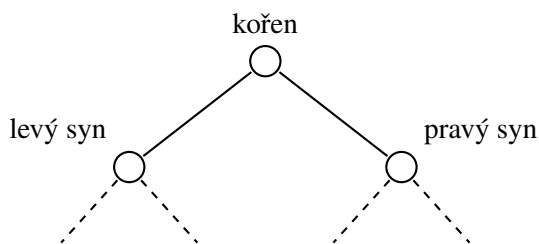


Obrázek 1.6: Příklad lesa.

Definice 1.2.3 (Binární strom). Strom $T = (V, E)$ nazveme binární, pokud

- (i) je zakořeněný
- (ii) a každý vrchol má nejvýše dva syny.

Jeden z vrcholů stromu T je označen jako *kořen*. Typicky jej zakreslujeme nahoru nad všechny ostatní vrcholy a syny každého vrcholu umísťujeme vždy níže oba na stejnou úroveň (viz obrázek 1.7).



Obrázek 1.7: Příklad binárního stromu.

1.3 Prohledávání do šířky

Jednou ze základních úloh je procházení grafu z určitého vrcholu a zjištění dosažitelnosti ostatních vrcholů. Nejjednodušším algoritmem v tomto ohledu je tzv. *prohledávání do šířky* (anglicky *breadth-first search*, zkráceně BFS). Jeho základní princip spočívá v postupném objevování sousedů již nalezených vrcholů. Na počátku dostaneme graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$. Postupně objevíme všechny sousedy vrcholu v_0 , poté všechny dosud nenalezené sousedy těchto vrcholů atd. Na BFS lze nahlížet tak, že do počátečního vrcholu nalijeme vodu a sledujeme, jak grafem postupuje vzniklá vlna.

Pro každý vrchol si budeme uchovávat jeho *stav*.

- *Nenalezený* – vrchol jsme ještě během výpočtu neviděli.
- *Otevřený* – vrchol jsme již našli, ale ještě jsme neprozkoumali všechny jeho sousedy.
- *Uzavřený* – vrchol jsme prozkoumali společně se všemi jeho sousedy a dál se jím již netřeba zabývat.

Na počátku začneme s jedním otevřeným vrcholem, a to v_0 (zde začínáme). Po prozkoumání všech jeho dosud nenalezených sousedů se jejich stav změní na otevřený a počáteční vrchol v_0 se uzavře. Obdobně pokračujeme pro nově otevřené vrcholy. Pokud by náhodou mezi dvojicí otevřených vrcholů existovala hrana, pak si sousedního vrcholu všimnout nebudeme, neboť byl již otevřen. Pro každý vrchol si ještě dodatečně můžeme uchovávat informaci, jak daleko se nachází od v_0 , co do počtu hran ležících na cestě.

Algoritmus 1.3.1: BFS (prohledávání do šířky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

// Inicializace

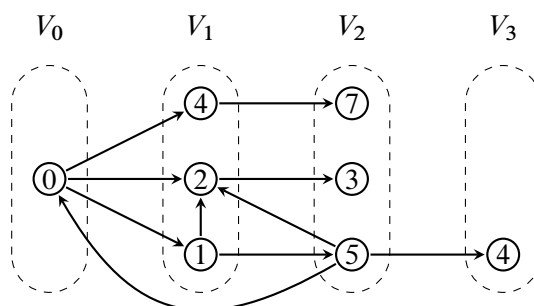
```

1 foreach vrchol  $v \in V$  do
2    $stav(v) \leftarrow \text{nenalezený}$ 
3    $D(v) \leftarrow \infty$ 
4  $stav(v_0) \leftarrow \text{otevřený}$ 
5  $D(v_0) \leftarrow 0$ 
6 Založ frontu  $Q$  a přidej do ní vrchol  $v_0$ 
7 while fronta  $Q$  je neprázdná do
8    $v \leftarrow$  první vrchol ve frontě  $Q$ , který z ní odebereme
   // Prozkoumáváme všechny dosud neobjevené sousedy
9   foreach sousední vrchol  $w$  vrcholu  $v$  do
10    if  $stav(w) = \text{nenalezený}$  then
11       $stav(w) \leftarrow \text{otevřený}$ 
12       $D(w) \leftarrow D(v) + 1$ 
13      Přidej  $w$  do fronty  $Q$ 
14  $stav(v) \leftarrow \text{uzavřený}$ 

```

Výstup: Seznam vzdáleností D

BFS rozděluje vrcholy do vrstev podle toho, v jaké vzdálenosti od počátečního vrcholu se nachází (viz obrázek 1.8). Nejdříve jsou prozkoumány vrcholy ve vzdálenosti 0 (tj. pouze v_0), poté ve vzdálenostech 1, 2, $\dots$. Z toho vyplývá, že kdykoliv při otevírání libovolného vrcholu v nastavujeme hodnotu $D(v)$, bude tato hodnota vždy odpovídat délce nejkratší cesty z v_0 do v . Tato hodnota bude však nastavena pouze u těch vrcholů, které jsou z v_0 dosažitelné (ostatní vrcholy zůstanou ve stavu nenalezený).



Obrázek 1.8: Vrcholy rozdělené do vrstev podle průběhu BFS.

Zbývá prozkoumat časovou a paměťovou složitost BFS. Označme si počet vrcholů grafu G na vstupu n a počet jeho hran m .

Věta 1.3.1 (Složitost BFS). Algoritmus BFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Inicializace potrvá $\mathcal{O}(n)$, neboť cyklus iteruje přes všechny vrcholy. Vnější cyklus provede maximálně n iterací, protože každý z vrcholů uzavřeme nejvýše jednou, tj. $\mathcal{O}(n)$.

S vnitřním cyklem je to trochu složitější, protože jeho počet iterací závisí na tom, který z vrcholů otevíráme (resp. na počtu jeho sousedů). Označme vrcholy $v_1, \dots, v_n$. Celkový počet iterací vnitřního cyklu přes všechny vrcholy bude $\sum_{i=1}^n \deg(v_i)$. Lze si ovšem všimnout jedné užitečné věci. Pokaždé, když prozkoumáváme sousední vrchol w nějakého vrcholu v , mohou nastat dva případy podle toho, jestli je G orientovaný graf, nebo neorientovaný.

- (i) Graf G je neorientovaný. Pak hranu, která spojuje v a w prozkoumáme právě *dvakrát* (jednou z vrcholu v a podruhé z vrcholu w). Každá hrana se tak započítá dvakrát, tzn. vnitřní cyklus se celkově provede max $2m$ -krát (po všech iteracích vnějšího cyklu).
- (ii) Graf G je orientovaný. Pak se hrana započítá pouze jednou, a to z vrcholu, z něhož vede. Celkově se vnitřní cyklus provede m -krát.

V prvním případě tak pro časovou složitost bude platit

$$\mathcal{O}\left(n + \sum_{i=1}^n \deg(v_i)\right) = \mathcal{O}(n + 2m) \stackrel{*}{=} \mathcal{O}(n + m).$$

(V kroku označeném $*$ zanedbáváme konstantu, neboť pracujeme s $\mathcal{O}$ -notací.)

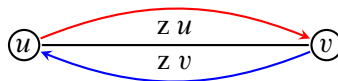
V druhém případě dojdeme ke stejnému výsledku, akorát zmíněná suma bude, kvůli orientaci hran, rovna přesně počtu hran, tj.

$$\mathcal{O}\left(n + \sum_{i=1}^n \deg(v_i)\right) = \mathcal{O}(n + m).$$

Nyní k prostorové složitosti algoritmu. Na reprezentaci grafu (např. pomocí seznamu sousedů) spotřebujeme paměť $\mathcal{O}(n + m)$ (n vrcholů, m hran)⁵. Dále si uchováváme vrcholy ve frontě, v níž může být v jednu chvíli maximálně n vrcholů, tj. $\mathcal{O}(n)$, a k tomu máme seznamy $stav$ a D , oba s n položkami. Celkově spotřebujeme paměti

$$\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m).$$

□



Obrázek 1.9: Znázornění situace v důkazu části (i) věty 1.3.1.

1.4 Prohledávání do hloubky

Na podobném přístupu jako BFS je založeno tzv. *prohledávání do hloubky* (anglicky *depth-first search*, zkráceně DFS). Vrcholy však tentokrát budeme zpracovávat rekurzivně. Pokaždé, když otevřeme nový vrchol, rekurzivně

⁽⁵⁾Resp. každá hrana je v neorientovaných grafech započítána dvakrát, protože každý vrchol obsahuje v seznamu svých sousedů vždy „protějščí“ vrchol (stejný argument jako u odvozování časové složitosti BFS).

zpracujeme každého jeho dosud nenalezeného souseda. Po prozkoumání všech sousedů daný vrchol uzavřeme. Stejně jako u BFS si budeme uchovávat pole *stavů* jednotlivých vrcholů.

Algoritmus 1.4.1: DFS (prohledávání do hloubky)

```

Vstup: Graf  $G = (V, E)$  a počáteční vrchol  $v_0 \in V$ 
// Inicializace
1 foreach vrchol  $v \in V$  do
2    $stav(v) \leftarrow \text{nenalezený}$ 
3 Zavolej DFS2( $v_0$ )
// Rekurzivní volání na sousední vrcholy
4 Funkce DFS2(vrchol  $v \in V$ )
5    $stav(v) \leftarrow \text{otevřený}$ 
6   foreach sousední vrchol  $w$  vrcholu  $v$  do
7     if  $stav(w) = \text{nenalezený}$  then
8       Zavolej DFS2( $w$ )
9    $stav(v) \leftarrow \text{uzavřený}$ 

```

Věta 1.4.1 (Složitost DFS). Algoritmus DFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Algoritmus DFS se oproti BFS liší pouze v pořadí, v jakém dosažitelné vrcholy zpracovává. To však nemá na časovou složitost žádný vliv. Argument pro její odvození je stejný jako u BFS, viz věta 1.3.1 v minulé sekci.

V paměti si musíme uchovávat reprezentaci grafu, to zabere $\mathcal{O}(n + m)$ paměti (opět např. pomocí seznamu sousedů) a máme seznam *stav* s n prvky (vrcholy). Zároveň si při volání DFS2 musíme na zásobník rekurze ukládat jednotlivé *aktivační záznamy*. Protože vrcholů je v grafu n , pak na zásobníku rekurze bude v jednu chvíli maximálně n záznamů, tj. $\mathcal{O}(n)$. Celkově spotřebujeme $\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m)$ paměti. $\square$



Je dobré si uvědomit, že byť algoritmy BFS a DFS mají stejnou asymptotickou časovou i prostorovou složitost, nehodí se rovnocenně na stejné úlohy. BFS nalezne nejkratší cestu v neohodnoceném grafu. DFS se naopak často hodí tam, kde chceme nejprve prozkoumat jednu možnost opravdu do hloubky. Jeho zásobník obsahuje nejvýše jednu právě rozpracovanou cestu, zatímco fronta BFS může současně obsahovat celou širokou vrstvu grafu; v konkrétní úloze proto může DFS spotřebovat méně paměti, nejde však o obecnou asymptotickou záruku.



Cesta, kterou se DFS dostane do libovolného vrcholu v , nemusí být nutně nejkratší (silně závisí na pořadí, v jakém procházíme sousedy jednotlivých vrcholů).

1.5 Binární halda

Uvedme na úvod motivační příklad. Mějme seznam čísel, z něhož chceme co nejrychleji vybrat minimální nebo maximální hodnotu. Pro maximum bychom mohli sestavit následující jednoduchý algoritmus.

Časová složitost bude zjevně $\mathcal{O}(n)$, neboť algoritmus prochází všech n prvků. Pokud ale se seznamem pracujeme opakovaně a průběžně jej udržujeme vhodně uspořádaný, můžeme minimum či maximum získat rychleji. K tomu

Algoritmus 1.5.1: Vyhledání maximální hodnoty v seznamu (MAX)**Vstup:** Seznam čísel $x_1, x_2, \dots, x_n$

```

1  $m \leftarrow x_1$ 
2 for  $i = 2, 3, \dots, n$  do
3   if  $x_i > m$  then
4      $m \leftarrow x_i$ 
5 return  $m$ 

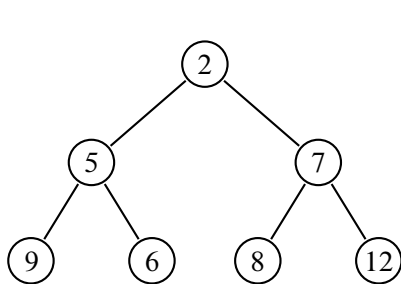
```

Výstup: Maximální hodnota seznamu m

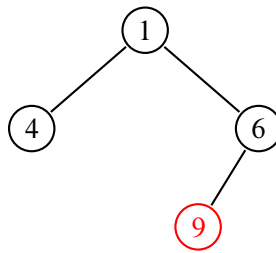
můžeme použít tzv. *haldu*.

Definice 1.5.1 (Minimová binární halda). Minimová binární halda je datová struktura tvaru binárního stromu s množinou vrcholů V a klíčovou funkcí $\text{key} : V \rightarrow \mathbb{Z}$, kde je v každém vrcholu uložena *právě jedna* hodnota $\text{key}(v)$ (tzv. *klíč*) a navíc platí:

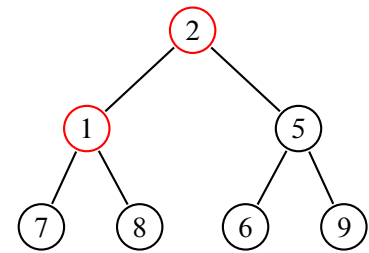
- (i) každá hladina je *plně obsazena*, kromě poslední, přičemž vrcholy poslední hladiny jsou zaplněny *zleva*,
- (ii) a je-li v libovolný vrchol a s jeho syn, pak $\text{key}(v) \leq \text{key}(s)$.



(a) Korektní halda.



(b) Mezera před obsazeným místem v poslední hladině.



(c) Klíč levého syna je menší než klíč kořene.

Obrázek 1.10: Příklady korektních a nekorektních hald.

Zde se na chvíli pozastavme. Podmínka (ii) v definici binární haldy 1.5.1 má velmi příjemný důsledek. Pokud se budeme pohybovat od kořene směrem dolů, hodnoty klíčů se nemohou zmenšovat. Minimum se proto vždy nachází v *kořeni stromu*, takže jeho zjištění zvládneme v *konstantním čase* $\mathcal{O}(1)$.



Pokud bychom chtěli naopak *maximovou binární haldu*, bude definice 1.5.1 vypadat obdobně, akorát v podmínce (ii) bude obrácená nerovnost, tj. $\text{key}(v) \geq \text{key}(s)$ (klíč v rodiči má vždy stejnou nebo vyšší hodnotu než klíče v jeho synech). Maximum se bude opět nacházet v kořeni haldy.

Je však více operací, které bychom rádi s haldou prováděli. Vypišme si všechny, které nás budou zajímat.

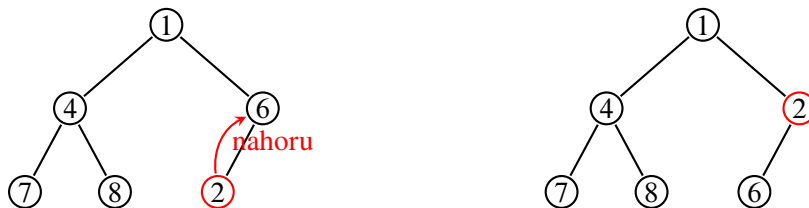
- $\text{INSERT}(x)$ – *vložení* nového vrcholu s klíčem x do haldy.
- $\text{MIN}()$ – *nalezení* vrcholu s nejmenším klíčem (už jsme zmínili).
- $\text{EXTRACTMIN}()$ – *odebrání* vrcholu s nejmenším klíčem.
- $\text{INCREASE}(v)$ – *zvýšení* hodnoty klíče ve zvoleném vrcholu v .
- $\text{DECREASE}(v)$ – *snížení* hodnoty klíče ve zvoleném vrcholu v .

Podívejme se postupně na jednotlivé operace a jejich realizaci. S dovolením si zde nyní odpustíme zápis pomocí pseudokódu a pro jednoduchost si dané operace pouze slovně vysvětlíme. Pro následné odvození časové složitosti

nám to bude stačit.

1.5.1 Vkládání nového vrcholu

Při vkládání nového vrcholu (INSERT) nejdříve potřebujeme rozhodnout, kam jej umístíme. S tím nám pomůže podmínka (i): nový vrchol vložíme na první volné místo v pořadí zleva doprava. Pokud je dosavadní poslední hladina plná, založíme novou hladinu a vrchol umístíme zcela vlevo. Tím zachováme tvar haldy, ale nemusí být splněna podmínka (ii). Klíč nového vrcholu může být menší než klíč jeho rodiče. Haldu opravíme postupným prohazováním klíče s rodičem, dokud nebude podmínka splněna (viz obrázek 1.11). Postup můžeme zapsat



Obrázek 1.11: Vkládání nového vrcholu do haldy.

zkráceně ve třech bodech takto:

- Vložíme nový vrchol s klíčem x na první volnou pozici zleva na poslední hladině.
- pokud je klíč rodiče větší než x , prohodíme vrcholy (resp. jejich klíče)⁶.
- Opakujeme druhý krok.

1.5.2 Odstranění minima

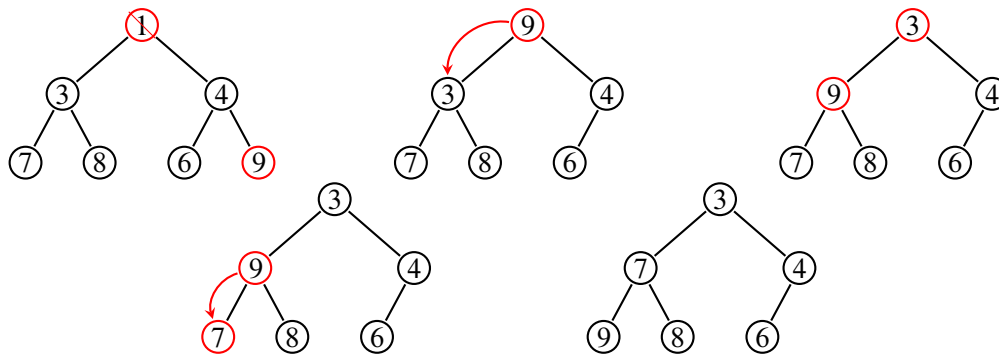
Získání minimálního klíče v haldě je triviální: přečteme klíč v kořeni a jsme hotovi. Pokud však chceme minimum operací EXTRACTMIN také odstranit, je to o něco složitější. Kořen nemůžeme jen smazat, jeho pozici musí zastoupit jiný vrchol. Abychom neporušili tvar haldy, přesuneme do kořene *poslední vrchol v pořadí zleva doprava*, tedy nejpravější obsazený vrchol poslední hladiny, a jeho původní pozici odstraníme.

Nyní však (stejně jako u vkládání vrcholu) nastává problém s druhou podmínkou, neboť touto operací jsme ji mohli porušit. Provedeme obdobný proces, jako při vkládání vrcholu do haldy, ale vrchol nyní bude „probublávat“ směrem dolů. Podíváme se na syny daného vrcholu a pokud klíč některého z nich je menší než klíč nového kořene, pak jej s ním prohodíme (v případě, že klíče obou synů jsou menší, než klíč kořene, prohodíme s menším z nich). V bodech můžeme zapsat celou operaci následovně:

- Smažeme kořen a nahradíme jej nejpravějším obsazeným vrcholem na poslední hladině.
- Tento vrchol (resp. jeho klíč) porovnáme s jeho syny a případně prohodíme s tím, jehož klíč je menší.
- Opakujeme druhý krok.

Operace, při nichž vrchol během vkládání „probublává“ nahoru nebo během odebírání minima dolů, se někdy nazývají BUBBLEUP a BUBBLEDOWN. Ještě se nám budou hodit u operací INCREASE a DECREASE.

⁽⁶⁾ Vrchol v podstatě takto „probublá“ do správné pozice ve stromě (příp. až do pozice kořene).



Obrázek 1.12: Odebírání vrcholu (kořene) s minimálním klíčem.

1.5.3 Zvýšení a snížení hodnoty klíče ve vrcholu

Poslední dvojicí operací, která se nám bude hodit, je zvýšení (INCREASE) a snížení (DECREASE) klíče ve vybraném vrcholu. Předpokládáme přitom, že umíme daný vrchol v haldě přímo najít. Pokud v haldě uchováváme složitější objekty, můžeme si například vést pomocný slovník, který každému objektu přiřadí jeho současnou pozici v haldě. Při každém prohození pak aktualizujeme i slovník. Ten spotřebuje navíc $\mathcal{O}(n)$ paměti.

Realizace samotných operací je již jednoduchá. Pokud provedeme INCREASE klíče v určitém vrcholu, může se halda pokazit pouze směrem dolů. Provedeme tedy BUBBLEDOWN, stejně jako při odebírání kořene.

Operaci DECREASE provedeme zcela stejně, akorát nyní se halda bude kazit „směrem nahoru“, takže provedeme BUBBLEUP, jako při vkládání nového vrcholu (INSERT).



V případě *maximové binární haldy* by operace vypadaly zcela stejně, akorát při operaci INCREASE budeme opravovat haldu směrem nahoru a u DECREASE směrem dolů.

1.5.4 Časové složitosti operací

Pojďme si nyní rozebrat časové složitosti zmíněných operací INSERT, MIN, EXTRACTMIN, INCREASE a DECREASE a podívat se, jak moc jsme si pomohli oproti práci s polem (seznamem). Pokud se pozorně podíváme na operace popsané výše, je zjevné, že nejvíce bude naše operace zpomalovat ono „proublávání“ vrcholu při opravování haldy (vyjma operace MIN, kde jen přečteme klíč v kořeni), tj. BUBBLEUP a BUBBLEDOWN. Na nich bude celková časová složitost záviset.

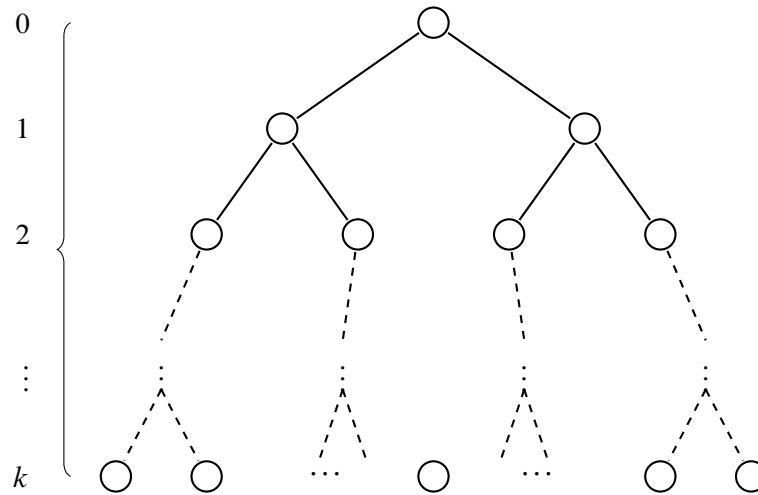


Všimněme si, že po každém prohození vrcholů při BUBBLEUP nebo BUBBLEDOWN se vrchol posune o jednu hladinu výše/níže.

Víme tak, že při opravování haldy se vrchol může posunout *maximálně o počet hladin*, který má daná halda. Časová složitost daných operací bude tak závislá na jejich počtu. Tím jsme se dostali o trochu blíže řešení, ale rádi bychom věděli, jak se bude době trvání daných operací měnit v závislosti na *počtu vrcholů v haldě*, nikoliv počtu hladin. Mezi těmito proměnnými však existuje hezká závislost.

Dejme tomu, že naše halda má n vrcholů v celkově h hladinách. Halda s jednou plnou hladinou má jeden vrchol, se dvěma plnými hladinami tři vrcholy, se třemi sedm vrcholů atd. (viz tabulka 1.1). Je-li všech h hladin plných, platí

Počet hladin h	1	2	3	4	5	6
Počet vrcholů n	1	3	7	15	31	63

Tabulka 1.1: Vztah mezi počtem hladin h a počtem vrcholů n v plně obsazené haldě.Obrázek 1.13: Plná binární halda s obecně h hladinami.

$n = 2^h - 1$. V obecné haldě jsou první $h - 1$ hladiny plné a poslední obsahuje alespoň jeden vrchol. Proto

$$2^{h-1} \leq n \leq 2^h - 1.$$

Z levé nerovnosti dostáváme $h \leq \log_2 n + 1$, tedy počet hladin je $\mathcal{O}(\log n)$. Tím máme vše připravené k odvození složitosti operací.

Věta 1.5.2 (Složitost operací v binární haldě). Operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají časovou složitost $\mathcal{O}(\log n)$. Operace MIN má časovou složitost $\mathcal{O}(1)$.

Důkaz. V případě MIN jen přečteme klíč v kořeni, což potrvá $\mathcal{O}(1)$. U zbývajících operací opravujeme haldu pomocí BUBBLEUP nebo BUBBLEDOWN. Při každém prohození se zpracováváný klíč posune o jednu hladinu. Provede se tedy nejvýše $h - 1$ prohození. Protože $h \leq \log_2 n + 1$, dostáváme

$$\mathcal{O}(h) = \mathcal{O}(\log n).$$

Operace INSERT, EXTRACTMIN, INCREASE a DECREASE proto mají logaritmickou časovou složitost. $\square$

Zkusme si na závěr udělat menší porovnání oproti poli, se kterým jsme začínali (viz tabulka 1.2). Logaritmická

	INSERT	MIN	EXTRACTMIN	INCREASE	DECREASE
Pole (seznam)	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Binární halda	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Tabulka 1.2: Porovnání časových složitostí operací v haldě a v poli.

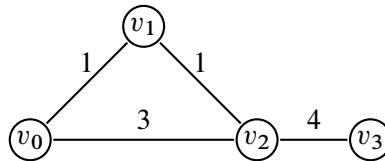
časová složitost je určitě lepší, než lineární, je však vidět, že jsme si zároveň pohoršili v případě vkládání a úpravy hodnot klíčů. Je tomu tak z důvodu, že pole má svou výhodu v téměř nulové režii, kdežto halda (kvůli své pevně definované struktuře) spotřebuje nějaký čas, aby byla uvedena do korektního stavu.

1.6 Dijkstrův algoritmus

Již jsme si ukázali, že v *neohodnoceném* grafu $G = (V, E)$ umíme relativně jednoduše najít délku nejkratší cesty z výchozího vrcholu $v_0 \in V$ do ostatních vrcholů. Hrany však nemusí mít stejnou cenu. Města mohou například představovat vrcholy a silnice mezi nimi hrany, jejichž váhou bude délka nebo doba průjezdu.



Zde již nestačí počítat počet hran. Na obrázku 1.14 je cesta (v_0, v_1, v_2) levnější než přímá cesta (v_0, v_2) , přestože obsahuje více hran.



Obrázek 1.14: Příklad ohodnoceného grafu.

Pro kladné celočíselné váhy se nabízí převést ohodnocený graf na neohodnocený tak, že hranu váhy k nahradíme cestou o k hranách. Na nově vzniklý graf již lze aplikovat např. BFS, které jsme popsali v sekci 1.3. Ne každý graf však musí mít „malé“ váhy hran. Pokud bychom vzali např. graf, kde váhy hran jsou v řádech tisíců, bude pro BFS potřeba provést zbytečně mnoho práce, neboť pro zpracování jedné ohodnocené hrany bude třeba vytvořit a projít tisíce hran.

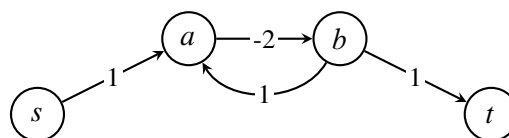
Jedním z nejznámějších algoritmů pro tuto úlohu je *Dijkstrův algoritmus*, který popsal v roce 1959 nizozemský informatik *Edsger W. Dijkstra*.⁷ Podobně jako u BFS a DFS budeme postupně otevírat a uzavírat vrcholy, přičemž každý uzavřeme nejvýše jednou.



První nalezená cesta do libovolného vrcholu již nemusí být nutně nejkratší. Cestu do daného vrcholu bude třeba postupně vylepšovat.

Ohodnocený graf budeme v souladu s prezentacemi zapisovat jako $G = (V, E, w)$, kde $w : E \rightarrow \mathbb{R}$ je váhová funkce. Je-li hrana určena koncovými vrcholy u, v , píšeme pro stručnost $w(u, v)$.

Zaměříme se na grafy, jejichž váhy hran jsou nezáporné. Dijkstrův algoritmus pro graf se zápornou hranou obecně nefunguje, a to ani tehdy, když graf neobsahuje záporný cyklus. Pokud navíc na cestě k cíli leží dosažitelný cyklus se zápornou celkovou vahou, nejkratší cesta konečné délky vůbec neexistuje. Na obrázku 1.15 lze cyklus (a, b, a) projít libovolněkrát a každým průchodem cenu cesty snížit o 1. Infimum cen cest z s do t je proto $-\infty$. Podobným



$$d(s, t) = -\infty$$

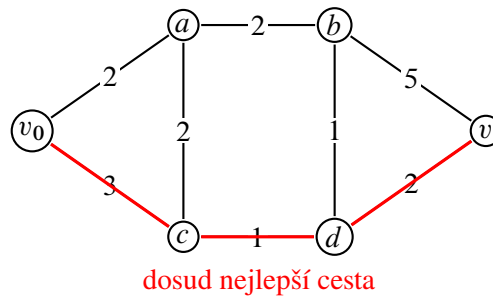
Obrázek 1.15: Graf, kde mezi v_0 a v_4 neexistuje (konečná) nejkratší cesta.

⁽⁷⁾Dijkstrův stručný článek vyšel roku 1959; samotný algoritmus Dijkstra navrhl již roku 1956. Jeho příjmení se přibližně vyslovuje „dajkstra“.

grafům se tak chceme vyhnout, neboť by naše úvahy pak již nemusely fungovat (cestu mezi vrcholy bychom mohli potenciálně do nekonečně vylepšovat a algoritmus by se tak nikdy nezastavil).

Algoritmus 1.6.1: DIJKSTRA

Vstup: Nezáporně ohodnocený graf $G = (V, E, w)$ a počáteční vrchol $v_0 \in V$
 // Inicializace
 1 **foreach** vrchol $v \in V$ **do**
 2 $stav(v) \leftarrow \text{nenalezený}$
 3 $D(v) \leftarrow \infty$
 4 $stav(v_0) \leftarrow \text{otevřený}$
 5 $D(v_0) \leftarrow 0$
 6 **while** existují otevřené vrcholy **do**
 7 // Kandidát na nejkratší cestu do sousedních vrcholů
 8 $v \leftarrow$ otevřený vrchol s nejmenším D
 9 **foreach** soused $s \in V$ vrcholu v **do**
 10 // Našli jsme lepší cestu
 11 **if** $stav(s) \neq \text{uzavřený}$ a $D(v) + w(v, s) < D(s)$ **then**
 12 $D(s) \leftarrow D(v) + w(v, s)$
 13 $stav(s) \leftarrow \text{otevřený}$
 14 $stav(v) \leftarrow \text{uzavřený}$
Výstup: Seznam vzdáleností D



Obrázek 1.16: Znázornění situace v průběhu Dijkstrova algoritmu.

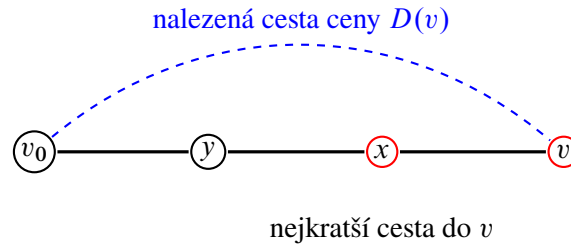
1.6.1 Správnost Dijkstrova algoritmu

Nejprve je dobré si uvědomit, proč algoritmus funguje. Hodnota $D(v)$ je vždy cenou nějaké již nalezené cesty z v_0 do v , a proto nemůže být menší než skutečná nejkratší vzdálenost. Zbývá ukázat opačnou nerovnost ve chvíli, kdy vrchol uzavíráme.

Nejkratší vzdálenost mezi vrcholy u a v označme $d(u, v)$. Důkaz vedeme indukcí podle pořadí uzavírání vrcholů. Předpokládejme pro spor, že v je první uzavíraný vrchol, pro který $D(v) > d(v_0, v)$. Vezměme jednu nejkratší cestu z v_0 do v . Na této cestě musí existovat první dosud neuzavřený vrchol x ; jeho předchůdce y už uzavřený je. Podle indukčního předpokladu platilo při uzavření y rovnost $D(y) = d(v_0, y)$. Algoritmus tehdy zkontroloval hranu (y, x) , a proto nastavil

$$D(x) \leq d(v_0, y) + w(y, x) = d(v_0, x) \leq d(v_0, v) < D(v).$$

Využili jsme zde nezápornost hran: vzdálenost podél nejkratší cesty se směrem k vrcholu v nemůže zmenšovat. Vrchol x je tedy otevřený a má menší hodnotu D než v . Algoritmus by proto vybral x , nikoli v , což je spor.



Obrázek 1.17: Situace při výběru vrcholu v , kdy $D(v)$ neodpovídá délce nejkratší cesty.

1.6.2 Časová složitost Dijkstrova algoritmu

Zbývá nám ještě analyzovat časovou složitost. Pokud se zpětně podíváme na pseudokód algoritmu, lze si všimnout, že podstatnou roli bude hrát vybírání vrcholu s minimální hodnotou $D(v)$. Nabízí se tak otázka, jak danou operaci implementovat. První variantou je hodnoty D realizovat jako prostý seznam (popř. pole).

Věta 1.6.1 (Dijkstra se seznamem). Dijkstrův algoritmus, kde hodnoty D ukládáme do seznamu (pole), doběhne v čase $\mathcal{O}(n^2 + m)$.

Důkaz. • Inicializace zabere $\mathcal{O}(n)$ (máme n vrcholů).

- Každý vrchol uzavřeme opět nejvýše jednou, tzn. vnější cyklus se provede $\mathcal{O}(n)$ -krát.
- Hledání minimálního D zabere v rámci každé iterace čas $\mathcal{O}(n)$.
- Stejně jako u BFS a DFS, i zde každou hranu zkontroluji nejvýše dvakrát (záleží, zda graf G je orientovaný), tzn. $\mathcal{O}(2m) = \mathcal{O}(m)$.

Celkově tedy máme

$$\overbrace{\mathcal{O}(n)}^{\text{Init}} + \underbrace{\mathcal{O}(n) \cdot \mathcal{O}(n)}_{\text{Výběr minima}} + \mathcal{O}(m) = \mathcal{O}(n^2 + n + m) = \mathcal{O}(n^2 + m).$$

□

Kvadratická časová složitost není úplně zlá, ale můžeme si ještě trochu přilepšit. Pokud si vzpomeneme na binární haldy z oddílu 1.5, všimneme si v konečném důsledku, že při použití v Dijkstrově algoritmu se některé operace zrychlí.

Věta 1.6.2 (Dijkstra s haldou). Dijkstrův algoritmus, kde hodnoty D ukládáme do binární haldy, doběhne v čase $\mathcal{O}((n + m) \cdot \log n)$.

Důkaz. Inicializace trvá zase $\mathcal{O}(n)$. U operací s binární haldou víme, jak dlouho potrvají (viz tabulka 1.2). Musíme si jen rozmyslet, kdy se která operace provádí.

- Při prvním *otevření* vrcholu provedeme v haldě operaci INSERT. Celkově vložíme nejvýše všech n vrcholů, což potrvá $\mathcal{O}(n \log n)$.
- Při *uzavírání* vrcholu provádíme operaci EXTRACTMIN (opět max. n -krát), tj. $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při každé další *aktualizaci vzdálenosti* $D(v)$ provádíme DECREASE (tato hodnota se nikdy nemůže zvýšit). Aktualizací je nejvýše tolik, kolik kontrol hran, tedy $\mathcal{O}(m)$. Dostáváme nejvýše $m \cdot \mathcal{O}(\log n) = \mathcal{O}(m \log n)$.

Po sečtení dostaneme

$$\begin{aligned}\mathcal{O}(n + n \log n + n \log n + m \log n) &= \mathcal{O}(n + 2n \log n + m \log n) \\ &= \mathcal{O}(n + n \log n + m \log n) \\ &= \mathcal{O}(n \log n + m \log n) \\ &= \mathcal{O}((n + m) \cdot \log n).\end{aligned}$$

□

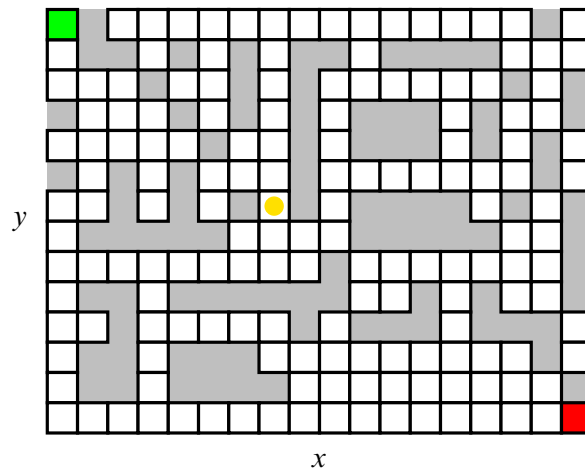
Ačkoliv to není úplně zjevné, výraz $(n + m) \cdot \log n$ roste o dost pomaleji oproti $n^2 + m$, tedy binární haldou si skutečně pomůžeme oproti seznamu.

Závěrem uvedme, že často nehledáme cestu do všech vrcholů, ale pouze mezi konkrétní dvojicí. Výpočet můžeme zastavit ve chvíli, kdy cílový vrchol vybereme jako minimum a uzavřeme jej: tehdy už je jeho hodnota D konečná. Hledání lze v některých situacích dále urychlit, protože Dijkstrův algoritmus může prozkoumávat i vrcholy směřující úplně jinam než k cíli. Na to se podíváme v další sekci 1.7.

1.7 Algoritmus A*

Nyní se omezíme na případy, kdy hledáme cestu pouze do konkrétního cílového vrcholu $g \in V$ ⁸. Stále pracujeme s grafy, jejichž hrany mají nezáporné ohodnocení. Dijkstrův algoritmus by samozřejmě fungoval i zde: skončili bychom ve chvíli, kdy cílový vrchol vybereme jako minimum. Nabízí se však otázka, zda nemůžeme znalost cíle nějak využít. Pokud bychom například hledali nejkratší cestu v mřížce s překážkami, dávalo by smysl upřednostňovat políčka, která vypadají být cíli blíž.

Mějme bludiště, v němž se hráč může pohybovat o jedno pole vodorovně nebo svisle; na některá pole vstoupit nesmí. Cíl se nachází vpravo dole. Situaci na obrázku 1.18 můžeme znázornit grafem, v němž každá přípustná dvojice sousedních polí tvoří hranu s vahou 1. Pokud bychom použili BFS nebo Dijkstrův algoritmus, prozkoumávali

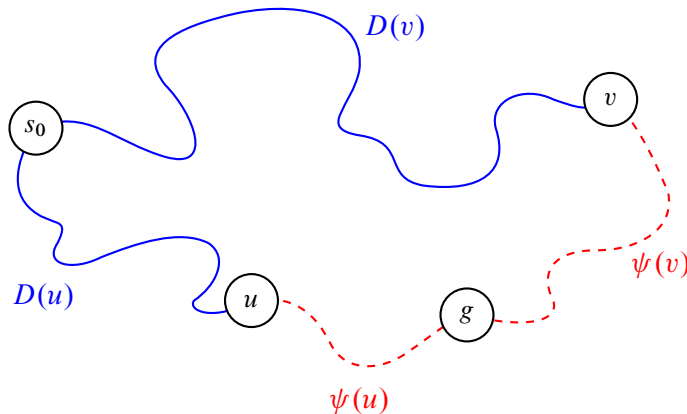


Obrázek 1.18: Mřížka a optimální směr pohybu hráče.

bychom okolí startu rovnoměrně do všech směrů. Algoritmus A*, který roku 1968 popsali Peter Hart, Nils Nilsson a Bertram Raphael, se snaží hledání směřovat k cíli.

⁽⁸⁾Písmeno g od anglického *goal*.

Myšlenka je stále podobná, avšak vrcholy budeme vybírat podle hodnoty $D(v) + \psi(v)$ místo samotného $D(v)$. Zobrazení $\psi : V \rightarrow \mathbb{R}_0^+$ se nazývá *heuristická funkce* nebo zkráceně *heuristika* a odhaduje zbývající vzdálenost z vrcholu v do cíle (viz obrázek 1.19). Součtu $D(v) + \psi(v)$ budeme říkat *f-skóre*⁹, značíme jej $f(v)$. V každé



Obrázek 1.19: Znázornění heuristické funkce ψ .

iteraci vybereme otevřený vrchol s nejnižším f -skóre.

Ne každá heuristika zachová správnost této strategie. Budeme požadovat

$$\psi(g) = 0 \quad \text{a} \quad \psi(u) \leq w(u, v) + \psi(v)$$

pro každou orientovanou hranu $(u, v) \in E$. Takové heuristice říkáme *konzistentní*. Druhá podmínka je obdobou trojúhelníkové nerovnosti: odhad z u nesmí být větší než cena jednoho kroku do v a odhad z v . Opakovaným použitím podél libovolné cesty z u do cíle dostaneme, že $\psi(u)$ nikdy nepřeceňuje skutečnou nejkratší vzdálenost do cíle.

Oproti Dijkstrovu algoritmu tedy nenastává mnoho změn, jen si navíc uchováváme hodnoty f -skóre.

1.7.1 Správnost algoritmu A*

Podobně jako u Dijkstrova algoritmu platí, že při výběru vrcholu v odpovídá $D(v)$ délce nejkratší cesty z v_0 do v . Důkaz provedeme indukcí podle pořadí uzavíraných vrcholů.

Předpokládejme pro spor, že vybíráme první vrchol v , pro který je $D(v) > d(v_0, v)$. Na jedné nejkratší cestě do v vezměme první dosud neuzavřený vrchol x a jeho již uzavřeného předchůdce y . Podle indukčního předpokladu bylo $D(y) = d(v_0, y)$, takže po zpracování hrany (y, x) platí $D(x) = d(v_0, x)$. Konzistenci můžeme opakovaně použít na část této cesty od x do v . Dostaneme

$$\psi(x) \leq d(v_0, v) - d(v_0, x) + \psi(v).$$

Proto

$$f(x) = D(x) + \psi(x) \leq d(v_0, x) + d(v_0, v) - d(v_0, x) + \psi(v) = d(v_0, v) + \psi(v) < D(v) + \psi(v) = f(v).$$

Vrchol x je otevřený a má menší f -skóre než v , takže algoritmus měl vybrat x . To je spor. Každý uzavřený vrchol tedy dostane správnou vzdálenost; zvlášť to platí pro cílový vrchol g .

⁽⁹⁾ V anglických textech se pro dosud nalezenou vzdálenost často používá označení g a pro heuristiku h , tedy $f(v) = g(v) + h(v)$.

Algoritmus 1.7.1: A*

Vstup: Nezáporně ohodnocený graf $G = (V, E, w)$, vrcholy $v_0, g \in V$ a konzistentní heuristika ψ
 // Inicializace
 1 **foreach** vrchol $v \in V$ **do**
 2 $stav(v) \leftarrow \text{nenalezený}$
 3 $D(v) \leftarrow \infty, f(v) \leftarrow \infty$
 4 $stav(v_0) \leftarrow \text{otevřený}$
 5 $D(v_0) \leftarrow 0, f(v_0) \leftarrow \psi(v_0)$
 6 **while** existují otevřené vrcholy **do**
 7 $v \leftarrow$ otevřený vrchol s nejmenším f -skóre
 8 **if** $v = g$ **then**
 9 **return** $D(g)$
 10 **foreach** soused s vrcholu v **do**
 11 // Našli jsme lepší cestu
 12 **if** $stav(s) \neq \text{uzavřený}$ a $D(v) + w(v, s) < D(s)$ **then**
 13 $D(s) \leftarrow D(v) + w(v, s)$
 14 $f(s) \leftarrow D(v) + w(v, s) + \psi(s)$
 15 $stav(s) \leftarrow \text{otevřený}$
 16 $stav(v) \leftarrow \text{uzavřený}$
 17 **return** ∞
Výstup: Vzdálenost v_0 od g



Konzistence heuristiky je pro uvedenou variantu algoritmu důležitá. Pokud by odhad tuto podmínku porušoval a algoritmus uzavřené vrcholy znovu neotevíral, mohl by některý vrchol uzavřít předčasně a vrátit neoptimální cestu.

Poznámka 1.7.1. • Dijkstrův algoritmus je speciálním případem A*. Stačí pro všechny vrcholy v položit $\psi(v) = 0$.

- Heuristiku bychom měli být schopni pro každý vrchol spočítat rychle. Ideální je konstantní čas $\mathcal{O}(1)$.

V nejhorším případě může A* prozkoumat celý graf, takže při použití binární haldy má stejný asymptotický horní odhad jako Dijkstrův algoritmus, tedy $\mathcal{O}((n + m) \log n)$. Dobrá heuristika však obvykle výrazně sníží počet skutečně zpracovaných vrcholů. Volba $\psi = 0$ neposkytne žádné směřování a dostaneme přesně Dijkstrův algoritmus.

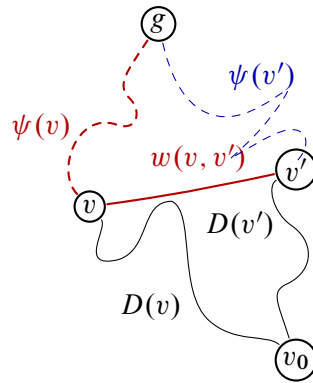
1.7.2 Ukázky heuristických funkcí

Zbývá zodpovědět otázku, jakou heuristiku použít. Odpověď záleží na konkrétní úloze. Vraťme se k bludišti, v němž se hráč pohybuje vodorovně nebo svisle o jedno pole. Políčka identifikujeme souřadnicemi (x, y) .

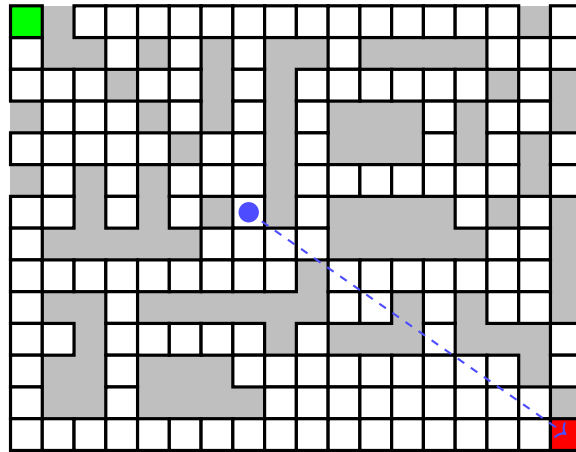
Jednou možností je tzv. *eukleidovská vzdálenost*. Ta je pro libovolnou dvojici bodů v rovině $X = (x_1, y_1)$ a $Y = (x_2, y_2)$ definována jako

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Známe-li souřadnice cílového políčka, můžeme eukleidovskou vzdálenost použít jako heuristiku ψ_E . Překážky ignorujeme, takže dostaneme dolní odhad skutečné délky cesty. Situaci lze sledovat na obrázku 1.21. Protože se v našem bludišti nelze pohybovat po diagonálách, ještě přirozenější volbou je *manhattanská vzdálenost* ψ_M



Obrázek 1.20: Situace při výběru vrcholu v při heuristice ψ , kdy $D(v)$ neodpovídá délce nejkratší cesty.

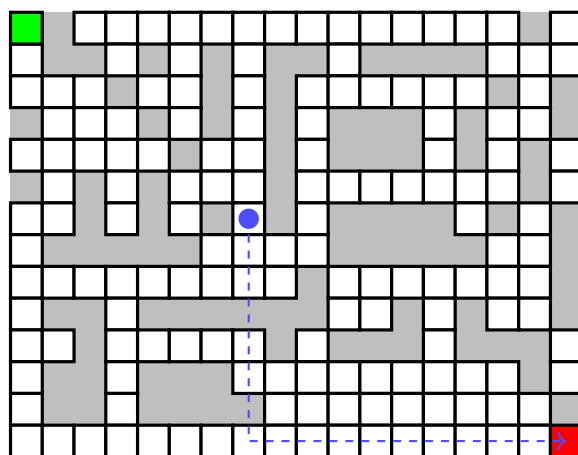


Obrázek 1.21: Příklad bludiště s eukleidovskou heuristikou.

definovaná vztahem

$$|x_1 - x_2| + |y_1 - y_2|.$$

Eukleidovská i manhattanská vzdálenost splňují trojúhelníkovou nerovnost, a proto v této mřížce dávají konzistentní heuristiky. Manhattanská vzdálenost je zde navíc přesnější: bez překážek se rovná skutečnému počtu vodorovných a svislých kroků do cíle. Její větší hodnota tedy není nevýhodou, dokud nepřeceňuje skutečnou cenu cesty. Naopak zpravidla lépe směřuje hledání.



Obrázek 1.22: Příklad bludiště s manhattanskou heuristikou.

Algoritmicky těžké problémy

Mnoho problémů lze v informatice řešit pomocí polynomiálních algoritmů, tzn. algoritmů běžících v čase $\mathcal{O}(n^k)$ pro nějaké pevné k . Například prvky v poli můžeme řadit různými algoritmy; jednoduchý *BubbleSort* má v nejhorším případě časovou složitost $\mathcal{O}(n^2)$, kde n je počet prvků pole. Také nejkratší cesty v grafu dokážeme hledat polynomiálně, třeba Dijkstrovým algoritmem v čase $\mathcal{O}(n^2 + m)$ při použití pole, kde n je počet vrcholů a m počet hran.¹ S jistou rezervou lze říci, že polynomiální algoritmy bývají prakticky použitelné.

Bohužel ne vždy je situace tak příznivá. Lze narazit na problémy, pro které žádný polynomiální algoritmus neznáme, a u některých dokonce umíme dokázat, že je v polynomiálním čase řešit nelze. Dobrým příkladem úlohy s prokazatelně exponenciálně dlouhým řešením jsou *Hanojské věže*: přesunutí n disků vyžaduje nejméně $2^n - 1$ tahů.

Nás budou zajímat ty nejdůležitější problémy z této oblasti, protože mezi nimi lze nalézt zajímavé vztahy, a posléze si uděláme menší ochutnávku z problematiky P a NP.

2.1 Problém splnitelnosti

Začneme zdánlivě možná jednoduchým problémem, který však na poli informatiky hraje dosti důležitou roli. Předpokládejme, že máme zadanou nějakou logickou formuli φ o libovolném počtu logických proměnných, označme např. $x_1, x_2, \dots, x_n$. Naší otázkou je, jestli je možné hodnoty proměnných $x_1, x_2, \dots, x_n$ nastavit tak (tj. dosadit hodnoty 0 a 1), aby byla formule splněna², tj. $\varphi(\dots) = 1$. Takovému ohodnocení budeme říkat *splňující*.

SPLNITELNOST OBECNÉ LOGICKÉ FORMULE

Vstup problému: Logická formule φ .

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nejdříve si zopakujeme logické operace a spojky. Mezi ně řadíme $\neg$, $\wedge$, $\vee$, $\implies$, $\iff$.

- **Negace** $\neg x$. Převrací logickou hodnotu x .
- **Konjunkce** $a \wedge b$. Pravdivá, jsou-li obě hodnoty operandů a, b pravdivé.
- **Disjunkce** $a \vee b$. Pravdivá, je-li alespoň jedna z hodnot operandů a, b pravdivá.

⁽¹⁾U jednoduchého neorientovaného grafu je $m \leq n(n-1)/2$, u jednoduchého orientovaného grafu $m \leq n(n-1)$. V obou případech je tedy m polynomiálně omezeno pomocí n .

⁽²⁾Může se na první pohled zdát, že se jedná o dosti teoretickou záležitost, nicméně později uvidíme, že mnoho jiných problémů má se SAT silnou spojitost.

- **Implikace** $a \implies b$. Je-li předpoklad a splněn, je splněn i závěr b . Tj. implikace je nepravdivá, pokud platí a a neplatí b .
- **Ekvivalence** $a \iff b$. a platí právě tehdy, když platí b . Tj. ekvivalence je pravdivá, jsou-li hodnoty operandů a, b současně 0 nebo 1.

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$a \implies b$	$a \iff b$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Zde konstatujeme, že ve všech příkladech bude mít negace $\neg$ vždy *nejvyšší prioritu* oproti ostatním operacím. Prioritu ostatních operací budeme stanovovat pomocí závorek.

Příklad 2.1.1. • $\varphi(x) = x \wedge \neg x \dots$ **Není splnitelná**, pro $x = 0$ i $x = 1$ je $\varphi = 0$.

- $\psi(x) = x \vee \neg x \dots$ **Je splnitelná**, a to jak pro $x = 0$, tak $x = 1$.
- $\chi(a, b, c) = ((a \wedge b) \vee \neg c) \implies (\neg a \wedge \neg c) \dots$ **Je splnitelná**, např. pro $a = 0, b = 0$ a $c = 0$.

V příkladu 2.1.1 výše se jedná o poměrně jednoduché instance, kdy jsme schopni vidět hned, zda je formule splnitelná. Pokud bychom však uvážili nějakou složitější formuli, problém se již značně zkomplikuje.



Naivní postup pro formuli o n proměnných prozkoumá až 2^n možných ohodnocení. Algoritmus zkoušející všechny možnosti je tak *exponenciální*.

Pro SAT není dodnes známý žádný algoritmus, který by jej uměl řešit pro libovolnou logickou formuli v polynomiálním čase.

2.1.1 Konjunktivní normální forma

Zkusíme se podívat na formule ve speciálním tvaru, a to tzv. *konjunktivní normální formě*, neboli zkráceně *CNF*.³

Formule v CNF se skládá z **klauzulí**, které obsahují **literály**. Literálem je buď proměnná, nebo její negace.

- Klauzule jsou ve formuli odděleny *logickou spojkou* $\wedge$
- a v každé klauzuli jsou literály odděleny *logickou spojkou* $\vee$.

$$\varphi(\dots) = \underbrace{(\dots \vee \dots \vee \dots)}_{\text{klauzule}} \wedge (\dots \vee \overbrace{x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \overbrace{\neg x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \underbrace{y}_{\text{literál}} \vee \dots) \wedge \dots$$

Příklad 2.1.2. • $\varphi(x, y, z) = \neg x \wedge (y \vee z) \dots$ **Je v CNF**. Formule φ obsahuje klauzule $\neg x$ (klauzule o jednom literálu) a $y \vee z$.

- $\psi(a, b) = a \wedge b \dots$ **Je v CNF**.
- $\chi(a, b, c, d) = a \wedge (b \vee (c \wedge d)) \dots$ **Není v CNF**, protože výraz $c \wedge d$ není literál.

Poslední formuli lze však převést do CNF: $a \wedge (b \vee c) \wedge (b \vee d)$.

⁽³⁾Z angl. *conjunctive normal form*.

Z příkladu 2.1.2 se nabízí otázka, zda je možné *libovolnou formuli* převést do CNF. Existuje pro každou formuli ekvivalentní⁴ formule v CNF? Ano.

Věta 2.1.3. Pro každou logickou formuli ψ existuje ekvivalentní formule ψ' v CNF.

Toto tvrzení lze, pochopitelně, dokázat, ale zde se tím zabývat nebudeme a zkrátka jej přijmeme jako fakt. Podstatná věc, která z toho plyne, je, že pokud je nějaká formule ψ splnitelná, pak je splnitelná i jí ekvivalentní formule ψ' v CNF a naopak pokud ψ není splnitelná, není splnitelná ani ψ' . Symbolicky

$$\psi \text{ je splnitelná} \iff \psi' \text{ je splnitelná.}$$

2.1.2 Převod do CNF pomocí tabulky

Podívejme se na způsob, jak lze převést libovolnou logickou formuli do CNF.

Máme obecnou formuli φ s proměnnými $x_1, x_2, \dots, x_n$, pro kterou budeme konstruovat ekvivalentní formuli φ' v CNF. Vycházíme z tabulky pravdivostních hodnot, přičemž pro každé **nepravdivé** ohodnocení vytvoříme *klauzuli* s n literály, kde obecně i -tý literál je

- x_i , pokud v daném ohodnocení je $x_i = 0$,
- jinak $\neg x_i$ (tj. když $x_i = 1$).

Podívejme se na úvod na příklad 2.1.4 níže.

Příklad 2.1.4. Máme formuli $\psi(x, y, z) = (x \implies y) \implies z$, pro kterou chceme najít ekvivalentní formuli ψ' v CNF. Nejprve si tedy sestavíme tabulku pravdivostních hodnot. Z tabulky můžeme vidět, že formule ψ je

x	y	z	$x \implies y$	$(x \implies y) \implies z$
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

nepravdivá pro

- $x = 0, y = 0, z = 0$,
- $x = 0, y = 1, z = 0$
- a $x = 1, y = 1, z = 0$.

Tedy celkově sestavíme 3 klauzule:

- $\left(\overbrace{x}^{x=0} \vee \underbrace{y}_{y=0} \vee \overbrace{z}^{z=0} \right),$
- $\left(x \vee \underbrace{\neg y}_{y=1} \vee z \right),$

⁽⁴⁾Formule φ a ψ jsou ekvivalentní, pokud mají při každém ohodnocení společných proměnných stejnou pravdivostní hodnotu.

- $(\neg x \vee \neg y \vee z)$.

Výsledná formule v CNF bude mít tvar:

$$\psi'(x, y, z) = (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee z).$$

Čtenář se sám může přesvědčit, že formule ψ a ψ' pro libovolné ohodnocení mají stejnou pravdivostní hodnotu.

Nabízí se otázka: „Proč tento postup funguje?“ Pokud formule v CNF není při určitém ohodnocení pravdivá, alespoň jedna její klauzule není pravdivá. Klauzule obsahující všechny proměnné je nepravdivá právě při jediném ohodnocení: všechny její literály musí být nepravdivé. Pro každé ohodnocení, při němž je původní formule nepravdivá, tedy sestojíme klauzuli, která zakáže právě toto ohodnocení.

Příklad 2.1.5. $\chi(x_1, x_2, x_3, x_4) = (x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$.

x_1	x_2	x_3	x_4	$x_1 \iff x_2$	$\neg(x_3 \iff x_4)$	$(x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$
0	0	0	0	1	0	1
0	0	0	1	1	1	1
0	0	1	0	1	1	1
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	1	1
0	1	1	0	0	1	1
0	1	1	1	0	0	0
1	0	0	0	0	0	0
1	0	0	1	0	1	1
1	0	1	0	0	1	1
1	0	1	1	0	0	0
1	1	0	0	1	0	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	0	1

$$\chi'(x_1, x_2, x_3, x_4) = (x_1 \vee \neg x_2 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3 \vee \neg x_4).$$



Problém: Generování tabulky (resp. procházení všech možných ohodnocení) nás přivádí zpět k původnímu problému, a to sice *exponenciální časové složitosti*.

Tento postup nelze obecně zachránit pouhou chytřejší implementací: pokud trváme na ekvivalentní CNF bez nových proměnných, může být výsledná formule exponenciálně delší. Například při roznásobení formule

$$(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \dots \vee (x_n \wedge y_n)$$

vzniká pro každou volbu jednoho literálu z každé dvojice samostatná klauzule, tedy celkem 2^n klauzulí.

2.1.3 Tseitinova transformace

Značně rychlejší převod do CNF navrhl sovětský matematik *Grigorij S. Cejtin*⁵. Předchozí postup trval na tom, že ve výsledku ponecháme pouze původní proměnné. Tseitinova transformace zavede pomocné proměnné a získá

⁽⁵⁾ Jeho příjmení se v anglických textech obvykle přepisuje jako *Tseitin*; odtud pochází ustálený název transformace.

formuli, která sice obecně není s původní formulí ekvivalentní nad rozšířenou množinou proměnných, ale je s ní *stejně splnitelná*.

Postup lze rozdělit do čtyř kroků.

1. Formuli rozložíme na podformule odpovídající uzlům jejího syntaktického stromu.
2. Pro každou složenou podformuli zavedeme novou proměnnou.
3. Pro každou novou proměnnou přidáme klauzule, které vynutí, aby měla stejnou hodnotu jako příslušná podformule.
4. Nakonec přidáme jednotkovou klauzuli tvořenou proměnnou zastupující celou původní formuli; tím požadujeme, aby její hodnota byla 1.

Potřebné klauzule jsou krátké a lze je zapsat přímo. Pro literály nebo pomocné proměnné a, b platí

$$y \iff \neg a \rightsquigarrow (\neg y \vee \neg a) \wedge (y \vee a), \quad (2.1)$$

$$y \iff (a \wedge b) \rightsquigarrow (\neg y \vee a) \wedge (\neg y \vee b) \wedge (y \vee \neg a \vee \neg b), \quad (2.2)$$

$$y \iff (a \vee b) \rightsquigarrow (y \vee \neg a) \wedge (y \vee \neg b) \wedge (\neg y \vee a \vee b). \quad (2.3)$$

Implikaci nejprve přepíšeme pomocí $a \implies b \equiv \neg a \vee b$. Každý výskyt spojky tak přidá nejvýše tři klauzule konstantní délky.

Pojďme se podívat na konkrétní příklad.

Příklad 2.1.6. Máme formuli $\alpha(p, q, r) = ((p \vee q) \wedge r) \implies \neg p$. Chceme pro ni sestavit Tseitinovu transformaci novou formuli β .

Formule α obsahuje tyto dílčí podformule:

- $\neg p$,
- $p \vee q$,
- $(p \vee q) \wedge r$,
- a $((p \vee q) \wedge r) \implies \neg p$ (původní formule α).

Pro ně zavedeme proměnné y_1, y_2, y_3 a y_4 . Implikaci nejprve přepíšeme jako disjunkci:

- $y_4 \iff \neg p$,
- $y_3 \iff (p \vee q)$,
- $y_2 \iff (y_3 \wedge r)$
- a $y_1 \iff (\neg y_2 \vee y_4)$.

Pomocí vztahů (2.1)–(2.3) dostaneme formuli v CNF:

$$\begin{aligned} \beta = & y_1 \\ & \wedge (y_1 \vee y_2) \wedge (y_1 \vee \neg y_4) \wedge (\neg y_1 \vee \neg y_2 \vee y_4) \\ & \wedge (\neg y_2 \vee y_3) \wedge (\neg y_2 \vee r) \wedge (y_2 \vee \neg y_3 \vee \neg r) \\ & \wedge (y_3 \vee \neg p) \wedge (y_3 \vee \neg q) \wedge (\neg y_3 \vee p \vee q) \\ & \wedge (\neg y_4 \vee \neg p) \wedge (y_4 \vee p). \end{aligned}$$

Zkusme ohodnocení $p = 1, q = 0$ a $r = 0$. Původní formule je pravdivá:

$$\alpha(1, 0, 0) = ((1 \vee 0) \wedge 0) \implies \neg 1 = 0 \implies 0 = 1.$$

Pomocné proměnné musí nabývat hodnot

- $y_4 = 0$, protože $\neg p = 0$,
- $y_3 = 1$, protože $p \vee q = 1$,
- $y_2 = 0$, protože $y_3 \wedge r = 0$,
- a $y_1 = 1$, protože $\neg y_2 \vee y_4 = 1$.

Po dosazení jsou všechny klauzule formule β splněny.

Je-li původní formule splnitelná, nastavíme každou pomocnou proměnnou podle hodnoty její podformule a splníme všechny nové klauzule. Je-li naopak výsledná formule splnitelná, klauzule vynucují správné hodnoty všech pomocných proměnných a jednotková klauzule pro kořen říká, že původní formule musí být pravdivá. Obě formule jsou tedy stejně splnitelné.



Každá spojka původní formule přidá jednu pomocnou proměnnou a nejvýše tři klauzule. Velikost výsledné formule i doba konstrukce jsou proto lineární ve velikosti původní formule.

2.2 Problém SAT

V podsekcí 2.1.1 jsme viděli, že převod obecné formule do CNF „hrubou silou“ je problematický. Nová formule se totiž může až exponenciálně prodloužit. V podsekcí 2.1.3 jsme si proto ukázali Tseitinovu transformaci, která pomocí nových proměnných sestrojí stejně splnitelnou CNF efektivně. Zkusme si tedy původní problém zjednodušit. Předpokládejme nyní, že formule na vstupu jsou již v CNF. Tento problém je v informatice velmi význačný a označuje se zkratkou SAT (z angl. *satisfiability*, neboli splnitelnost).

SPLNITELNOST LOGICKÉ FORMULE V CNF (SAT)

Vstup problému: Logická formule φ v CNF.

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nyní se zaměříme na samotnou splnitelnost vstupní formule. V praxi se používají tzv. SAT *solvery*, které rozhodují, zda je formule v CNF splnitelná. Ukážeme si klasický algoritmus *DPLL*, jehož název připomíná příjmení Martina Davise, Hilaryho Putnama, George Logemanna a Donalda Lovelanda. Jeho základní podoba pochází z počátku 60. let 20. století.

2.2.1 Algoritmus DPLL

Tento algoritmus je založený na rekurzivním postupu. Na vstupu algoritmu je libovolná formule φ v CNF, kterou algoritmus postupně vyhodnocuje. Využívá dvojici procedur – **Unit propagation** a **Pure literal elimination**.

Unit propagation

Tato procedura hledá v dané formuli φ tzv. *jednotkovou klauzuli*. To je klauzule, která obsahuje **pouze jeden literál**. Například formule ω níže obsahuje jednotkovou klauzuli.

$$\omega(x, y) = (x \vee \neg y) \wedge \underbrace{\neg y}_{\text{jednotková kl.}} \wedge (\neg x \vee y).$$

Pokud se taková klauzule nachází ve formuli, lze toho využít, neboť pro danou proměnnou existuje právě jedno ohodnocení, které tuto klauzuli splní. Obecně pokud formule φ obsahuje jednotkovou klauzuli s proměnnou x , pak mohou nastat dva případy. Označme φ' zbytek klauzulí ve formuli φ .

1. Pokud má formule tvar $\varphi = x \wedge \varphi'$, pak nutně $x = 1$.
2. Jinak pokud má formule tvar $\varphi = \neg x \wedge \varphi'$, pak musí být $x = 0$.

Fakt, že jsme schopni hned rozhodnout, jaké ohodnocení musí mít daná proměnná, je v tomto případě hodně ku prospěchu, neboť formuli můžeme zjednodušit. Protože ohodnocení dané jednotkové klauzule již známe, můžeme ji z formule φ odstranit. Zároveň můžeme **odstranit všechny klauzule**, které se díky proměnné x vyhodnotí na 1, neboť již nemohou nijak ovlivnit logickou hodnotu zbylých klauzulí.

Např. pokud formule obsahuje literál x v jednotkové klauzuli, pak hodnota proměnné musí být 1. Tzn. klauzule obsahující x se vyhodnotí též na 1. tj.

$$(\dots \vee x \vee \dots) = 1.$$

Naopak pokud obsahuje literál $\neg x$, pak nutně $x = 0$ a tedy

$$(\dots \vee \neg x \vee \dots) = 1.$$

Dále ještě odstraníme výskyty literálu x , resp. $\neg x$ v *opačné polaritě*, tedy negaci původního literálu. Tzn. pokud literál v jednotkové klauzuli byl x , pak opačná polarita literálu je $\neg x$ a naopak. Důvod, proč jej můžeme odstranit, je ten, že v rámci kterékoliv jiné klauzule se daný literál v opačné polaritě vyhodnotí vždy na 0. Protože je však spojen s ostatními literály pomocí $\vee$, nemůže nijak ovlivnit výslednou logickou hodnotu klauzule a tudíž jej můžeme odstranit. Tedy např. pokud $x = 1$, pak můžeme klauzule tvaru

$$(\dots \vee y \vee \underbrace{\neg x}_{\text{odstraníme}} \vee z \vee \dots)$$

zjednodušit na

$$(\dots \vee y \vee z \vee \dots).$$

Podobně pro $x = 0$:

$$(\dots \vee y \vee x \vee z \vee \dots) = (\dots \vee y \vee z \vee \dots).$$

Příklad 2.2.1. Máme formuli

$$\chi(x, y, z) = (x \vee y) \wedge y \wedge (z \vee \neg y) \wedge (\neg x \vee \neg y \vee \neg z).$$

Aplikujme na ni proceduru *Unit propagation*. Můžeme si všimnout, že klauzule y je jednotková a proměnná y tedy musí být nastavena na 1.

Protože $y = 1$, je zároveň splněna i klauzule $x \vee y$, takže ji můžeme odstranit. Zároveň odstraníme literál $\neg y$ ze zbývajících klauzulí $z \vee \neg y$ a $\neg x \vee \neg y \vee \neg z$. Celkově dostaneme novou zjednodušenou formuli

$$\chi'(x, y, z) = z \wedge (\neg x \vee \neg z).$$

Pure literal elimination

Ve vstupní formuli φ může nastat situace, kdy se zde určitá proměnná nachází pouze v jedné polaritě. Tzn. pokud se ve formuli nachází proměnná x , pak se v ní nikde nenachází $\neg x$ a naopak pokud se v ní nachází $\neg x$, pak v ní nikde není x . V obou případech můžeme rovnou určit hodnotu dané proměnné, neboť tím splníme všechny klauzule, v nichž se vyskytuje. Ty můžeme odstranit, neboť jsou tím pádem splněny.

$$\dots \wedge \underbrace{(\dots \vee x \vee \dots)}_1 \wedge \dots \wedge \underbrace{(\dots \vee x \vee \dots)}_1 \wedge \dots$$

Podobně pro negaci, tj.

$$\dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots$$

Poté, co jsme si vysvětlili základní dvojici procedur, se nyní nabízí trojice případů, které je ještě třeba prozkoumat.

- *Co když postupným odstraňováním proměnných mi ve formuli vznikne prázdná klauzule?* Taková situace může nastat v případě procedury Unit propagation, kdy odstraňujeme proměnné v opačné polaritě. Pokud nastane situace, že je nějaká klauzule prázdná, pak to znamená, že daná klauzule pod zvoleným ohodnocením **není splnitelná** a tudíž ani celá formule. V takovou chvíli můžeme prohlásit formuli za **nesplnitelnou pod daným ohodnocením**⁶.
- *Co když odstraníme postupně všechny klauzule?* K takové situaci může dojít, pokud jsou všechny klauzule splněny. V takovou chvíli určitě víme, že je původní formule **splnitelná**.
- *Co když po aplikaci obou procedur stále nevíme, zda je φ splnitelná?* V takovou chvíli nám nezbyvá nic jiného než si zvolit nějakou dosud neohodnocenou proměnnou a prozkoumat obě možnosti jejího ohodnocení. Vybereme tedy proměnnou x a zkusíme ji nastavit jednou na hodnotu 1 a podruhé na hodnotu 0. Tím můžeme formuli φ zjednodušit a na nově vzniklou formuli φ' rekurzivně aplikovat stejný algoritmus. Pokud lze splnit zbytek formule φ' , pak lze splnit i původní formuli φ .

Toho můžeme dosáhnout tak, že k formuli φ přidáme uměle jednotkovou klauzuli x , resp. v druhém případě $\neg x$. Tuto jednotkovou klauzuli si pak vybere procedura Unit propagation, která proměnnou ohodnotí. Zavoláme tedy algoritmus DPLL rekurzivně na formule

$$\varphi \wedge x \quad \text{a} \quad \varphi \wedge \neg x.$$

S těmito informacemi můžeme dát dohromady pseudokód algoritmu.

Algoritmus 2.2.1: DPLL

Vstup: Formule φ v CNF

```

1 while existuje jednotková klauzule ( $\ell$ ) ve formuli  $\varphi$  do
2    $\varphi \leftarrow \text{UnitPropagation}(\ell, \varphi)$ 
3 while existuje literál  $\ell$  v jediné polaritě ve  $\varphi$  do
4    $\varphi \leftarrow \text{PureLiteralElimination}(\ell, \varphi)$ 
   // Všechny klauzule jsou pravdivé
5 if je  $\varphi$  prázdná then return 1;
   // Všechny literály v klauzuli jsou nepravdivé
6 if obsahuje  $\varphi$  prázdnou klauzuli then return 0;
   // Zkusíme obě ohodnocení proměnné  $x$ 
7 Vyber dosud neohodnocenou proměnnou  $x$ 
8 return DPLL( $\varphi \wedge x$ )  $\vee$  DPLL( $\varphi \wedge \neg x$ )
```

Výstup: 1, je-li formule φ splnitelná, jinak 0.

Zkusme si algoritmus odsimulovat na příkladu.

Příklad 2.2.2. Máme formuli

$$\gamma(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge \neg x_4 \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

⁽⁶⁾ Je potřeba si uvědomit, že ohodnocení proměnných si během výpočtu volíme, tedy v takovém případě to ještě nutně neznamená, že je formule celkově nesplnitelná. Pouze víme, že dané ohodnocení určitě není splňující

Chceme pomocí algoritmu DPLL rozhodnout, zda je splnitelná.

Když se podíváme na pseudokód 2.2.1 výše, tak první máme aplikovat proceduru *Unit propagation*. Hledáme tedy jednotkové klauzule ve formuli γ , dokud nějaké existují.

- První jednotková klauzule, které si můžeme všimnout, je $\neg x_4$. Hodnota proměnné x_4 tedy nutně musí být 0. Klauzuli tedy odstraníme. Proměnná se nachází ve formuli pouze jednou, tedy daný literál v opačné polaritě nikde odstraňovat nemusíme. Máme nyní formuli

$$\gamma'(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

- Další jednotková klauzule je x_1 . Proměnná x_1 musí být nutně 1 a klauzuli můžeme odstranit. Dále si však můžeme všimnout, že tím pádem splněna i klauzule $(x_1 \vee x_3)$. Tu můžeme též odstranit. Literál x_1 se dále nachází v opačné polaritě v klauzuli $(\neg x_1 \vee \neg x_3 \vee x_3)$. Literál $\neg x_1$ můžeme také odstranit z formule. Tím se nám formule γ' zjednoduší na

$$\gamma''(x_1, x_2, x_3, x_4) = (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_3).$$

Nově vzniklá formule γ'' již žádné další jednotkové klauzule neobsahuje. Dále tedy můžeme aplikovat proceduru *Pure literal elimination*.

- Formule γ'' obsahuje pouze jeden literál v jediné polaritě, a to $\neg x_2$. Tím pádem hodnotu x_2 můžeme nastavit na 0, čímž splníme dvojici klauzulí $(\neg x_2 \vee \neg x_3)$ a $(\neg x_2 \vee x_3)$. Ty můžeme odstranit, čímž dostaneme formuli

$$\gamma'''(x_1, x_2, x_3, x_4) = \neg x_3 \vee x_3.$$

Protože formule γ''' není ani prázdná, ani neobsahuje prázdnou klauzuli, vybereme náhodně jeden literál a prozkoumáme jeho obě možná ohodnocení. V tomto případě nám zbývají pouze literály x_3 a $\neg x_3$. Vyberme např. literál $\neg x_3$. Nyní máme sestavit dvojici nových formulí $(\neg x_3 \vee x_3) \wedge \neg x_3$ a $(\neg x_3 \vee x_3) \wedge x_3$, na které rekurzivně zavoláme DPLL.

- **Formule** $(\neg x_3 \vee x_3) \wedge \neg x_3$. Opět začneme aplikací Unit propagation. K původní formuli γ''' jsme uměle přidali jednotkovou klauzuli $\neg x_3$. Z ní je zjevné, že x_3 musí být 0. Tím zároveň splníme i klauzuli $(\neg x_3 \vee x_3)$, kterou můžeme odstranit, čímž dostáváme prázdnou formuli. Pure literal elimination aplikovat nemůžeme. Protože je nově vzniklá formule prázdná, vrátíme na výstup 1.
- **Formule** $(\neg x_3 \vee x_3) \wedge x_3$. I zde začneme eliminací jednotkových klauzulí. Tu zde představuje literál x_3 , jehož ohodnocení tedy musí být 1. Tím opět splníme i klauzuli $(\neg x_3 \vee x_3)$, jejímž odstraněním dostaneme zase prázdnou formuli. Tedy opět vrátíme 1.

Obě ohodnocení pro x_3 jsou tedy splňující a tedy i původní formule γ je *splnitelná*.

Ukažme si ještě jeden jednodušší příklad.

Příklad 2.2.3. Mějme formuli $\tau(x, y, z) = x \wedge (\neg x \vee y \vee z) \wedge \neg x$. Chceme opět pomocí DPLL určit splnitelnost.

Jako první jednotkovou klauzuli můžeme vzít x . Tedy můžeme ji odstranit společně se všemi výskyty daného literálu v opačné polaritě, tj. $\neg x$. Tzn. dostaneme formuli

$$\tau'(x, y, z) = (y \vee z) \wedge ().$$

Všimněte si, že odstraněním literálu $\neg x$ jsme dostali ve formuli prázdnou klauzuli⁷ (). Aplikací Pure literal elimination se zbavíme i literálů y a z .

$$\tau''(x, y, z) = ()$$

Formule τ'' obsahuje pouze prázdnou klauzuli. V tuto chvíli algoritmus prohlásí formuli τ za *nesplnitelnou*.

⁽⁷⁾ Z jednotkových klauzulí jsme odebrali její jediný literál v rámci procedury Unit propagation; klauzule samotná ve formuli zůstala.

Poslední otázka, která se nabízí, je (jako obvykle) časová složitost algoritmu. Vezmeme-li v potaz nejhorší případ, časová složitost bohužel nevychází úplně příznivě.

Věta 2.2.4 (Složitost DPLL). Nechť formule φ obsahuje v proměnných a její délka je s . Jednoduchá implementace DPLL doběhne v čase $\mathcal{O}(s2^v)$. Při kopírování formule v každé úrovni rekurze spotřebuje nejvýše $\mathcal{O}(sv)$ pomocné paměti.

Důkaz. V nejhorším případě nepomůže ani jednotková propagace, ani eliminace čistých literálů. Algoritmus pak zvolí proměnnou a vytvoří dvě větve, jednu pro hodnotu 0 a druhou pro hodnotu 1. Hloubka stromu je nejvýše v a počet jeho uzlů

$$1 + 2 + 4 + \dots + 2^v = 2^{v+1} - 1 = \mathcal{O}(2^v).$$

V každém uzlu lze naivně projít celou formuli v čase $\mathcal{O}(s)$, odkud plyne čas $\mathcal{O}(s2^v)$. Rekurse probíhá do hloubky, takže současně uchováváme nejvýše v kopií formule délky nejvýše s ; to dává $\mathcal{O}(sv)$ pomocné paměti. $\square$



Praktické SAT solvery formulí při každém volání celou nekopírují. Změny zapisují do pomocných struktur a při návratu je vracejí zpět, takže paměťový odhad lze podstatně zlepšit.

Se exponenciální časovou složitostí se sotva spokojíme, avšak v tomto případě je dobré si říci, že zkoumáme složitost v nejhorším případě a v praxi bývají zkoumané logické formule poměrně rychle řešitelné pomocí DPLL.



Problém je však dodnes ten, že na rozhodnutí splnitelnosti logické formule (i těch v CNF) není známý žádný algoritmus, který by běžel v polynomiálním čase, tj. $\mathcal{O}(n^k)$, pro nějaké k . K tomuto tématu se ještě vrátíme v sekci 2.6.

2.3 Problém 3-SAT

Zkusme nyní původní problém ještě omezit. Přidáme podmínku, že každá klauzule může obsahovat nejvýše tři literály. Tím dostáváme variantu problému SAT známou jako 3-SAT. O formuli, která splňuje uvedenou podmínku, budeme říkat, že je v 3-CNF.

SPLNITELNOST LOGICKÉ FORMULE V 3-CNF (3-SAT)

Vstup problému: Formule φ v 3-CNF

Výstup problému: 1, je-li φ splnitelná, jinak 0.

Nabízí se otázka, zda jsme si skutečně pomohli. Zdánlivě ano, protože máme kratší klauzule. Překvapivě však 3-SAT zůstává stejně obtížný v tom smyslu, že SAT lze na 3-SAT převést v polynomiálním čase. Pro libovolnou instanci SAT sestrojíme instanci 3-SAT, která je splnitelná právě tehdy, když byla splnitelná původní formule.

2.3.1 Převod SAT na 3-SAT

Popíšeme si nyní obecný způsob, jak převést libovolnou formuli v CNF (instance problému SAT) na formuli v 3-CNF (instance problému 3-SAT). Předpokládejme, že máme zadanou nějakou formuli φ v CNF a chceme pro ni sestřit formuli ψ v 3-CNF, takovou, aby byla splnitelná právě tehdy, když je splnitelná i původní formule φ . Pro formuli φ mohou nastat 2 případy:

1. formule obsahuje klauzule délky 3 a kratší,
2. nebo ve formuli existuje klauzule délky $k > 3$.

V prvním případě není co řešit, neboť φ již je v 3-CNF, tedy stačí položit $\psi = \varphi$ a jsme hotovi. V druhém případě musíme dlouhou klauzuli o k literálech rozložit na kratší. Tuto klauzuli⁸ rozložíme na dvojici nových klauzulí pomocí nové proměnné x . Označme si literály dané dlouhé klauzule $\ell_1, \ell_2, \dots, \ell_k$. První dvojici ℓ_1 a ℓ_2 si označíme α a zbytek β .

$$\underbrace{\ell_1 \vee \ell_2}_{\alpha} \vee \overbrace{\ell_3 \vee \dots \vee \ell_k}^{\beta} = \alpha \vee \beta$$

Nyní přidáme novou proměnnou x a klauzuli $\alpha \vee \beta$ rozdělíme na dvojici klauzulí následovně:

$$(\alpha \vee x) \wedge (\beta \vee \neg x).$$

Podívejme se nyní na délky nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

- Klauzule $\alpha \vee x = \ell_1 \vee \ell_2 \vee x$ obsahuje 3 literály, takže s ní dále nic provádět nemusíme.
- Klauzule $\beta \vee \neg x = \ell_3 \vee \dots \vee \ell_k \vee \neg x$ obsahuje $k - 1$ literálů. To znamená, že jsme zkrátily původní klauzuli o jeden literál. Pokud $k - 1 > 3$, můžeme postup pro tuto klauzuli opakovat.

Nyní zbývá dořešit, jak to bude s hodnotou proměnné x . Podobně jako u Tseitinovy transformace (viz podsekcce 2.1.3), i zde bude její hodnota záviset na hodnotách ostatních proměnných, konkrétně výrazu α .

- Pokud $\alpha = 1$, pak nastavíme $x = 0$. Původní klauzule $\ell_1 \vee \dots \vee \ell_k = \alpha \vee \beta$ je splněna díky α . Nová dvojice klauzulí je pak splněna pomocí α a x :

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (1 \vee 0) \wedge (\beta \vee 1) = 1 \wedge 1 = 1.$$

- Pokud $\alpha = 0$, pak nastavíme $x = 1$. Klauzule $\alpha \vee \beta$ musí být splněna díky β . Potom platí:

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (1 \vee 0) = 1 \wedge 1 = 1.$$

Pokud by však $\beta = 0$, pak by původní klauzule $\alpha \vee \beta$ nebyla splněna a potom

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (0 \vee 0) = 1 \wedge 0 = 0,$$

tzn. ani nová dvojice klauzulí by nebyla splněna, což chceme.

Platí i opačný směr: kdyby byla $\alpha \vee \beta$ nepravdivá, muselo by být $\alpha = \beta = 0$. Nové klauzule by pak vyžadovaly současně $x = 1$ a $\neg x = 1$, což není možné. Nová dvojice je tedy splnitelná právě tehdy, když je splnitelná původní klauzule. Tím jsme dostali obecný postup pro převod formule z CNF do 3-CNF.

Příklad 2.3.1. Máme formuli

$$\varphi(a, b, c, d, e, f) = a \vee b \vee \neg c \vee d \vee \neg e \vee f$$

o jedné klauzuli, kterou chceme převést do 3-CNF. K tomu si pořídíme novou proměnnou x . Podle postupu popsaného výše sestavíme dvojici nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

$$\underbrace{a \vee b}_{\alpha} \vee \overbrace{\neg c \vee d \vee \neg e \vee f}^{\beta}$$

⁽⁸⁾ Pokud φ obsahuje více dlouhých klauzulí, postup opakujeme pro každou z nich zvlášť.

Tím dostáváme novou formuli

$$\varphi' = (a \vee b \vee x) \wedge (\neg c \vee d \vee \neg e \vee f \vee \neg x).$$

První klauzule $a \vee b \vee x$ obsahuje tři literály, avšak druhá klauzule je stále dlouhá (všimněte si ale, že obsahuje o jeden literál méně než původní klauzule). Postup tedy opakujeme.

Vytvoříme si opět novou proměnnou, označme ji y . Podobně jako výše i zde rozložíme danou klauzuli na dvě kratší.

$$\underbrace{(\neg c \vee d \vee y)}_{\alpha'} \wedge \underbrace{(\neg e \vee f \vee \neg x \vee \neg y)}_{\beta'}.$$

Tedy dostáváme formuli

$$\varphi'' = (a \vee b \vee x) \wedge (\neg c \vee d \vee y) \wedge (\neg e \vee f \vee \neg x \vee \neg y).$$

Třetí klauzule je stále ještě dlouhá, tedy postup zopakujeme ještě jednou s novou proměnnou z .

$$\underbrace{(\neg e \vee f \vee z)}_{\alpha''} \wedge \underbrace{(\neg x \vee \neg y \vee \neg z)}_{\beta''}.$$

Celkově dostáváme formuli, která je již v 3-CNF:

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee d \vee y) \wedge (\neg e \vee f \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Nyní si zkusíme určit hodnoty pomocných proměnných x, y, z na základě zadaného ohodnocení původních proměnných. Zkusme

$$a = 0, \quad b = 0, \quad c = 0, \quad d = 0, \quad e = 1, \quad f = 0.$$

Původní formule je díky literálu $\neg c$ splněna.

- Protože $a \vee b = 0$, položíme $x = 1$.
- Dále máme $\neg c \vee d = 1$, tedy položíme $y = 0$.
- Nakonec $\neg e \vee f = 0$, a proto položíme $z = 1$.

Můžeme se přesvědčit, že nová formule ψ je v tomto ohodnocení také splněna:

$$\begin{aligned} \psi &= (0 \vee 0 \vee 1) \wedge (1 \vee 0 \vee 0) \wedge (0 \vee 0 \vee 1) \wedge (0 \vee 1 \vee 0) \\ &= 1 \wedge 1 \wedge 1 \wedge 1 \\ &= 1. \end{aligned}$$

Tímto způsobem jsme ukázali, že problém SAT lze “převést” na 3-SAT. Opačný směr je triviální, protože každá formule v 3-CNF už je i v CNF, takže nic převádět nemusíme.

Zkusme se zamyslet nad rychlostí převodu. Klauzuli o $k > 3$ literálech nahradíme $k - 2$ klauzulemi a použijeme $k - 3$ nových proměnných. Celkový počet vytvořených symbolů je tedy lineární v délce původní formule.

Z toho plyne, že kdybychom uměli řešit 3-SAT v polynomiálním čase, uměli bychom polynomiálně řešit i SAT: nejprve bychom lineárně sestrojili formuli v 3-CNF a poté na ni spustili předpokládaný polynomiální algoritmus pro 3-SAT. Samotný převod je lineární; *to však neznamená, že lineárně umíme rozhodnout splnitelnost výsledné formule.*

Toto byla ukázka tzv. **polynomiálního převodu** mezi problémy. Pojďme si toto trochu rigorózněji definovat.

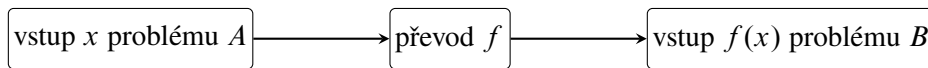
Definice 2.3.2 (Polynomiální převoditelnost). Řekneme, že libovolný problém A je polynomiálně převoditelný na problém B , píšeme $A \rightarrow B$, pokud každý vstup x problému A lze převést v polynomiálním čase na vstup x' problému B , přičemž $A(x) = B(x')$.

Co tato definice říká? Pokud platí $A \rightarrow B$, pak odpověď na vstup x' je zároveň i odpovědí na původní vstup x . V našem případě je formule v CNF φ splnitelná (to je naše x), právě když je nová formule v 3-CNF ψ (to je x') splnitelná.

Zároveň $A \rightarrow B$ znamená, že problém B je alespoň tak těžký jako problém A .⁹



Ze směru převodu plyne následující podmíněné tvrzení: pokud $A \rightarrow B$ a problém B lze řešit polynomiálně, pak lze polynomiálně řešit i A . Obrácený závěr obecně neplatí. Pokud by navíc bylo dokázáno, že A polynomiálně řešit nelze, pak by z $A \rightarrow B$ plynulo, že polynomiálně nelze řešit ani B . U SAT však takový dolní odhad zatím dokázán není.



$$x \in A \iff f(x) \in B$$

Obrázek 2.1: Schéma polynomiálního převodu.

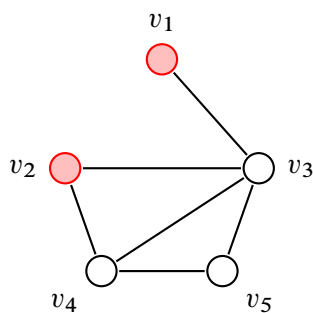
2.4 Problém nezávislé množiny v grafu

Od problému SAT, resp. 3-SAT se na chvíli přesuneme ke zdánlivě odlišnému problému z teorie grafů. Později si ukážeme i jeho praktické použití v podsekcí 2.4.2. Nejdříve však bude dobré začít definicí.

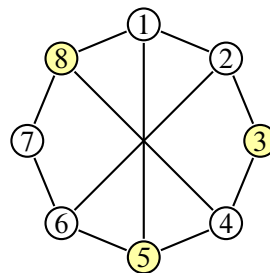
Definice 2.4.1 (Nezávislá množina). Mějme libovolný neorientovaný graf $G = (V, E)$. Množina $N \subseteq V$ je *nezávislá*, pokud žádné dva různé vrcholy z N nejsou spojeny hranou.

Definice 2.4.1 jednoduše říká, že jakákoliv vybraná množina vrcholů z daného grafu je nezávislá, pokud mezi všemi vybranými vrcholy nevede hrana. Podívejme se na konkrétní příklad.

Příklad 2.4.2. Nezávislé množiny M_1 a M_2 v grafech G_1 a G_2 (viz obrázky 2.2a a 2.2b).



(a) Nezávislá množina vrcholů $M_1 = \{v_1, v_2\}$ v G_1



(b) Nezávislá množina vrcholů $M_2 = \{3, 5, 8\}$ v G_2

⁽⁹⁾ Mnemotechnická pomůcka: „Obtížnost teče ve směru šipky.“

Zde poznamenejme, že na pouhou existenci neprázdné nezávislé množiny nemá smysl se ptát, neboť v libovolném neprázdném grafu existuje nezávislá množina: libovolná jednovrcholová množina je z definice nezávislá. Zajímavější otázkou je spíše, zda v zadaném grafu existuje nezávislá množina, která má alespoň určitou velikost (tzn. počet vrcholů).

NEZÁVISLÁ MNOŽINA V GRAFU (NzMna)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v grafu G existuje nezávislá množina velikosti alespoň k , jinak 0.

Jak později uvidíme, otázka existence nezávislé má i své aplikace při řešení praktických problémů (opět viz podsekcce 2.4.2), avšak ještě než tak učiníme, zkusme se podívat na souvislosti s problémy, které již známe.

2.4.1 Převod 3-SAT na NzMna

Mezi dvojicí problémů NzMna a 3-SAT existuje přímá souvislost oběma směry. Problém 3-SAT lze tedy převést na problém nezávislé množiny a naopak. Pro účely tohoto textu si však ukážeme pouze převod jedním směrem, ten druhý je již podstatně složitější¹⁰.

Začínáme s formulí φ , která je ve formě 3-CNF. Uvažujme, že obsahuje obecně k klauzulí, kde každá obsahuje maximálně 3 literály. Chceme pro tuto formuli sestavit graf G a číslo n takové, že v něm existuje nezávislá množina velikosti alespoň n .

$$\varphi = \underbrace{(\dots)}_{\text{max. 3 literály}} \wedge (\dots) \wedge \dots \wedge (\dots)$$

Aby byla formule φ splněná, musí být splněny všechny její klauzule. K tomu je potřeba, aby v každé klauzuli existoval alespoň jeden literál, který ji bude splňovat (může jich být i více). Z každé klauzule tedy vybereme jeden literál, který ji bude splňovat. Je však nutné upozornit, že nesmíme vybírat konfliktně, tedy např. nelze v jedné klauzuli vybrat jako splňující literál x a v jiné klauzuli $\neg x$.

$$(\dots) \wedge (\dots \vee \underbrace{x}_{\text{vybereme}} \vee \dots) \wedge \dots \wedge (\dots \vee \overbrace{\neg x}^{\text{nesmíme vybrat}} \vee \dots) \wedge \dots$$

Pro každý výskyt literálu vytvoříme v novém grafu jeden vrchol. Vrcholy náležející stejné klauzuli navzájem všechny spojíme; pro tříliterálovou klauzuli tak vznikne trojúhelník, pro kratší klauzuli hrana nebo jediný vrchol. Nakonec spojíme hranou každý výskyt literálu x s každým výskytem opačného literálu $\neg x$ v jiné klauzuli. Například pro formuli

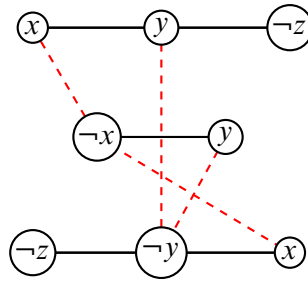
$$\psi(x, y, z) = (x \vee y \vee \neg z) \wedge (\neg x \vee y) \wedge (\neg z \vee \neg y \vee x)$$

je příslušný graf znázorněný na obrázku 2.3.

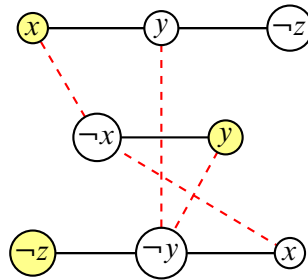
Pojďme nyní ověřit obě strany konstrukce. Je-li formule splnitelná, vybereme z každé klauzule jeden pravdivý literál. Získané vrcholy pocházejí z různých klauzulí a nemohou obsahovat současně x i $\neg x$, takže mezi nimi nevede žádná hrana. Tvoří proto nezávislou množinu velikosti k .

Naopak nechť graf obsahuje nezávislou množinu velikosti k . Z každé z k klauzulí může obsahovat nejvýše jeden vrchol, protože vrcholy jedné klauzule jsou navzájem spojené. Musí tedy obsahovat právě jeden vrchol z každé klauzule. Vybrané literály si navzájem neodporují, jinak by jejich vrcholy byly spojeny hranou. Nastavíme proměnné tak, aby byly všechny vybrané literály pravdivé; tím splníme každou klauzuli.

⁽¹⁰⁾ Pro zájemce viz kniha *Průvodce labyrintem algoritmů* (2. vydání), str. 471

Obrázek 2.3: Ukázka sestrojeného grafu pro formuli ψ .

Pro již zmíněnou formuli ψ , která je splnitelná např. pomocí ohodnocení $x = 1, y = 1$ a $z = 0$, je příslušná nezávislá množina znázorněna na obrázku 2.4. Vybrané splňující literály jednotlivých klauzulí jsou x , y a $\neg z$.

Obrázek 2.4: Ukázka nezávislé množiny příslušné splňujícím literálům pro formuli ψ .

Jako poslední musíme určit cílové číslo n . Protože formule obsahuje k klauzulí a z každé vybíráme právě jeden literál, položíme $n = k$.

K převodu NzMna na 3-SAT přidáme jen krátký komentář. Idea převodu je využití klasického SAT. Zadaný graf G a číslo k se převede na formuli v CNF a ta se následně převede do 3-CNF. Problém SAT tak využijeme v jistém smyslu jako “prostředníka” (viz obrázek 2.5).

Obrázek 2.5: Idea převodu NzMna $\rightarrow$ 3-SAT.

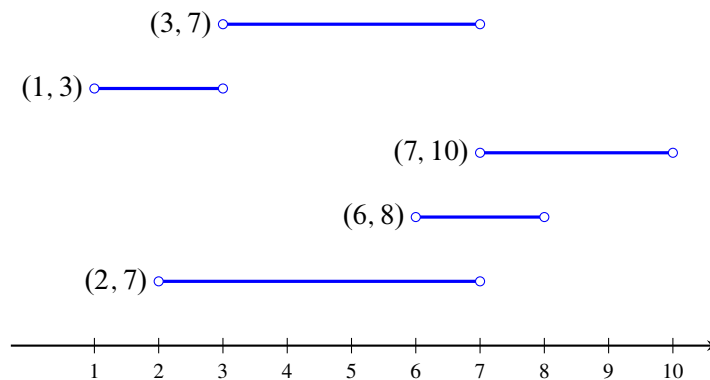
2.4.2 Aplikace NzMna – barvení intervalového grafu

Nyní se podíváme na jednu z aplikací nezávislé množiny v praktických problémech.

Představme si, že máme n vyučovacích hodin ve škole, které chceme rozvrhnout do co nejmenšího počtu učeben tak, aby se žádné dvě z nich nepřekrývaly. Každá vyučovací hodina má čas začátku a čas svého konce. Matematicky toto můžeme reprezentovat pomocí intervalu (a, b) , kde a je čas začátku a b je čas konce. Jednotlivé hodiny tedy můžeme reprezentovat pomocí intervalů $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_n, b_n)$. V tomto případě záměrně volíme otevřené intervaly. Později uvidíme, proč se nám toto hodí.

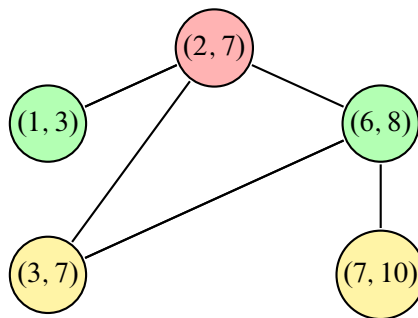
Dané hodiny tedy chceme rozdělit do co nejmenšího počtu učeben. Každou z učeben si můžeme označit nějakou barvou. Nyní sestojíme tzv. **intervalový graf**. Jeho vrcholy představují dané vyučovací hodiny, resp. příslušné intervaly, přičemž mezi dvojicí vrcholů vede hrana, právě když dané intervaly mají neprázdný průnik, tj. obecně $(a_i, b_i) \cap (a_j, b_j) \neq \emptyset$ (to znamená, že se hodiny svým časem kryjí). Zde si všimněme, proč se nám hodí pracovat

s otevřenými intervaly. Pokud by dvojice hodin navazovala těsně po sobě ve stejné učebně, např. $(7, 9)$ a $(9, 10)$, pak bychom takovou situaci také mohli shledat přípustnou. Pokud bychom však používali uzavřené intervaly, pak by průnik vycházel neprázdný. Např. viz obrázek 2.6. Nyní každý vrchol obarvíme barvou nějaké učebny. Přitom



Obrázek 2.6: Příklad intervalového grafu pro intervaly $(2, 7)$, $(6, 8)$, $(7, 10)$, $(1, 3)$, $(3, 7)$.

dvojice vyučovacích hodin nesmí být rozvržena do stejné učebny, pokud se časově kryjí. To znamená, že sousední vrcholy musí mít vždy různé barvy (viz obrázek 2.7). Zde si všimněte, že libovolná skupina vrcholů obarvená



Obrázek 2.7: Obarvení intervalového grafu pro intervaly $(2, 7)$, $(6, 8)$, $(7, 10)$, $(1, 3)$, $(3, 7)$.

stejnou barvou tvoří nezávislou množinu.

V praxi je efektivní rozvrhování běžnou záležitostí, například při přidělování úloh procesorům.



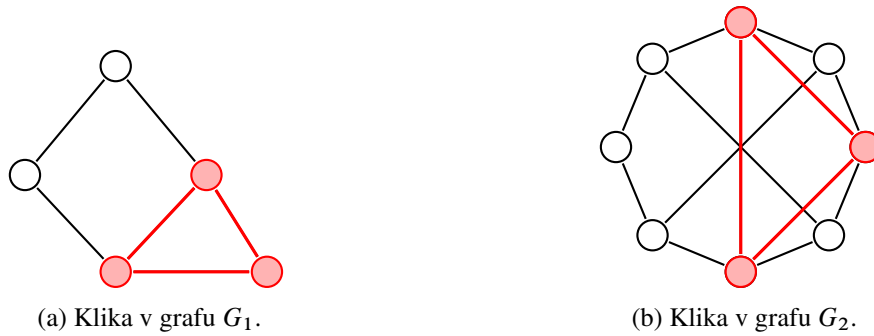
Barvení obecného grafu je algoritmicky těžké, ale intervalové grafy mají zvláštní strukturu. Minimální počet učeben lze určit polynomiálním hladovým algoritmem: hodiny zpracujeme podle času začátku a použijeme nejdříve uvolněnou dostupnou učebnu; není-li žádná volná, otevřeme novou. Tento příklad tedy ukazuje, že omezená varianta obecně těžkého problému může být naopak dobře řešitelná.

2.5 Problém kliky v grafu

Jako poslední si ukážeme ještě jeden typ problému, který (jak uvidíme) přímo souvisí s problémem nezávislé množiny. Nejdříve si však připomeňme některé důležité termíny z teorie grafů. Jako první si definujeme tzv. *kliku*.

Definice 2.5.1 (Klika). Nechť je dán neorientovaný graf $G = (V, E)$. Množinu $K \subseteq V$ nazveme *klikou*, pokud jsou každé dva různé vrcholy z K spojeny hranou.

Příklad 2.5.2. Příklady klik v grafech G_1 a G_2 , viz obrázky 2.8a a 2.8b.



Podobně jako u nezávislé množiny, i zde platí, že každý neprázdný graf jistě obsahuje kliku (např. samostatný vrchol je z definice kliky). Bude nás tedy opět zajímat, zda v zadaném grafu existuje kliky určité velikosti.

PROBLÉM KLIKY V GRAFU (Klika)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v G existuje kliky velikosti alespoň k , jinak 0.

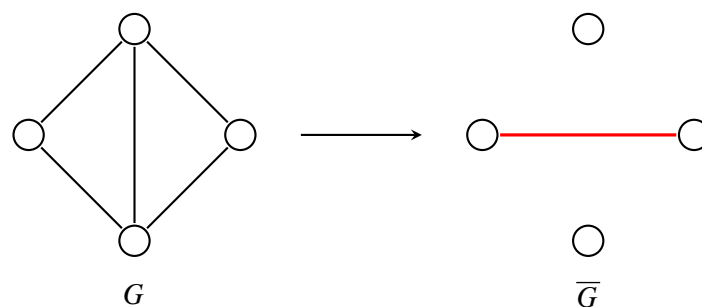
Nyní si ukážeme, že ve skutečnosti problém kliky je ekvivalentní s problémem nezávislé množiny. K tomu si však definujeme ještě jeden termín.

Definice 2.5.3 (Doplněk grafu). Je-li $G = (V, E)$ jednoduchý neorientovaný graf, pak jeho *doplněk* je graf $\overline{G} = (V, \overline{E})$, kde pro každé dva různé vrcholy $u, v \in V$ platí

$$\{u, v\} \in \overline{E} \iff \{u, v\} \notin E.$$

Co tato definice vlastně říká? Jednoduše to, že doplněk grafu G obsahuje právě ty hrany, které v původním grafu chybí. Podívejme se na příklady.

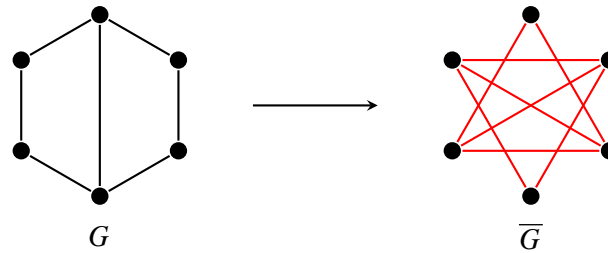
Příklad 2.5.4. Příklady doplňků grafů, viz obrázky 2.9 a 2.10.



Obrázek 2.9: Doplněk grafu na čtyřech vrcholech.

2.5.1 Převod Klika na NzMna

Jako poslední se zamyslíme, jaká je tedy souvislost mezi problémem nezávislé množiny a kliky v grafu. Předpokládejme, že máme zadaný graf G a číslo $k \in \mathbb{N}$. Chceme sestavit graf G' a číslo k' takové, že v G existuje kliky velikosti alespoň k právě tehdy, když v novém grafu G' existuje nezávislá množina velikosti alespoň k' . To zařídíme velice jednoduše.



Obrázek 2.10: Doplněk prasátkového grafu.

Pro daný graf G sestojíme jeho doplněk $\overline{G}$. Pokud je K klikou v G , pak každé dva různé vrcholy z K jsou v G spojeny hranou, a proto v $\overline{G}$ spojeny nejsou. Množina K je tedy nezávislá v $\overline{G}$. Stejný argument funguje i obráceně.

Tedy celkově stačí položit $G' = \overline{G}$ a $k' = k$ a jsme hotovi. Opačný převod provedeme naprosto stejně.

2.6 Třídy problémů P a NP

V této sekci se nyní ohledneme trochu zpět a podíváme se na celou problematiku z více abstraktního hlediska. Naším cílem je představit si dvojici základních tříd problémů a vysvětlit si jejich podstatu v současném světě informatiky. Jako první se trochu blíže podíváme na problémy, které jsme si již představili.

2.6.1 Rozbor předchozích problémů

V předchozích sekcích jsme si představili některé základní problémy. Všimněte si, že vždy jsme se ptali na to, zda nějaký objekt splňuje určitou vlastnost. Např. u logické formule jsme se ptali, zdali je či není splnitelná. U grafů jsme se ptali, zda v nich existuje nezávislá množina či klika určité velikosti. Výstupem problému tedy byly vždy pouze hodnoty 1 (pravda) nebo 0 (nepravda). Takovým problémům se říká tzv. **rozhodovací problémy**.

Mimo tento typ se však v praxi často setkáme s **optimalizačními problémy**. Nejspíše nás nebude zajímat jen to, zda v grafu existuje klika určité velikosti, ale jak velká je největší klika. U problémů s číselným parametrem, které zde probíráme, lze optimalizační hodnotu získat opakovaným voláním rozhodovací varianty. Graf $G = (V, E)$ ponecháme stejný a měníme parametr k .

Uvažujme, že podprogram $\text{ExistsClique}(G, k)$ určuje pro vstupní graf G a číslo k , zda v něm existuje klika velikosti alespoň k . Pomocí něj pak umíme vyřešit i optimalizační verzi, viz 2.6.1. Podobně tak lze řešit

Algoritmus 2.6.1: Určení maximální kliky v grafu

Vstup: Graf $G = (V, E)$

```

1  $n \leftarrow |V|$ 
2 for  $k = n, n - 1, n - 2, \dots, 1, 0$  do
3   if  $\text{ExistsClique}(G, k) = 1$  then return  $k$ ;

```

Výstup: Velikost maximální kliky v grafu G

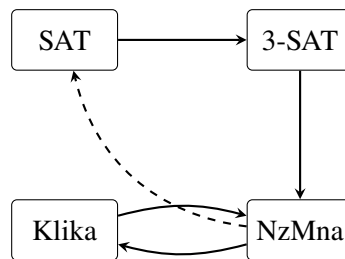
i ostatní problémy. Pro nás je však podstatné si uvědomit, že naše omezení na rozhodovací problémy má tedy hned dva důvody. Celkově nám to zjednodušuje pohled na danou problematiku a navíc od rozhodovacího problému k optimalizačnímu umíme přejít velice snadno.

Vraťme se tedy zpět k rozhodovacím variantám, které jsme již probírali. Zkusme si rozebrat rychlosti jednotlivých

převodů.

- SAT $\rightarrow$ 3-SAT. Klauzuli s $k > 3$ literály rozložíme v čase $\mathcal{O}(k)$. Převod tedy potrvá maximálně lineárně dlouho v délce formule, tedy počtu jejích literálů n .
- 3-SAT $\rightarrow$ SAT. Formule v 3-CNF jsou již v CNF, takže použijeme totožný vstup. Jeho případné zkopírování zabere lineární čas.
- 3-SAT $\rightarrow$ NzMna. Sestrojený graf má vrcholy v počtu literálů formule, to potrvá $\mathcal{O}(n)$. Hran v grafu bude maximálně kvadraticky mnoho (tj. úplný graf), tedy $\mathcal{O}(n^2)$. Celkově $\mathcal{O}(n + n^2) = \mathcal{O}(n^2)$.
- Klika $\rightarrow$ NzMna. Doplněk grafu má vrcholy původního grafu. Hran sestrojíme maximálně kvadraticky mnoho, tj. celkově $\mathcal{O}(n^2)$.
- NzMna $\rightarrow$ Klika. Stejně jako u předchozího převodu.

Můžeme si všimnout, že všechny převody jsou polynomiální, tedy jak jsme uváděli v úvodu kapitoly, můžeme je považovat za „rychlé“, viz obrázek 2.11. Co však dodnes neumíme, je kterýkoliv z těchto problémů v polynomiálním čase řešit. Uvedené převody nám je ale dávají hezky do souvislosti. Pokud bychom uměli řešit kterýkoliv z nich rychle, pak bychom uměli rychle řešit všechny. Libovolný jiný bychom totiž převedli na ten, který rychle řešit umíme. Např. pokud bychom byli schopni řešit problém kliky v polynomiálním čase, mohli bychom řešit



Obrázek 2.11: Shrnutí převodů mezi jednotlivými problémy.

i 3-SAT: formuli polynomiálně převedeme na instanci nezávislé množiny, sestrojíme doplněk vzniklého grafu a na něm spustíme algoritmus pro kliku. Složení konečného počtu polynomiálních algoritmů je opět polynomiální algoritmus.

2.6.2 Ověřování problémů

Už jsme si zmínili, že žádné ze zmíněných problémů (ani mnohé jiné) neumíme řešit v polynomiálním čase, ač instance mnohých z nich dnes umíme řešit v uspokojivých časech. U hodně z nich však umíme rychle ověřit, zda je nějaké řešení správné. Uvažujme situaci, kdy máme zadanou logickou formuli φ a nějaké ohodnocení jejích proměnných. Zajímá nás, zda je dané ohodnocení splňující. To není ale vůbec těžké. Zkrátka danou formuli vyhodnotíme. To však již umíme provést rychle!

- Pokud je zadané ohodnocení splňující, pak je φ splnitelná.
- Pokud ohodnocení není splňující, pak to ještě nutně *neznamená*, že je *nesplnitelná*.

Všimněme si tedy, že nyní se naše otázka změnila.

- Původní otázka: *Je zadaná formule φ splnitelná?*
- Současná otázka: *Je zadané ohodnocení proměnných zadané formule φ splňující?*

Pojďme se podívat touto optikou ještě na jiný problém. U problému NzMna nás zajímalo, zda zadaný graf G obsahuje nezávislou množinu velikosti alespoň k . Nyní nás bude zajímat následující otázka: *Je zadaná množina N*

v grafu G nezávislá velikosti alespoň k ? Opět jsme přeformulovali otázku do podoby, kdy původně hledaný¹¹ objekt již máme zadáný a pouze ověřujeme, zda splňuje danou vlastnost. Zde je řešení opět velice jednoduché, zkrátka zkontrolujeme, zda je množina dostatečně velká a zda mezi žádnou dvojicí vrcholů nevede hrana. Viz algoritmus 2.6.2.

Algoritmus 2.6.2: Ověření nezávislé množiny

Vstup: Graf $G = (V, E)$, číslo $k \in \mathbb{N}$ a množina $N \subseteq V$
 // Množina N není dostatečně velká
 1 **if** $|N| < k$ **then return** 0;
 // Dvojice vrcholů je spojena hranou
 2 **foreach** $u \in N$ **do**
 3 **foreach** $v \in N; v \neq u$ **do**
 4 **if** $\{u, v\} \in E$ **then return** 0;
 // Množina N je nezávislá
 5 **return** 1

Časová složitost je zjevně polynomiální. Označíme $|N| = m$, přičemž jistě platí $m \leq |V|$. Vnější cyklus se provede m -krát a vnitřní cyklus $(m - 1)$ -krát (vrchol u jako jediný znovu nevybíráme). Při reprezentaci grafu maticí sousednosti ověříme existenci hrany v konstantním čase. Tedy celkově

$$\mathcal{O}(m(m - 1)) = \mathcal{O}(m^2 - m) = \mathcal{O}(m^2),$$

což je polynom. Tedy umíme rychle ověřit, zda je množina nezávislá.

Podobně si můžete rozmyslet, jak budou vypadat takové “ověřovací” algoritmy pro ostatní problémy. Co je ovšem podstatou tohoto výkladu? Obecně existují problémy, které jsme schopni řešit s určitou “náповědou”. Např. v případě nezávislé množiny si můžeme nechat napovědět nějakou podmnožinu vrcholů $N \subseteq V$ a nám jen stačí ověřit, zda je skutečně nezávislá a dostatečně velká. Podobně u problému SAT si necháme napovědět ohodnocení proměnných a nám pouze stačí ověřit, zda danou formuli splňuje.

Poznámka 2.6.1. U třídy NP požadujeme, aby náповěda měla délku nejvýše polynomiální ve velikosti původního vstupu. Úplný optimální postup pro n -diskové Hanojské věže má $2^n - 1$ tahů, takže už samotný jeho zápis je exponenciálně dlouhý. Takový zápis proto nemůže sloužit jako polynomiálně dlouhý certifikát.

2.6.3 Zavedení tříd složitosti

Předchozí úvahy nám dávají rozdělení problémů do dvou základních kategorií.

- Ty, které *umíme řešit v polynomiálním čase*
- a ty, u nichž *dokážeme v polynomiálním čase ověřit správnost řešení*.

Tím dostáváme tzv. *třídy složitosti* P a NP (také jim říkáme *třídy problémů*). Pojdme se na ně podívat.

P je třída rozhodovacích problémů řešitelných v polynomiálním čase.

Třída P tedy zachycuje všechny problémy, které jsou v jistém smyslu rychle řešitelné¹².

⁽¹¹⁾Formálně nelze přímo říci, že takový objekt vyloženě “hledáme”. Pouze nás zajímá, zda existuje.

⁽¹²⁾Asi by čtenáře mohlo napadnout, že např. algoritmus s časovou složitostí $\mathcal{O}(n^{1000})$ je jen stěží použitelný. Z druhé strany může být algoritmus s časovou složitostí $\mathcal{O}(1,001^n)$ prakticky použitelný pro určité velikosti dat, byť je exponenciální. Polynomiální čas je teoretická hranice s mnoha příjemnými vlastnostmi, nikoliv záruka praktické rychlosti každého konkrétního algoritmu.

Příklad 2.6.2. Příklady některých (rozhodovacích) problémů, které jsou řešitelné v polynomiálním čase.

- (i) *Existence cesty mezi dvěma vrcholy.* Lze řešit např. pomocí BFS nebo DFS¹³; oba algoritmy jsou polynomiální.
- (ii) *Existence podřetězce v textu.* Máme zadaný textový řetězec délky n a slovo délky k . Naivně můžeme otestovat všech $n - k + 1$ možných počátečních pozic a na každé porovnat nejvýše k znaků. Dostaneme polynomiální čas $\mathcal{O}((n - k + 1)k)$.
- (iii) *Existence prvku v poli.* Lze řešit např. sekvenčním vyhledáváním, které běží v čase $\mathcal{O}(n)$.

Všechny zmíněné případy problémů jsou řešitelné v čase $\mathcal{O}(n^k)$.

NP je třída rozhodovacích problémů, u nichž lze kladnou odpověď doložit polynomiálně dlouhým certifikátem a ten ověřit v polynomiálním čase.



Třídy P a NP zde zavádíme neformálně a přeskakujeme technické podrobnosti o kódování vstupů a výpočetním modelu. Pro naše potřeby si s tímto pohledem vystačíme.¹⁴



Písmeno „N“ ve zkratce NP neznamena „nepolynomiální“. Třída se jmenuje podle nedeterministického polynomiálního času; v našem výkladu jej zachycuje právě polynomiálně ověřitelný certifikát.

Problémy třídy NP jsou již trochu zajímavější, neboť již nutně nepožadujeme, aby byl problém rychle řešitelný. Podmínkou je pouze, že musíme být schopni rychle ověřit, zda zadaný objekt splňuje požadovanou vlastnost. Alternativně též můžeme na třídu NP nahlížet tak, že daný problém jsme schopni řešit rychle s určitou nápovědou.

Příklad 2.6.3. Příklady problémů ležících ve třídě NP jsme si již ukazovali v sekci 2.6.1, ale ještě jednou si je zde shrneme.

- (i) SAT, 3-SAT. Jako nápověda zde slouží ohodnocení proměnných. Stačí ověřit, zda je splňující vyhodnocením zadané formule.
- (ii) *Nezávislá množina.* Na vstupu dostaneme kromě grafu G a čísla k ještě nějakou množinu vrcholů N . Nám posléze stačí rozhodnout, zda N je nezávislá a platí $|N| \geq k$. Viz algoritmus 2.6.2.
- (iii) Klika. Opět dostaneme množinu vrcholů N . Postup podobný jako u NzMna.

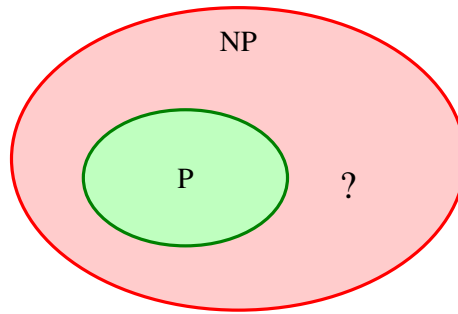
Může se na první pohled zdát, že třídy P a NP představují dva oddělené světy. Ve skutečnosti tomu tak není. Lze si totiž všimnout zajímavého vztahu, který má kupodivu jednoduché zdůvodnění.

Věta 2.6.4. Platí $P \subseteq NP$.

Věta 2.6.4 říká, že třída P leží uvnitř třídy NP, viz obrázek 2.12. Pokud totiž dokážeme nějaký problém $A \in P$ řešit v polynomiálním čase (tedy bez nápovědy), tím spíš jej dokážeme řešit rychle s nápovědou (např. algoritmus může nápovědu ignorovat a určit řešení přímo z původního vstupu). Tento fakt ovšem zahrnuje i potenciální variantu, že $P = NP$. To dodnes není známo a otázka, zda jsou třídy P a NP skutečně různé, zůstává nezodpovězena¹⁵. Obecně je však přijímána spíše hypotéza, že jsou tyto třídy různé.

⁽¹³⁾O DFS již víme, že nenalezne nutně nejkratší cestu, ale zde řešíme pouze její existenci.

⁽¹⁵⁾Problém P vs. NP patří mezi tzv. *Problémy tisíciletí*, které vyhlásil Clayův matematický institut v roce 2000. Na vyřešení každého z nich vypsál institut jeden milion amerických dolarů.



Obrázek 2.12: Schematická ilustrace vztahu $P \subseteq NP$; zda je inkluze ostrá, nevíme.

U všech zmíněných problémů v této kapitole s jistotou víme, že leží ve třídě NP, avšak nevíme, zda leží i ve třídě P, neboť pro žádný z nich neznáme polynomiální algoritmus. Avšak lze o nich ukázat, že jsou tyto problémy (a mnohé jiné) v jistém smyslu “ty nejtěžší” ve třídě NP.

Definice 2.6.5 (NP-úplný problém). Problém A nazveme NP-úplným, pokud $A \in NP$ a zároveň je na něj polynomiálně převoditelný každý problém z NP. Tzn.

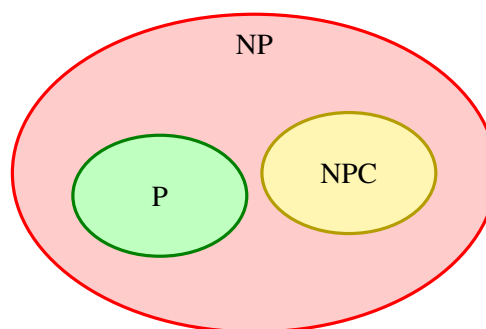
$$\forall L \in NP : L \rightarrow A.$$

Druhá podmínka v definici 2.6.5 se možná zdá hodně silná. Existuje vůbec takový problém? Ač se to může zdát nepravděpodobné, tak v roce 1971 dokázal americký informatik Stephen Cook překvapivou větu.

Věta 2.6.6. Problém SAT je NP-úplný.

Důkaz toho tvrzení si zde ukazovat nebudeme, neboť je zcela nad rámec tohoto výkladu. Tento výsledek má v této teorii velice silný důsledek. Pokud by se ukázalo, že nějaký NP-úplný problém A leží ve třídě P, tak by z toho již plynulo, že $P = NP$. Každý problém z NP bychom mohli jednoduše polynomiálně převést na A a ten pak vyřešit opět polynomiálně. Toto je přesně důvod, proč problém SAT hraje v současné informatice takovou důležitou roli, neboť se historicky ukázalo, že pomocí něj lze řešit kterýkoliv jiný problém¹⁶ v NP.

NP-úplných problémů existuje více. Třída takových problémů se označuje jako NPC (z anglického NP-Complete) a platí $NPC \subseteq NP$, viz obrázek 2.13. Pokud je NP-úplný problém A polynomiálně převoditelný na problém B , pak je B NP-těžký; aby byl také NP-úplný, musíme navíc ověřit $B \in NP$. Převody z této kapitoly společně se snadným ověřením certifikátů ukazují, že SAT, 3-SAT, NzMna a Klika jsou NP-úplné.



Obrázek 2.13: Obvyklé schematické znázornění vztahu P, NP a NPC za předpokladu $P \neq NP$.

Jak už jsme si ale uvedli, to zda platí $P = NP$, stále nevíme. Jaké by to ale mělo důsledky v praxi? Pojďme se alespoň na chvíli zamyslet nad oběma variantami.

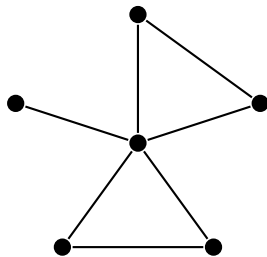
⁽¹⁶⁾Proto se někdy též říká, že NP-úplné problémy tvoří v jistém smyslu ty “nejtěžší” v NP, neboť pokud umíme řešit nějaký z nich, pak umíme vyřešit všechny.

- $P = NP$. Každý rozhodovací problém z NP by měl polynomiální algoritmus a u mnoha běžných NP-úplných úloh bychom pomocí opakovaných dotazů dokázali polynomiálně nalézt i řešení. Samotná rovnost by však nezaručovala prakticky malý exponent ani automaticky nezrušila úplně všechny druhy šifrování. Zásadně by ovšem zpochybnila běžné kryptografické předpoklady založené na výpočetní obtížnosti.
- $P \neq NP$. Žádný NP-úplný problém by neležel v P, takže by pro SAT, NzMna ani Klika neexistoval polynomiální algoritmus. Ani tato nerovnost sama o sobě však nestačí k existenci bezpečné kryptografie; ta vyžaduje silnější předpoklady o obtížnosti konkrétních problémů.

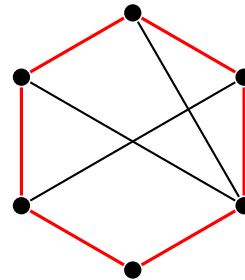
2.6.4 Další NP-úplné problémy

K závěru této kapitoly si ještě uvedeme nějaké další NP-úplné problémy a k nim i nějaké příklady jejich aplikací (resp. výskytů). NP-úplnost si zde však již dokazovat pro žádný z nich nebudeme. Pro žádný z těchto problémů není známý polynomiální algoritmus.

- (i) *Hamiltonovská kružnice*. Máme zadaný graf $G = (V, E)$. Zajímá nás, zda v něm existuje kružnice, která projde každý vrchol právě jednou. Viz obrázky 2.14a a 2.14b. Příbuzný problém *obchodního cestujícího*



(a) Příklad grafu, kde neexistuje hamiltonovská kružnice



(b) Graf a jeho hamiltonovská kružnice

navíc pracuje s cenami hran a hledá co nejlevnější okružní trasu. Právě tato optimalizační varianta se objevuje v logistice.

- (ii) *Součet podmnožiny*. Máme zadaný seznam přirozených čísel M a číslo $S \in \mathbb{N}$. Existuje výběr prvků seznamu, jehož součet je roven S ? Např. pro množinu $M = \{1, 3, 10, 11, 20\}$ (ne)existují součty uvedené v tabulce 2.1. Rychlé řešení součtu podmnožiny by pomohlo např. s optimalizací plánování nákladů

Součet S	13	2	22	16
Existence	✓	✗	✓	✗

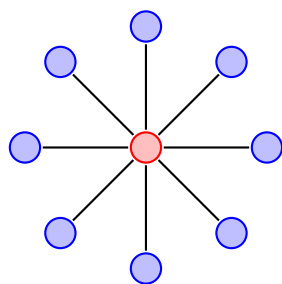
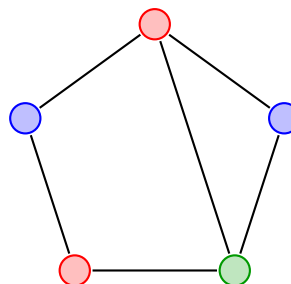
Tabulka 2.1: Tabulka součtů množiny M

rozpočtem (např. ve firmě). Dále se též používá u tzv. *jednosměrných funkcí* v kryptografii.

- (iii) *Barvení grafu*. Chceme zjistit, zda pro zadaný graf je možné jeho vrcholy obarvit k barvami (formálně přidělíme každému vrcholu číslo od 1 do k) tak, aby vrcholy stejné barvy nebyly nikdy spojeny hranou? Pro příklady viz obrázky 2.15a a 2.15b. Pro $k = 2$ lze barvení rozhodnout polynomiálně, ale pro každé pevné $k \geq 3$ je problém NP-úplný.
- (iv) *Problém batohu*. Jsou dány předměty s váhami a cenami a kapacita batohu. Chceme najít co nejdražší podmnožinu předmětů tak, abychom nepřekročili kapacitu batohu. V rozhodovací variantě se ptáme, zda existuje podmnožina předmětů s celkovou cenou alespoň C a celkovou vahou nejvýše W .

Formálněji máme váhy předmětů $w_1, \dots, w_n$ a jejich ceny $c_1, \dots, c_n$, přičemž hledáme hodnoty $x_1, \dots, x_n \in \{0, 1\}$ tak, aby platilo

$$\sum_{i=1}^n c_i x_i \geq C \quad \text{a} \quad \sum_{i=1}^n w_i x_i \leq W.$$

(a) Barvení grafu pro $k = 2$.(b) Barvení grafu pro $k = 3$.

Problém batohu se též objevuje v kryptografii.

- (v) *Dva loupežníci*. Ptáme se, zda lze zadaný seznam kladných celých čísel rozdělit na dvě části se stejným součtem.¹⁷ Například

$$L = \{1, 3, 4, 6, 8, 10\}$$

lze rozdělit na $L_1 = \{6, 10\}$ a $L_2 = \{1, 3, 4, 8\}$; obě části mají součet 16. S příbuznými úlohami se setkáme při vyvažování zátěže mezi procesory.

⁽¹⁷⁾Loupežníci tedy řeší, jak spravedlivě rozdělit lup.

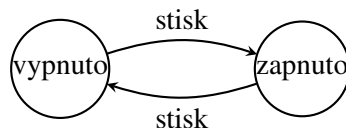
Konečné automaty

V této kapitole se budeme věnovat základům *teorie automatů*. Konečné automaty jsou jednoduché abstraktní stroje, které zpracovávají vstupní slovo znak po znaku a na konci rozhodnou, zda jej přijmou, nebo odmítnou. Množiny přijímaných slov tvoří tzv. *formální jazyky*.

Nejdříve si zavedeme potřebné operace nad slovy a jazyky. Poté se podíváme na deterministické a nedeterministické konečné automaty, regulární výrazy a formální gramatiky. Ukáže se, že konečné automaty, regulární výrazy a regulární gramatiky jsou tři různé způsoby popisu stejné třídy jazyků. Později tuto teorii využijeme při lexikální analýze v kapitole 4.

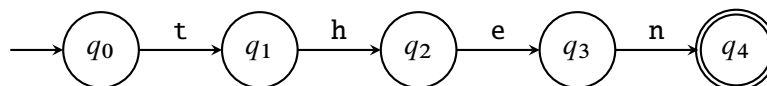
3.1 Úvod

Mnoho programů lze popsat pomocí konečného počtu *stavů* a pravidel, podle kterých se mezi nimi přechází. Obvyčejný spínač má například dva stavy: zapnuto a vypnuto. Každé stisknutí jej přesune do druhého stavu.



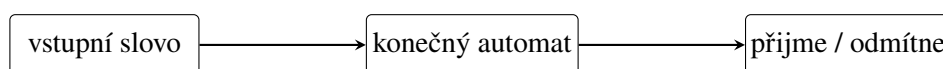
Obrázek 3.1: Jednoduchý stavový model spínače.

Podobně můžeme po jednotlivých znacích rozpoznávat klíčové slovo **then**. Stav si pamatuje, jak velkou část slova jsme již přečetli.



Obrázek 3.2: Schéma automatu rozpoznávajícího slovo **then**.

Konečný automat tedy dostane na vstupu konečnou posloupnost znaků, které říkáme *slovo*. Slovo čte zleva doprava a po každém znaku může změnit svůj stav. Po přečtení celého vstupu vydá odpověď: slovo přijme, nebo odmítne.



Obrázek 3.3: Zjednodušené schéma konečného automatu.



Automat slova *rozpoznává*: rozhoduje, která z nich patří do popisovaného jazyka. Později si ukážeme gramatiky, které naopak slova *generují*.

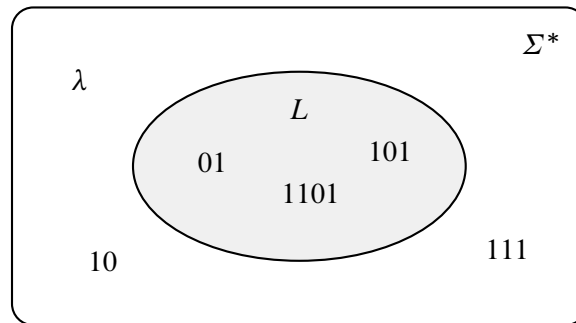
3.2 Formální jazyky

Než se pustíme do přesné definice automatu, potřebujeme si ujasnit, co vlastně automat dostává na vstupu a jak budeme popisovat množiny přípustných vstupů.

Definice 3.2.1 (Abeceda, slovo a jazyk). Mějme konečnou neprázdnou množinu znaků Σ , kterou nazýváme *abecedou*. Pak

- *slovo* (*řetězec*) je libovolná konečná posloupnost znaků z Σ ;
- *prázdné slovo* je slovo neobsahující žádný znak, značíme je λ ;
- množinu všech slov nad Σ značíme Σ^* ;
- množinu všech neprázdných slov značíme $\Sigma^+ = \Sigma^* \setminus \{\lambda\}$;
- *jazyk nad abecedou* Σ je libovolná množina $L \subseteq \Sigma^*$;
- délku slova w značíme $|w|$ a počet výskytů znaku a ve slově w značíme $|w|_a$.

Symbol λ není znakem abecedy. Je to označení posloupnosti délky nula. Proto pro každé slovo w platí $\lambda w = w\lambda = w$.



Obrázek 3.4: Jazyk L je libovolná podmnožina množiny všech slov Σ^* .

Definice 3.2.2 (Operace nad slovy). Nechť $u = u_1u_2 \dots u_n$ a $v = v_1v_2 \dots v_m$ jsou slova nad Σ .

- Jejich *konkatenace* (*zřetěžení*) je slovo

$$uv = u_1u_2 \dots u_nv_1v_2 \dots v_m.$$

- Mocniny slova definujeme vztahy $u^0 = \lambda$ a $u^{k+1} = u^k u$ pro $k \in \mathbb{N}_0$.

Definice 3.2.3 (Operace nad jazyky). Pro jazyky $K, L \subseteq \Sigma^*$ definujeme

$$\begin{aligned} K \cup L &= \{w \in \Sigma^* : w \in K \text{ nebo } w \in L\}, \\ KL &= \{uv : u \in K, v \in L\}, \\ L^0 &= \{\lambda\}, \quad L^{n+1} = L^n L, \\ L^* &= \bigcup_{n=0}^{\infty} L^n. \end{aligned}$$

Množině L^* říkáme *Kleeneho uzávěr* jazyka L .

Příklad 3.2.4. Mějme abecedu $\Sigma = \{a, b\}$ a slova $u = aabb$, $v = bba$.

- $|u| = 4$, $|u|_a = 2$ a $uv = aabbbba$.
- $v^3 = bbabbabba$.
- $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$.
- Jazyk všech slov délky nejvýše dva je

$$\{w \in \Sigma^* : |w| \leq 2\} = \{\lambda, a, b, aa, ab, ba, bb\}.$$

Jazyky mohou být zadané výčtem, vlastností nebo jiným popisem. Uveďme si tři o něco zajímavější příklady nad binární abecedou $\Sigma = \{0, 1\}$:

$$\begin{aligned} L_{\text{suff}} &= \{w \in \Sigma^* : w \text{ končí na } 01\} = \{u01 : u \in \Sigma^*\}, \\ L_{\text{bez } 11} &= \{w \in \Sigma^* : w \text{ neobsahuje podslovo } 11\}, \\ L_{\text{eq}} &= \{w \in \Sigma^* : |w|_0 = |w|_1\}. \end{aligned}$$

První dva jazyky budou regulární a později pro ně sestrojíme automaty. Třetí jazyk regulární není: konečný automat si neumí pamatovat neomezeně velký rozdíl mezi počtem nul a jedniček.

3.2.1 Rozhodovací problémy jako jazyky

V minulé kapitole jsme vstupy rozhodovacích problémů kódovali pomocí nul a jedniček. Rozhodovací problém tak můžeme ztotožnit s jazykem všech zakódovaných vstupů, pro které je odpověď kladná. Například

$$\text{SAT} = \{\langle \varphi \rangle : \varphi \text{ je splnitelná formule v CNF}\},$$

kde $\langle \varphi \rangle \in \{0, 1\}^*$ značí binární kód formule φ . Tento pohled nám později umožňuje studovat výpočetní problémy pomocí strojů, které přijímají jazyky.

3.3 Deterministické konečné automaty

Definice 3.3.1 (Deterministický konečný automat). *Deterministický konečný automat* (zkráceně DFA) je uspořádaná pětice

$$A = (Q, \Sigma, \delta, q_0, F),$$

kde

- Q je konečná množina stavů,
- Σ je vstupní abeceda,
- $\delta : Q \times \Sigma \rightarrow Q$ je přechodová funkce, která každé dvojici stavu a znaku přiřazuje právě jeden následující stav,
- $q_0 \in Q$ je počáteční stav a
- $F \subseteq Q$ je množina přijímajících stavů.

Při kreslení někdy kvůli přehlednosti vynecháme *odpadní stav*: všechny nezakreslené přechody si pak představujeme vedené do tohoto nepřijímajícího stavu, který má smyčku pro každý znak. Formálně je tak přechodová funkce stále definována pro každou dvojici.

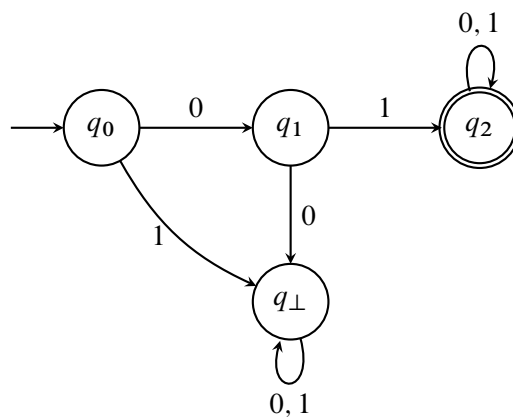
Automat zakreslujeme jako orientovaný graf. Stavy jsou vrcholy, přechody hrany s popisky. Počáteční stav označíme šipkou vedoucí „odnikud“ a přijímající stavy dvojitým okrajem.

Výpočet automatu. Na začátku se automat nachází ve stavu q_0 . Vstupní slovo čte zleva doprava. Nachází-li se ve stavu q a čte znak a , přejde do stavu $\delta(q, a)$. Po přečtení celého slova přijme právě tehdy, když skončil ve stavu z F .

Definice 3.3.2 (Jazyk automatu). Jazykem automatu A nad abecedou Σ rozumíme množinu

$$L(A) = \{w \in \Sigma^* : A \text{ přijme slovo } w\}.$$

Příklad 3.3.3 (Slova začínající na 01). Automat na obrázku 3.5 nejprve vyžaduje znaky 0 a 1. Jakmile je přečte, zůstane v přijímajícím stavu bez ohledu na zbytek vstupu. Jeho jazyk je



Obrázek 3.5: DFA přijímající právě slova začínající na 01.

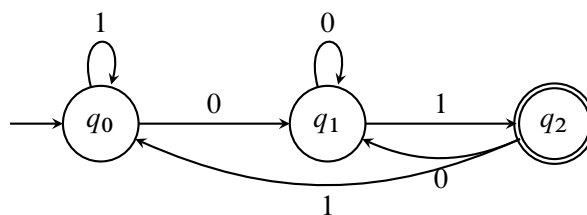
$$L(A) = \{01u : u \in \{0, 1\}^*\}.$$

Slovo 0110 přijme, zatímco 001 odmítne už při čtení druhého znaku.

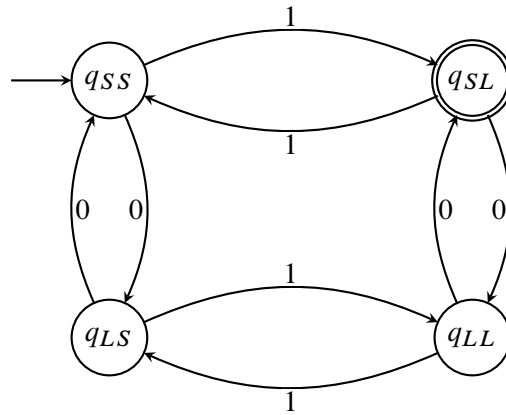
Složitější příklady.

Příklad 3.3.4 (Slova končící na 01). Nyní už nestačí zkontrolovat první dva znaky. Automat si musí průběžně pamatovat nejdelší konec dosud přečtené části, který by mohl být začátkem slova 01:

- q_0 : poslední znak není 0 nebo jsme zatím nic nepřečetli;
- q_1 : poslední znak je 0;
- q_2 : poslední dva znaky jsou 01.



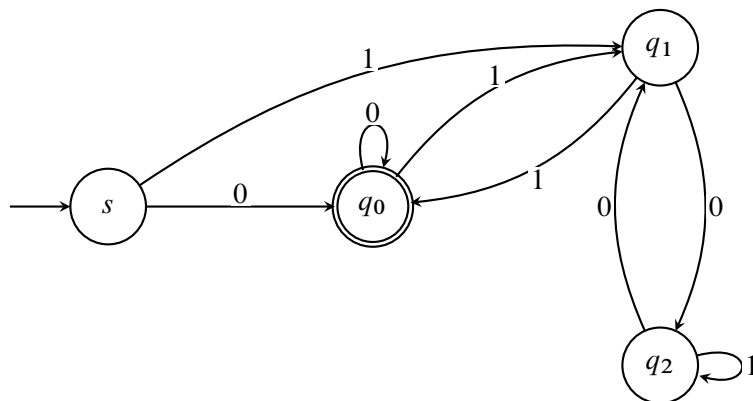
Obrázek 3.6: DFA pro jazyk $L_{\text{suff}} = \{u01 : u \in \{0, 1\}^*\}$.



Obrázek 3.7: DFA přijímající slova se sudým počtem nul a lichým počtem jedniček.

Příklad 3.3.5 (Sudý počet nul a lichý počet jedniček). Potřebujeme si pamatovat dvě informace, každou se dvěma možnostmi. Dostaneme proto čtyři stavy q_{SS} , q_{SL} , q_{LS} , q_{LL} ; první index udává paritu počtu nul a druhý paritu počtu jedniček. Znak 0 změní první index, znak 1 druhý. Jediným přijímajícím stavem je q_{SL} .

Příklad 3.3.6 (Binární čísla dělitelná třemi). Čteme-li binární číslo zleva doprava a dosud přečtená část dává po dělení třemi zbytek r , pak po připsání bitu $b \in \{0, 1\}$ dostaneme zbytek čísla $2r + b$. Stačí si tedy pamatovat zbytek 0, 1 nebo 2. Kvůli vyloučení prázdného slova používáme zvláštní počáteční stav s . Automat například



Obrázek 3.8: DFA pro neprázdné binární zápisy čísel dělitelných třemi.

přijme $110_2 = 6$, ale odmítne $101_2 = 5$.

3.4 Nedeterministické konečné automaty

U deterministického automatu vede pro daný stav a znak nejvýše jeden přechod. Někdy je však návrh mnohem jednodušší, když automatu dovolíme několik možností a představíme si, že vyzkouší všechny paralelně.

Definice 3.4.1 (Potenční množina). Nechť M je množina. *Potenční množinou* množiny M nazveme množinu všech jejích podmnožin. Značíme ji

$$\mathcal{P}(M) = \{N : N \subseteq M\}.$$

Například pro $M = \{q_0, q_1\}$ dostaneme

$$\mathcal{P}(M) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}.$$

Definice 3.4.2 (Nedeterministický konečný automat). *Nedeterministický konečný automat* (zkráceně NFA) je pětice

$$A = (Q, \Sigma, \delta, q_0, F),$$

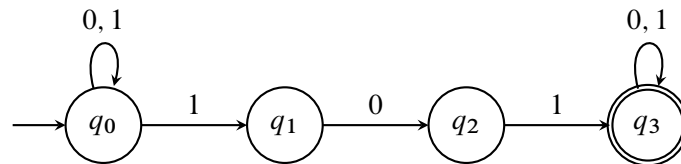
kde Q , Σ , q_0 a F mají stejný význam jako u DFA a

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

přiřazuje dvojici stavu a znaku množinu možných následujících stavů.

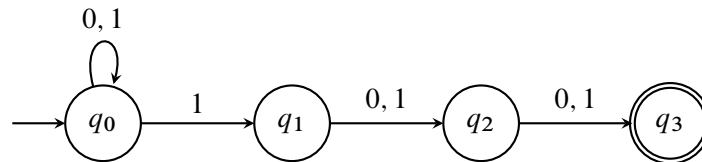
Přečte-li NFA ve stavu q znak a , výpočet se rozvětví do všech stavů z $\delta(q, a)$. Je-li tato množina prázdná, daná větev zanikne. Automat přijme vstupní slovo právě tehdy, když alespoň jedna větev po přečtení celého slova skončí v přijímajícím stavu.

Příklad 3.4.3 (Slova obsahující podslovo 101). Počáteční stav q_0 zpracovává libovolný začátek slova. Při každé jedničce může automat zároveň „uhádnout“, že právě začalo hledané podslovo, a spustit novou větev ve stavu q_1 . Pro slovo 01101 vznikne přijímající větev, která po posledních třech znacích projde stavy q_1, q_2, q_3 .



Obrázek 3.9: NFA přijímající právě slova obsahující podslovo 101.

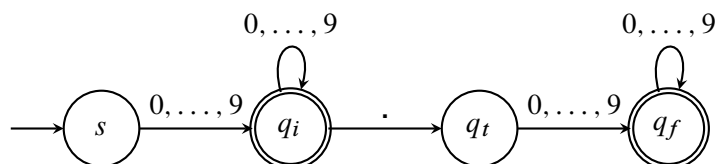
Příklad 3.4.4 (Třetí znak od konce je 1). NFA může při každé přečtené jedničce odhadnout, že do konce zbývají právě dva znaky. Pak už pouze odpočítá dva další přechody. Tento automat má jen čtyři stavy. Ekvivalentní DFA



Obrázek 3.10: NFA pro jazyk slov, jejichž třetí znak od konce je 1.

si musí pamatovat všechny možné konce délky tři a potřebuje osm stavů.

Příklad 3.4.5 (Jednoduchý zápis desetinného čísla). Automat na obrázku 3.11 přijímá neprázdný blok číslic a případně desetinnou tečku následovanou dalším neprázdným blokem číslic. Přijme tedy 17 a 17.25, ale odmítne .25, 17. i 1.2.3.



Obrázek 3.11: Konečný automat pro jednoduchý zápis nezáporného desetinného čísla.

3.4.1 Vztah DFA a NFA

Nedeterminismus sice může výrazně zjednodušit zápis automatu, ale nerozšíří množinu jazyků, které konečné automaty dokážou přijímat.

Věta 3.4.6 (Podmnožinová konstrukce). Ke každému NFA A existuje DFA B takový, že $L(A) = L(B)$.

Důkaz. Mějme NFA $A = (Q, \Sigma, \delta, q_0, F)$. Jeden stav nového DFA bude popisovat celou množinu stavů, v nichž se NFA může po přečtení stejného začátku slova nacházet. Položíme tedy

$$B = (\mathcal{P}(Q), \Sigma, \delta', \{q_0\}, F'),$$

kde

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a)$$

a množina $S \subseteq Q$ je přijímajícím stavem právě tehdy, když $S \cap F \neq \emptyset$.

Indukcí podle délky přečteného slova dostaneme, že stav DFA B je vždy přesně množinou všech stavů, ve kterých mohou být jednotlivé větve NFA A . DFA proto přijme právě tehdy, když alespoň jedna větev NFA skončí v F . $\square$

Každý DFA je zároveň NFA, jen všechny množiny $\delta(q, a)$ mají nejvýše jeden prvek. Oba modely tedy přijímají stejné jazyky. Cena za odstranění nedeterminismu může být vysoká: NFA s n stavy může podmnožinovou konstrukcí vytvořit DFA až s 2^n stavy.

3.4.2 Přechody bez čtení znaku

Pro důkaz Kleeneho věty se nám bude hodit ještě drobné technické rozšíření.

Definice 3.4.7 (λ -NFA). λ -NFA je NFA s přechodovou funkcí

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q),$$

který smí navíc použít přechod označený λ . Při takovém přechodu změní stav, aniž by přečetl znak vstupu.

Takový automat stále nepřijímá nic navíc. Pro množinu stavů S označme $E(S)$ množinu všech stavů dosažitelných ze S pouze pomocí nulového nebo více λ -přechodů. Obvyčejný NFA sestrojíme tak, že pro každý stav q a znak a položíme

$$\delta'(q, a) = E\left(\bigcup_{r \in E(\{q\})} \delta(r, a)\right)$$

a stav q prohlásíme za přijímající, pokud $E(\{q\}) \cap F \neq \emptyset$. Nový automat před každým znakem i po něm provede předem všechny možné λ -přechody, takže přijímá tentýž jazyk.

3.4.3 Regulární jazyky

Definice 3.4.8 (Regulární jazyk). Jazyk L nazveme *regulární*, pokud existuje DFA A takový, že $L = L(A)$.

Podle věty 3.4.6 můžeme v definici stejně dobře použít NFA nebo λ -NFA. Ne každý jazyk je však regulární.

Věta 3.4.9. Jazyk

$$L_{\text{eq}} = \{w \in \{0, 1\}^* : |w|_0 = |w|_1\}$$

není regulární.

Důkaz. Předpokládejme pro spor, že existuje DFA A s m stavy a $L(A) = L_{eq}$. Uvažujme $m + 1$ slov

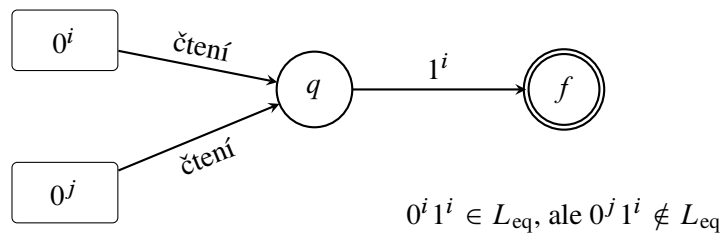
$$\lambda, 0, 00, \dots, 0^m.$$

Automat má jen m stavů, takže po přečtení dvou různých slov 0^i a 0^j , kde $0 \leq i < j \leq m$, musí být ve stejném stavu.

Nyní k oběma slovům připojme stejný konec 1^i . Automat z téhož stavu čte tentýž zbytek, a proto buď přijme obě slova, nebo obě odmítne. Jenže

$$0^i 1^i \in L_{eq}, \quad 0^j 1^i \notin L_{eq}.$$

Automat by je musel rozlišit. Dostáváme spor. □



Obrázek 3.12: Po slovech 0^i a 0^j je automat ve stejném stavu, takže stejný konec 1^i už nedokáže rozlišit.

3.5 Regulární výrazy

Automat je názorný, ale pro větší jazyky se rychle rozroste. Regulární výraz popisuje tentýž druh jazyků stručněji: skládá malé jazyky pomocí sjednocení, zřetězení a Kleeneho hvězdy.

Definice 3.5.1 (Regulární výraz). Mějme abecedu Σ . Množinu regulárních výrazů nad Σ , značenou $\text{RegE}(\Sigma)$, definujeme následovně:

1. $\emptyset$, λ a každý znak $a \in \Sigma$ jsou regulární výrazy;
2. jsou-li $R, S \in \text{RegE}(\Sigma)$, pak také

$$(R \mid S), \quad (RS), \quad (R^*)$$

jsou regulární výrazy;

3. nic dalšího regulárním výrazem není.

Každému regulárnímu výrazu přiřadíme jazyk:

$$\begin{aligned} L(\emptyset) &= \emptyset, & L(\lambda) &= \{\lambda\}, & L(a) &= \{a\}, \\ L(R \mid S) &= L(R) \cup L(S), & L(RS) &= L(R)L(S), & L(R^*) &= L(R)^*. \end{aligned}$$

Řekneme, že výraz R popisuje jazyk $L(R)$.

Závorky se obvykle vynechávají. Nejvyšší prioritu má hvězda, potom zřetězení a nakonec sjednocení. Výraz $01^* \mid 10$ tedy znamená $(0(1^*)) \mid (10)$.

Příklad 3.5.2. Nad abecedou $\Sigma = \{0, 1\}$ platí:

1. $(0 \mid 1)^*$ popisuje všechna binární slova;
2. $(0 \mid 1)^*01$ popisuje slova končící na 01;

3. $0^*10^*10^*$ popisuje slova s právě dvěma jedničkami;
4. $0^*(10^*10^*)^*$ popisuje slova se sudým počtem jedniček;
5. $(0 \mid 10)^*(\lambda \mid 1)$ popisuje slova bez dvou jedniček vedle sebe;
- 6.

$$0(0 \mid 1)^*0 \mid 1(0 \mid 1)^*1 \mid 0 \mid 1$$

popisuje neprázdná slova, která začínají a končí stejným znakem.



U složitějšího regulárního výrazu se vyplatí nejdřív slovně říct, co znamenají jeho části. Například v $0^*(10^*10^*)^*$ přidá každý průchod vnější hvězdou právě dvě jedničky; nuly mohou být kdekoliv mezi nimi.

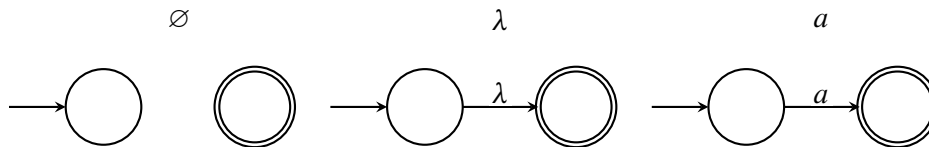
3.5.1 Kleeneho věta

Regulární výrazy nejsou jen pohodlnou zkratkou pro několik oblíbených jazyků. Popisují přesně totéž co konečné automaty.

Věta 3.5.3 (Kleeneho věta). Jazyk je regulární právě tehdy, když jej popisuje nějaký regulární výraz.

Důkaz. Dokážeme obě implikace konstrukcí.

Z regulárního výrazu na automat. Postupujeme indukcí podle stavby výrazu. Pro základní výrazy $\emptyset$, λ a $a \in \Sigma$ sestojíme λ -NFA na obrázku 3.13. Automat pro $\emptyset$ nemá mezi počátečním a přijímajícím stavem žádnou cestu, automat pro λ má jediný λ -přechod a automat pro a jediný přechod označený a .



Obrázek 3.13: Základní automaty pro výrazy $\emptyset$, λ a a .

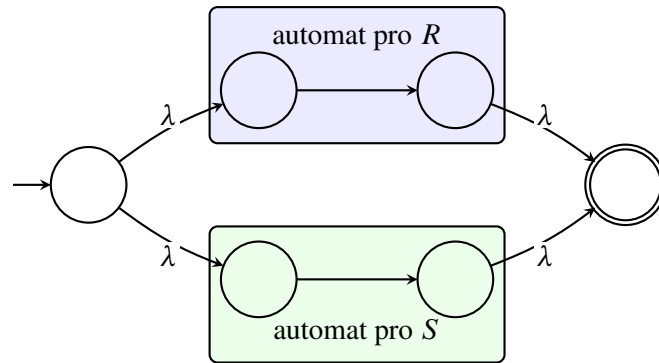
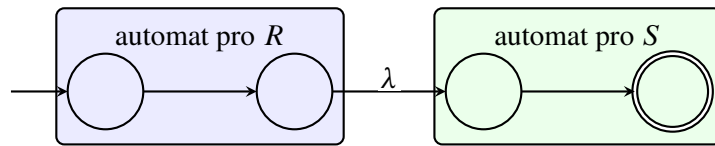
Předpokládejme nyní, že už máme automaty pro výrazy R a S . Jejich stavy případně přejmenujeme, aby se množiny stavů nepřekrývaly. Navíc můžeme bez újmy předpokládat, že každý z nich má právě jeden přijímající stav bez odchozích přechodů: stačí přidat nový přijímající stav a ze všech původních přijímajících stavů do něj vést λ -přechod.

Pro sjednocení $R \mid S$ přidáme nový počáteční a nový přijímající stav. Z nového začátku se automat λ -přechodem vydá do automatu pro R nebo S a z jejich konců přejde do nového přijímajícího stavu.

Pro zřetězení RS spojíme přijímající stav automatu pro R λ -přechodem s počátečním stavem automatu pro S . První část vstupu tedy musí přijmout automat pro R a zbytek automat pro S .

Pro R^* přidáme nový začátek a konec. Přímý λ -přechod mezi nimi přijme nula opakování. Další přechody umožní spustit automat pro R , po jednom průchodu skončit, nebo se vrátit a spustit jej znovu.

Každá konstrukce přijímá právě jazyk odpovídající operaci. Indukce tedy pro každý regulární výraz vytvoří ekvivalentní λ -NFA. Přechody bez čtení znaku umíme odstranit a podmnožinovou konstrukcí dostaneme DFA. Jazyk výrazu je proto regulární.

Obrázek 3.14: Konstrukce automatu pro sjednocení $R \mid S$.Obrázek 3.15: Konstrukce automatu pro zřetězení RS .

Z automatu na regulární výraz. Mějme DFA $A = (Q, \Sigma, \delta, q_s, F)$ a očísľujme jeho stavy

$$Q = \{q_1, \dots, q_n\}.$$

Pro dvojici stavů q_i, q_j a číslo $k \in \{0, \dots, n\}$ sestojíme regulární výraz $R_{ij}^{(k)}$. Bude popisovat právě slova, po jejichž přečtení se automat dostane z q_i do q_j a všechny mezilehlé stavy cesty patří do množiny $\{q_1, \dots, q_k\}$. Počáteční ani koncový stav se za mezilehlý nepovažují.

Pro $k = 0$ nesmíme použít žádný mezilehlý stav. Výraz $R_{ij}^{(0)}$ proto vytvoříme jako sjednocení všech znaků a , pro které $\delta(q_i, a) = q_j$. Je-li $i = j$, přidáme do sjednocení také λ ; není-li co sjednotit, použijeme $\emptyset$.

Pro $k \geq 1$ položíme

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}. \quad (3.1)$$

Každá cesta povolená pro $R_{ij}^{(k)}$ buď stav q_k jako mezilehlý vůbec nepoužije, a pak ji popisuje první člen, nebo jej použije. Ve druhém případě cestu rozřizneme při prvním příchodu do q_k a při posledním odchodu z něj. Dostaneme cestu z q_i do q_k , libovolný počet návratů z q_k do něj samého a cestu z q_k do q_j ; žádná z těchto částí už nemá q_k uvnitř. Právě to popisuje druhý člen vztahu (3.1). Obráceně lze části popsané kterýmkoliv členem vždy spojit v povolenou cestu. Rekurence je tedy správná.

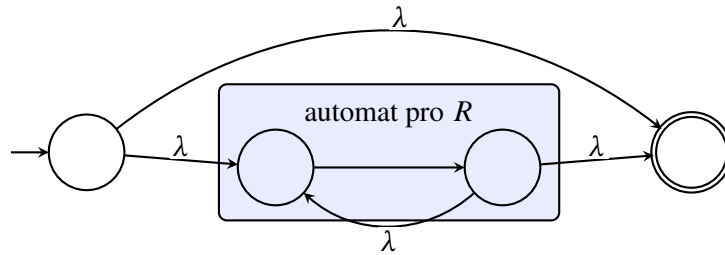
Po dosazení $k = n$ dovolíme jako mezilehlé všechny stavy automatu. Hledaný regulární výraz je sjednocení

$$\bigcup_{q_j \in F} R_{s_j}^{(n)},$$

kde q_s je počáteční stav. Je-li $F = \emptyset$, vezmeme místo sjednocení výraz $\emptyset$. Jeho jazyk tvoří právě slova, která dovedou automat z počátečního stavu do některého přijímajícího stavu. Je tedy roven $L(A)$. $\square$



Obě poloviny důkazu jsou zároveň algoritmy. Z výrazu umíme mechanicky postavit automat a z automatu zase výraz. Výsledek může být mnohem větší než zadání; věta slibuje existenci a postup, nikoliv vždy hezkému výsledku.

Obrázek 3.16: Konstrukce automatu pro Kleeneho hvězdu R^* .

Regulární výrazy v praxi. Programovací jazyky a nástroje pro hledání v textu často používají rozšířené regulární výrazy. Zápisy jako $[0-9]$ nebo $+$ bývají jen zkratkami pro naše tři operace. Některé nástroje však přidávají třeba zpětné odkazy na dříve nalezenou část textu. Takový „regex“ už může popisovat i neregulární jazyk, a není tedy regulárním výrazem ve smyslu této kapitoly.

3.6 Gramatiky

Automat dostane hotové slovo a rozhodne, zda do jazyka patří. Gramatika jde opačným směrem: z počátečního symbolu slova jazyka postupně vyrábí.

Definice 3.6.1 (Gramatika). *Gramatika* je čtveřice

$$G = (V, T, P, S),$$

kde

- V je konečná množina *neterminálů*,
- T je konečná množina *terminálů*, přičemž $V \cap T = \emptyset$,
- P je konečná množina *přepisovacích pravidel*. Pravidlo zapisujeme ve tvaru $\alpha \rightarrow \beta$: na levé straně stojí řetězec terminálů a neterminálů obsahující alespoň jeden neterminál, na pravé straně může stát libovolný řetězec terminálů a neterminálů, případně prázdné slovo λ ,
- $S \in V$ je *počáteční neterminál*.

Levá strana pravidla tedy musí obsahovat alespoň jeden neterminál, pravá strana může být i prázdné slovo λ . Terminály už dále nepřepisujeme; budou tvořit výsledné slovo.

Definice 3.6.2 (Odvození a jazyk gramatiky). Řekneme, že se slovo $u\alpha v$ *přímo odvodí* na $u\beta v$, a píšeme

$$u\alpha v \Rightarrow_G u\beta v,$$

jestliže $\alpha \rightarrow \beta$ je pravidlo gramatiky. Zápis

$$x \Rightarrow_G^* y$$

znamená, že z x lze dostat y pomocí nula nebo více přímých odvození.

Jazyk generovaný gramatikou G je

$$L(G) = \{w \in T^* : S \Rightarrow_G^* w\}.$$

Příklad 3.6.3 (Všechna binární slova). Gramatika

$$S \rightarrow 0S \mid 1S \mid \lambda$$

generuje jazyk $\{0, 1\}^*$. Při každém kroku přidá na konec nulu nebo jedničku a pravidlem $S \rightarrow \lambda$ skončí. Například

$$S \Rightarrow 1S \Rightarrow 10S \Rightarrow 101S \Rightarrow 101.$$

Příklad 3.6.4 (Stejný počet blokových znaků). Gramatika

$$S \rightarrow 0S1 \mid \lambda$$

generuje jazyk

$$\{0^n 1^n : n \geq 0\}.$$

Každé použití prvního pravidla přidá jednu nulu vlevo a jednu jedničku vpravo. Třeba

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111.$$

Tento jazyk není regulární, takže gramatiky dovedou popisovat i jazyky, na které konečné automaty nestačí.

Příklad 3.6.5 (Dva nezávislé bloky). Gramatika

$$S \rightarrow aS \mid B, \quad B \rightarrow bB \mid \lambda$$

generuje $\{a^n b^m : n, m \geq 0\}$. Po přechodu z S na B už nelze přidat žádné další a , ale počty obou druhů znaků na sobě nezávisí.

Příklad 3.6.6 (Správně uzávorkované výrazy). Gramatika

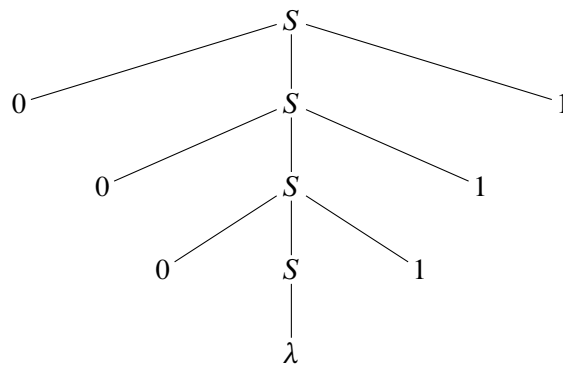
$$S \rightarrow SS \mid (S) \mid \lambda$$

generuje správně uzávorkovaná slova. Pravidlo $S \rightarrow (S)$ vloží již hotový výraz do závorek a $S \rightarrow SS$ spojí dva výrazy za sebe. Například

$$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()((S)) \Rightarrow ()(()).$$

3.6.1 Derivační stromy

U gramatik, jejichž pravidla mají na levé straně jediný neterminál, můžeme průběh odvození znázornit stromem. Takový tvar mají všechna pravidla v našich dosavadních příkladech. Kořenem je počáteční neterminál. Použijeme-li pravidlo $A \rightarrow x_1 \cdots x_k$, přidáme vrcholu A zleva doprava syny $x_1, \dots, x_k$. Terminály v listech přečtené zleva doprava vytvoří výsledné slovo.



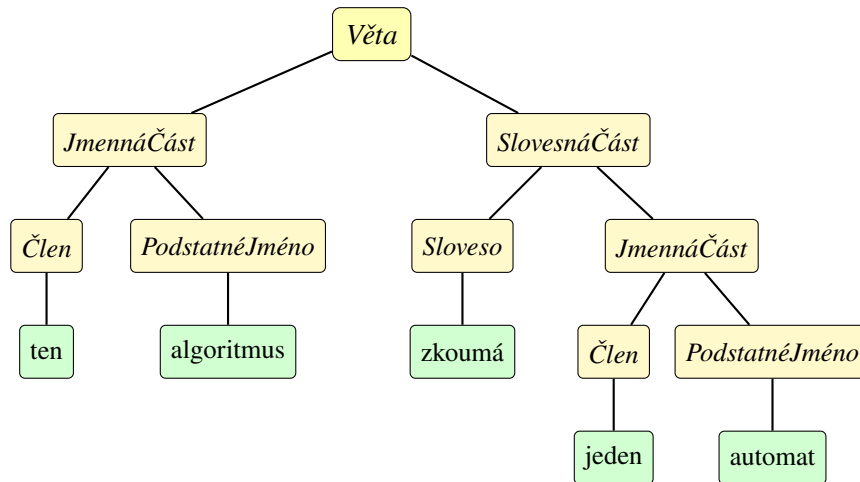
Obrázek 3.17: Derivační strom pro slovo 000111 v gramatice $S \rightarrow 0S1 \mid \lambda$.

Pořadí přepisování neterminálů se může lišit, aniž by se změnil derivační strom. U některých gramatik však může mít totéž slovo dva skutečně různé derivační stromy. Takovou gramatiku nazýváme *nejednoznačnou*. Třeba gramatika $S \rightarrow SS \mid a$ odvodí slovo aaa seskupením $(aa)a$ i $a(aa)$.

Příklad z přirozeného jazyka. Gramatika může popisovat i jednoduché věty:

$$\begin{aligned} Věta &\rightarrow JmennáČást\ SlovesnáČást, \\ JmennáČást &\rightarrow Člen\ PodstatnéJméno, \\ SlovesnáČást &\rightarrow Sloveso\ JmennáČást, \\ Člen &\rightarrow \text{ten} \mid \text{jeden}, \\ PodstatnéJméno &\rightarrow \text{algoritmus} \mid \text{automat}, \\ Sloveso &\rightarrow \text{zkoumá} \mid \text{napodobuje}. \end{aligned}$$

Vygeneruje například větu „ten algoritmus zkoumá jeden automat“. Jde samozřejmě o hodně zjednodušený model češtiny; ukazuje ale, že pravidla určují nejen použitá slova, nýbrž i jejich stavbu.



Obrázek 3.18: Derivační strom věty „ten algoritmus zkoumá jeden automat“.

3.6.2 Regulární gramatiky

Definice 3.6.7 (Regulární gramatika). Gramatika $G = (V, T, P, S)$ je *regulární*, pokud má každé její pravidlo jeden z tvarů

$$A \rightarrow wB \quad \text{nebo} \quad A \rightarrow w,$$

kde $A, B \in V$ a $w \in T^*$.

Na pravé straně je tedy nejvýše jeden neterminál a ten navíc stojí až na konci. Gramatika pro $0^n 1^n$ regulární není: pravidlo $S \rightarrow 0S1$ má terminál také za neterminálem. Naproti tomu gramatika pro $a^n b^m$ z předchozího příkladu regulární je.

Věta 3.6.8. Jazyk je regulární právě tehdy, když jej generuje nějaká regulární gramatika.

Důkaz. Opět ukážeme konstrukci v obou směrech.

Z DFA na gramatiku. Mějme DFA $A = (Q, \Sigma, \delta, q_0, F)$. Pro každý stav $q \in Q$ vytvoříme stejnojmenný neterminál, terminály budou znaky z Σ a počátečním neterminálem bude q_0 . Za každý přechod $\delta(q, a) = r$ přidáme pravidlo

$$q \rightarrow ar$$

a za každý přijímající stav $f \in F$ pravidlo $f \rightarrow \lambda$. Všechna pravidla jsou regulární.

Každý krok automatu po znaku a odpovídá použití jednoho pravidla $q \rightarrow ar$. Slovo lze dovést z q_0 do přijímajícího stavu právě tehdy, když jej gramatika může vypsát a zbývající přijímající neterminál nakonec odstranit pravidlem $f \rightarrow \lambda$. Jazyky automatu a gramatiky jsou proto stejné.

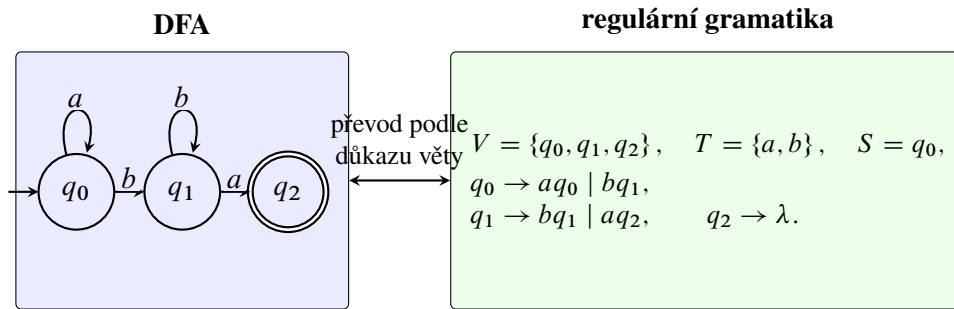
Z gramatiky na automat. Mějme regulární gramatiku $G = (V, T, P, S)$. Neterminály použijeme jako stavy λ -NFA, stav S bude počáteční a přidáme jediný nový přijímající stav f .

Pro pravidlo

$$A \rightarrow a_1 a_2 \cdots a_k B$$

vytvoříme cestu ze stavu A do stavu B označenou postupně $a_1, \dots, a_k$. Je-li $k > 1$, vložíme na cestu nové pomocné stavy, které nepoužijeme u žádného jiného pravidla. Pro $k = 0$, tedy pro pravidlo $A \rightarrow B$, přidáme λ -přechod. Stejně zpracujeme pravidlo $A \rightarrow a_1 \cdots a_k$, jen cesta skončí v f ; pravidlo $A \rightarrow \lambda$ tedy vytvoří λ -přechod z A do f .

Použití pravidla v odvození přesně odpovídá průchodu po jemu přidělené cestě. Gramatika proto odvodí terminální slovo w právě tehdy, když sestrojený λ -NFA po přečtení w dojde do f . λ -NFA umíme převést na DFA, takže $L(G)$ je regulární. $\square$

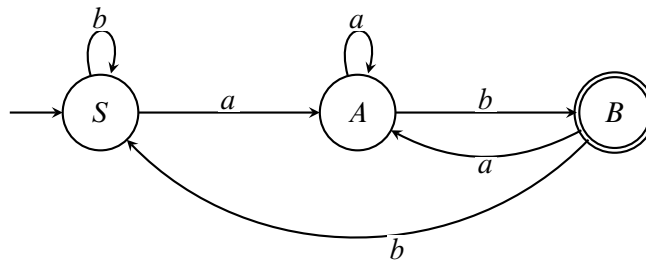


Obrázek 3.19: Ukázka převodu DFA na ekvivalentní regulární gramatiku.

Jeden jazyk třemi způsoby. Jazyk všech slov nad $\{a, b\}$ končících na ab můžeme popsat regulárním výrazem

$$(a \mid b)^* ab.$$

Přijímá jej také automat na obrázku 3.20.



Obrázek 3.20: DFA pro slova končící na ab .

Stejný jazyk generuje regulární gramatika

$$\begin{aligned} S &\rightarrow aA \mid bS, \\ A &\rightarrow aA \mid bB, \\ B &\rightarrow aA \mid bS \mid \lambda. \end{aligned}$$

Neterminály S, A, B odpovídají stavům automatu. Každé pravidlo se znakem opisuje jeden přechod a pravidlo $B \rightarrow \lambda$ označuje přijímající stav.



Automat se hodí, když chceme slova rychle rozpoznávat. Regulární výraz bývá nejkratším popisem a gramatika je přirozená při generování. Kleeneho věta a věta 3.6.8 říkají, že u regulárních jazyků si můžeme vybrat ten pohled, který se zrovna hodí nejvíc.

Kompilátory

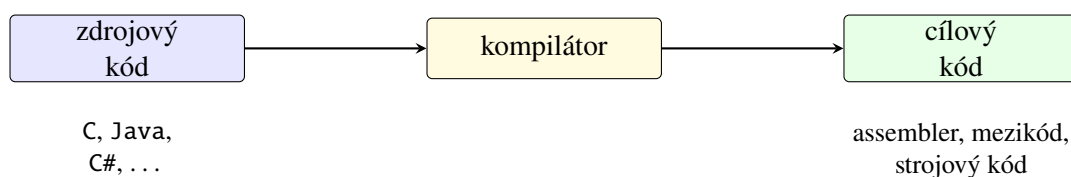
V kapitole 3 jsme se naučili popisovat jazyky pomocí konečných automatů, regulárních výrazů a gramatik. Zatím šlo hlavně o abstraktní modely. Nyní uvidíme jednu z jejich nejdůležitějších praktických aplikací: překlad programovacích jazyků. Regulární výrazy a konečné automaty nám pomohou rozdělit zdrojový kód na jednotlivé symboly, gramatiky potom popíší, jak z nich lze sestavit příkazy a výrazy.

Překladač ovšem nekončí u ověření správného zápisu. Musí také zjistit, zda program dává smysl, převést jej do vhodné vnitřní reprezentace, případně jej optimalizovat a nakonec vytvořit kód, který lze vykonat. V této kapitole si celý postup ukážeme na malém jazyce implementovaném v Pythonu.

4.1 Úvodem

Kompilátor je program, který překládá program zapsaný ve zdrojovém jazyce do ekvivalentního programu v cílovém jazyce. Vstup obvykle nazýváme *zdrojovým kódem* a výstup *cílovým kódem*.

Cílovým jazykem nemusí být přímo strojový kód konkrétního procesoru. Překladač může vytvářet assembler, bajtkód pro virtuální stroj, jiný programovací jazyk nebo obecnější mezikód. Podstatné je, aby se zachoval význam programu: pro stejné vstupy má cílový program poskytovat stejné výsledky jako program zdrojový.



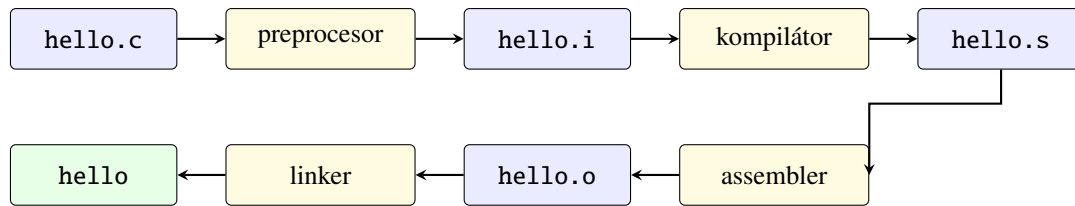
Obrázek 4.1: Základní pohled na kompilátor.

4.1.1 Překladačový řetězec

Slovem *kompilátor* někdy neformálně označujeme celý systém, který vytvoří spustitelný program. Ve skutečnosti může být práce rozdělena mezi několik nástrojů. Typický překlad jednoduchého programu v jazyce C znázorňuje obrázek 4.2.

Jednotlivé části mají následující úlohy:

- *preprocesor* zpracovává direktivy, například vložení hlavičkového souboru pomocí `#include`;



Obrázek 4.2: Zjednodušený překladačový řetězec jazyka C.

- vlastní *kompilátor* převede předzpracovaný program do assembleru;
- *assembler* vytvoří objektový soubor se strojovými instrukcemi a pomocnými informacemi;
- *linker* neboli sestavovací program spojí objektové soubory a potřebné knihovny do výsledného programu.

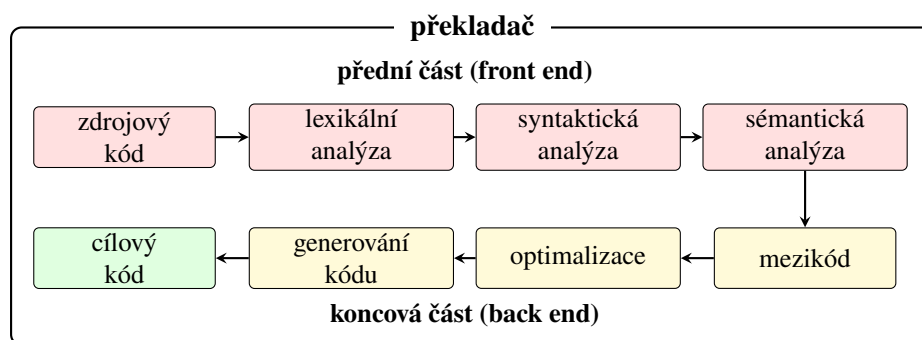
Uživatel obvykle spustí jediný ovladač překladače, například `gcc`, který jednotlivé nástroje zavolá za něj. Rozdělení je však důležité při hledání chyb: chyba kompilace, chybějící symbol při linkování a chyba vzniklá až za běhu programu jsou tři různé situace.

Strojový kód a mezikód. *Strojový kód* obsahuje instrukce konkrétního procesoru. Procesor jej může vykonávat přímo, ale stejný soubor zpravidla nelze přenést na počítač s jinou instrukční sadou nebo jiným operačním systémem.

Mezikód je vnitřní reprezentace programu ležící mezi zdrojovým a strojovým kódem. Může být navržen hlavně pro další analýzu a optimalizaci nebo jako přenositelný bajtkód pro virtuální stroj. Díky mezikódu nemusíme pro každou dvojici zdrojového jazyka a cílové platformy vytvářet celý překladač znovu: přední část přeloží různé jazyky do společné reprezentace a koncová část ji převede pro konkrétní počítač.

4.1.2 Fáze překladače

Překladač zdrojový program obvykle nezpracuje jediným obřím krokem. Postupně jej převádí do reprezentací, se kterými se stále snáze pracuje.



Obrázek 4.3: Základní fáze překladače.

1. *Lexikální analýza* seskupí znaky do tokenů.
2. *Syntaktická analýza* z tokenů sestaví strom podle gramatiky.
3. *Sémantická analýza* zkontroluje například deklarace, typy a platnost operací.
4. Z ověřeného stromu vznikne *mekikód*.
5. *Optimalizace* mezikód upraví, aniž by změnila význam programu.
6. *Generování kódu* vytvoří cílový kód.

První tři fáze tvoří *přední část překladače* (anglicky *front end*). Ta závisí hlavně na zdrojovém jazyce. Optimalizace a generování cílového kódu se obvykle řadí do *koncové části* (anglicky *back end*), která více závisí na cílové platformě. Hranice ale není u všech překladačů úplně stejná.



Jednotlivé fáze nemusí odpovídat samostatným programům ani jedinému průchodu zdrojovým textem. Jde především o logické rozdělení odpovědností. Praktický překladač může několik fází spojit nebo některou z nich provést vícekrát.

Průběžný příklad. V dalších sekcích vytvoříme malý jazyk s číselnými výrazy, deklarací proměnné příkazem `let` a výpisem příkazem `print`. Program

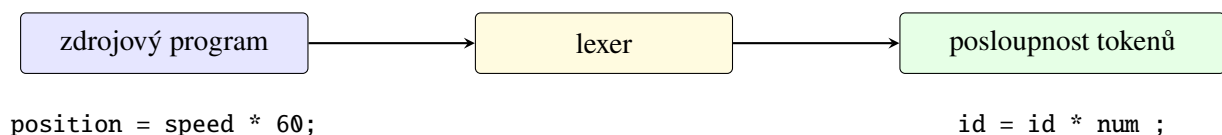
```
let x = 2 + 3 * 4;
print x;
```

má vypsát číslo 14. Překladač jej převede do jednoduchého zásobníkového bajtkódu. Ten potom vykoná náš virtuální stroj. Příklad je záměrně malý, ale projde všemi hlavními fázemi z obrázku 4.3.

4.2 Lexikální analýza

Zdrojový soubor je pro počítač nejprve jen posloupností znaků. Syntaktický analyzátor by však pracoval velmi nepohodlně, kdyby musel znovu a znovu rozpoznávat, které znaky tvoří jméno proměnné, číslo nebo operátor. Tuto práci proto oddělíme.

Lexikální analyzátor neboli *lexer* čte znaky zdrojového programu a seskupuje je do posloupnosti *tokenů*. Každý token určuje druh rozpoznané části programu a může nést také její hodnotu.



Obrázek 4.4: Vstup a výstup lexikálního analyzátoru.

4.2.1 Lexémy a tokeny

Lexém je konkrétní úsek zdrojového textu. *Token* je informace, kterou o něm lexer předá další fázi. Pro příkaz

```
position = speed * 60;
```

můžeme získat například následující tokeny:

lexém	druh tokenu	předaná hodnota
position	identifikátor	position
=	přiřazení	—
speed	identifikátor	speed
*	násobení	—
60	číslo	60
;	středník	—

U identifikátoru potřebuje parser obvykle znát jeho text, u číselného literálu zase číselnou hodnotu. U operátoru + často stačí samotný druh tokenu. Lexer si navíc obvykle ukládá pozici lexému, aby později mohl vypsat srozumitelnou chybovou zprávu.

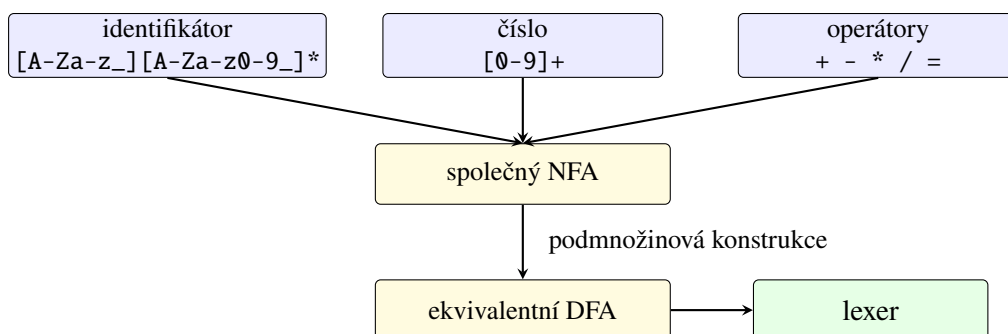
4.2.2 Od regulárního výrazu k lexeru

Typické lexémy mají pravidelnou stavbu:

- identifikátor může být popsán zkratkou $[A-Za-z_][A-Za-z0-9_]*$;
- celé nezáporné číslo zápisem $[0-9]^+$;
- mezery a konce řádků tvoří jazyk, který lexer rozpozná, ale nepředá parseru;
- každý pevný operátor, například + nebo =, tvoří jednoprvkový jazyk.

Zápisy v hranatých závorkách a symbol + jsou běžné zkratky rozšířených regulárních výrazů: $[0-9]$ znamená volbu jedné číslice a R^+ jedno nebo více opakování výrazu R . Pomocí základních operací zavedených v sekci 3.5 je lze rozepsat pouze pomocí sjednocení, zřetězení a Kleeneho hvězdy.

Pro každý druh tokenu můžeme sestavit konečný automat. Jejich počáteční stavy spojíme novým počátečním stavem pomocí λ -přechodů, čímž získáme společný NFA. Pomocí věty 3.4.6 jej převedeme na DFA, který pak čte zdrojový text znak po znaku.



Obrázek 4.5: Využití regulárních výrazů a automatů při konstrukci lexeru.

Nejdelší lexém a priorita. Při čtení vstupu může automat projít přijímajícím stavem a později přijmout ještě delší část. Lexer proto obvykle používá pravidlo *nejdelšího lexému*: pokračuje, dokud lze získat delší platný lexém, a vrátí poslední přijatou možnost. Ze vstupu `abc123` tak vznikne jeden identifikátor, nikoliv šest jednopísmenných tokenů.

Pokud stejný lexém vyhovuje více pravidlům, rozhodne jejich priorita. Slovo `let` například splňuje obecný předpis identifikátoru, ale chceme jej rozpoznat jako klíčové slovo. Podobně musí lexer před jedním znakem `=` upřednostnit dvouznakový operátor `==`, pokud jej jazyk obsahuje.



Lexer nerozhoduje, zda tokeny tvoří správný program. Posloupnost `let + ; 7` lze bez problému rozdělit na tokeny, ale její syntaktickou chybu odhalí až parser.

Implementace tokenů. V našem jazyce použijeme dva druhy příkazů a výrazy s identifikátory, čísla a čtyřmi aritmetickými operátory. Potřebujeme také tokeny pro přiřazení, středník a závorky. Jejich druhy a samotný token zachycuje program 4.2.1.

Program 4.2.1: Druhy tokenů a datová třída tokenu.

```

1 from dataclasses import dataclass
2 from enum import Enum, auto
3
4
5 class TokenKind(Enum):
6     LET = auto()
7     PRINT = auto()
8     IDENTIFIER = auto()
9     NUMBER = auto()
10    PLUS = auto()
11    MINUS = auto()
12    STAR = auto()
13    SLASH = auto()
14    EQUAL = auto()
15    SEMICOLON = auto()
16    LEFT_PAREN = auto()
17    RIGHT_PAREN = auto()
18    EOF = auto()
19
20
21 KEYWORDS = {
22     "let": TokenKind.LET,
23     "print": TokenKind.PRINT,
24 }
25
26
27 @dataclass(frozen=True)
28 class Token:
29     kind: TokenKind
30     lexeme: str
31     literal: float | None
32     position: int

```

Implementace lexeru. Lexer v programu 4.2.2 prochází vstup zleva doprava. Čísla a identifikátory čte pomocí samostatných metod, jednoznakové symboly vyhledává ve slovníku. Bílé znaky ignoruje a text od `//` do konce řádku považuje za komentář.

Program 4.2.2: Jednoduchý lexikální analyzátor.

```

1 from .tokens import KEYWORDS, Token, TokenKind
2
3
4 class LexerError(ValueError):
5     pass
6
7
8 class Lexer:

```

```
9 SINGLE_CHAR_TOKENS = {
10     "+": TokenKind.PLUS,
11     "-": TokenKind.MINUS,
12     "*": TokenKind.STAR,
13     "/": TokenKind.SLASH,
14     "=": TokenKind.EQUAL,
15     ";": TokenKind.SEMICOLON,
16     "(": TokenKind.LEFT_PAREN,
17     ")": TokenKind.RIGHT_PAREN,
18 }
19
20 def __init__(self, source: str) -> None:
21     self.source = source
22     self.current = 0
23     self.tokens: list[Token] = []
24
25 def scan_tokens(self) -> list[Token]:
26     while not self._at_end():
27         start = self.current
28         char = self._advance()
29
30         if char.isspace():
31             continue
32         if char == "/" and self._peek() == "/":
33             self._skip_comment()
34         elif char.isdigit():
35             self._scan_number(start)
36         elif char.isalpha() or char == "_":
37             self._scan_identifier(start)
38         elif char in self.SINGLE_CHAR_TOKENS:
39             self._add(self.SINGLE_CHAR_TOKENS[char], start)
40         else:
41             raise LexerError(
42                 f"Unexpected character {char!r} at position {start}."
43             )
44
45     self.tokens.append(
46         Token(TokenKind.EOF, "", None, self.current)
47     )
48     return self.tokens
49
50 def _scan_number(self, start: int) -> None:
51     while self._peek().isdigit():
52         self._advance()
53
54     if self._peek() == "." and self._peek_next().isdigit():
55         self._advance()
56         while self._peek().isdigit():
57             self._advance()
58
59     lexeme = self.source[start:self.current]
60     self.tokens.append(
61         Token(TokenKind.NUMBER, lexeme, float(lexeme), start)
62     )
63
64 def _scan_identifier(self, start: int) -> None:
65     while self._peek().isalnum() or self._peek() == "_":
66         self._advance()
```

```

67     lexeme = self.source[start:self.current]
68     kind = KEYWORDS.get(lexeme, TokenKind.IDENTIFIER)
69     self.tokens.append(Token(kind, lexeme, None, start))
70
71
72     def _skip_comment(self) -> None:
73         while not self._at_end() and self._peek() != "\n":
74             self._advance()
75
76     def _add(self, kind: TokenKind, start: int) -> None:
77         lexeme = self.source[start:self.current]
78         self.tokens.append(Token(kind, lexeme, None, start))
79
80     def _advance(self) -> str:
81         char = self.source[self.current]
82         self.current += 1
83         return char
84
85     def _peek(self) -> str:
86         if self._at_end():
87             return "\0"
88         return self.source[self.current]
89
90     def _peek_next(self) -> str:
91         if self.current + 1 >= len(self.source):
92             return "\0"
93         return self.source[self.current + 1]
94
95     def _at_end(self) -> bool:
96         return self.current >= len(self.source)

```

Pro začátek programu `let x = 2 + 3 * 4;` vrátí lexer druhy tokenů

```

LET, IDENTIFIER, EQUAL, NUMBER, PLUS,
NUMBER, STAR, NUMBER, SEMICOLON, EOF.

```

Zvláštní token EOF označuje konec vstupu a parseru usnadní rozhodování, zda už přečetl celý program.

4.3 Syntaktická analýza

Lexer už zdrojový text rozdělil na tokeny. Nyní potřebujeme rozhodnout, zda jsou seřazeny podle pravidel programovacího jazyka, a zachytit jejich strukturu.

Syntaktický analyzátor neboli *parser* dostane posloupnost tokenů a podle gramatiky rozhodne, zda tvoří syntakticky správný program. Jeho výstupem bývá derivační nebo syntaktický strom, případně zpráva o syntaktické chybě.

4.3.1 Bezkontextové gramatiky

Obecná gramatika zavedená v sekci 3.6 dovoluje mít na levé straně pravidla celý řetězec. Syntaxi programovacích jazyků však obvykle popisujeme jednodušším druhem gramatik.

Definice 4.3.1 (Bezkontextová gramatika). Gramatika $G = (V, T, P, S)$ je *bezkontextová*, pokud má každé pravidlo tvar

$$A \rightarrow \alpha,$$

kde $A \in V$ je jediný neterminál a $\alpha \in (V \cup T)^*$.

Název vyjadřuje, že neterminál A smíme přepsat bez ohledu na symboly stojící kolem něj. Regulární gramatiky jsou zvláštním případem bezkontextových gramatik, ale opačná inkluze neplatí. Například pravidlo $S \rightarrow aSb$ je bezkontextové, nikoliv však regulární.

4.3.2 Aritmetické výrazy

Prioritu běžných aritmetických operátorů zachycuje gramatika

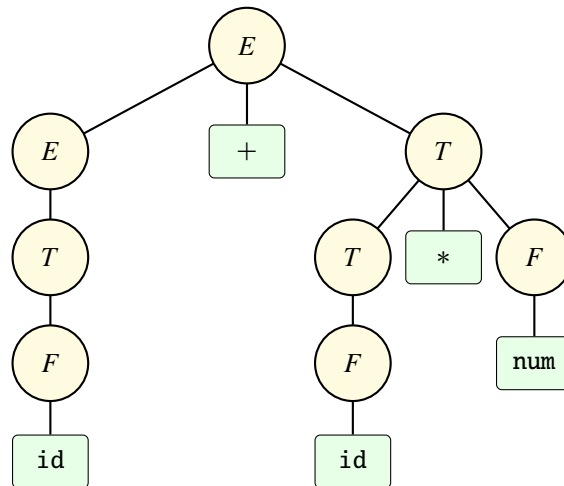
$$E \rightarrow E + T \mid E - T \mid T,$$

$$T \rightarrow T * F \mid T / F \mid F,$$

$$F \rightarrow (E) \mid \text{id} \mid \text{num}.$$

Neterminál E představuje výraz, T term a F faktor. Násobení a dělení se objeví hlouběji ve stromu než sčítání a odčítání, a proto se vyhodnotí dříve. Strom na obrázku 4.6 odpovídá posloupnosti tokenů

`id + id * num.`



Obrázek 4.6: Derivační strom aritmetického výrazu.

Derivační strom, se kterým jsme se poprvé setkali v sekci 3.6, obsahuje i neterminály a pomocné kroky gramatiky. Překladač si často vytvoří úspornější *abstraktní syntaktický strom* (zkráceně AST), v němž zůstanou jen informace potřebné pro další zpracování. Ve stromu výrazu $2 + 3 * 4$ bude kořenem sčítání, jeho levým synem číslo 2 a pravým synem násobení čísel 3 a 4.

4.3.3 Analýza shora dolů a zdola nahoru

Parser *shora dolů* začíná počátečním neterminálem a volí pravidla tak, aby vytvořil vstupní posloupnost tokenů. Strom tedy staví od kořene k listům. Jednoduchým příkladem je rekurzivní sestup, v němž každému neterminálu odpovídá jedna funkce.

Parser *zdola nahoru* naopak začíná tokeny. Hledá úseky odpovídající pravým stranám pravidel a nahrazuje je jejich levou stranou. Strom staví od listů ke kořeni. Praktické generátory parserů často používají některou z variant této metody.

Levá rekurze v pravidle $E \rightarrow E + T$ by pro přímočarý rekurzivní sestup vedla k nekonečnému volání funkce pro E . Pro implementaci proto použijeme ekvivalentní tvar

$$\begin{aligned} E &\rightarrow TE', & E' &\rightarrow +TE' \mid -TE' \mid \lambda, \\ T &\rightarrow FT', & T' &\rightarrow *FT' \mid /FT' \mid \lambda, \\ F &\rightarrow (E) \mid \text{id} \mid \text{num}. \end{aligned}$$

Opakování v E' a T' lze v Pythonu přirozeně zapsat cyklem.

Gramatika malého jazyka. Celý průběžný jazyk popíšeme pravidly

$$\begin{aligned} \text{Program} &\rightarrow \text{Příkaz Program} \mid \lambda, \\ \text{Příkaz} &\rightarrow \text{let id} = E; \mid \text{print } E;, \end{aligned}$$

doplněnými předchozími pravidly pro E , T a F . Jazyk tedy dovoluje deklarovat číselnou proměnnou a vypsát hodnotu výrazu.

Datové třídy v programu 4.3.1 popisují AST. Společní předkové `Expr` a `Stmt` umožňují snadno rozlišovat výrazy a příkazy.

Program 4.3.1: Vrcholy abstraktního syntaktického stromu.

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from .tokens import TokenKind
6
7
8 class Expr:
9     pass
10
11
12 @dataclass(frozen=True)
13 class NumberExpr(Expr):
14     value: float
15
16
17 @dataclass(frozen=True)
18 class NameExpr(Expr):
19     name: str
20
21
22 @dataclass(frozen=True)
23 class BinaryExpr(Expr):
24     left: Expr
25     operator: TokenKind
26     right: Expr
27
28
29 class Stmt:
30     pass

```

```

31
32
33 @dataclass(frozen=True)
34 class LetStmt(Stmt):
35     name: str
36     initializer: Expr
37
38
39 @dataclass(frozen=True)
40 class PrintStmt(Stmt):
41     expression: Expr
42
43
44 @dataclass(frozen=True)
45 class Program:
46     statements: list[Stmt]

```

Parser v programu 4.3.2 přímo odpovídá gramatice. Metoda `_statement` rozpoznává oba druhy příkazů. Metody `_expression` a `_term` zpracovávají operátory, zatímco `_factor` číslo, identifikátor nebo závorku.

Program 4.3.2: Parser metodou rekurzivního sestupu.

```

1  from .ast_nodes import (
2      BinaryExpr,
3      Expr,
4      LetStmt,
5      NameExpr,
6      NumberExpr,
7      PrintStmt,
8      Program,
9      Stmt,
10 )
11 from .tokens import Token, TokenKind
12
13
14 class ParserError(ValueError):
15     pass
16
17
18 class Parser:
19     def __init__(self, tokens: list[Token]) -> None:
20         self.tokens = tokens
21         self.current = 0
22
23     def parse(self) -> Program:
24         statements: list[Stmt] = []
25         while not self._check(TokenKind.EOF):
26             statements.append(self._statement())
27         return Program(statements)
28
29     def _statement(self) -> Stmt:
30         if self._match(TokenKind.LET):
31             name = self._consume(
32                 TokenKind.IDENTIFIER, "Expected a variable name."
33             )
34             self._consume(TokenKind.EQUAL, "Expected '='.")
35             initializer = self._expression()
36             self._consume(TokenKind.SEMICOLON, "Expected ';'.")

```

```

37         return LetStmt(name.lexeme, initializer)
38
39     if self._match(TokenKind.PRINT):
40         expression = self._expression()
41         self._consume(TokenKind.SEMICOLON, "Expected ';'")
42         return PrintStmt(expression)
43
44     raise self._error("Expected 'let' or 'print'")
45
46     def _expression(self) -> Expr:
47         expression = self._term()
48         while self._match(TokenKind.PLUS, TokenKind.MINUS):
49             operator = self._previous().kind
50             right = self._term()
51             expression = BinaryExpr(expression, operator, right)
52         return expression
53
54     def _term(self) -> Expr:
55         expression = self._factor()
56         while self._match(TokenKind.STAR, TokenKind.SLASH):
57             operator = self._previous().kind
58             right = self._factor()
59             expression = BinaryExpr(expression, operator, right)
60         return expression
61
62     def _factor(self) -> Expr:
63         if self._match(TokenKind.NUMBER):
64             literal = self._previous().literal
65             assert literal is not None
66             return NumberExpr(literal)
67
68         if self._match(TokenKind.IDENTIFIER):
69             return NameExpr(self._previous().lexeme)
70
71         if self._match(TokenKind.LEFT_PAREN):
72             expression = self._expression()
73             self._consume(TokenKind.RIGHT_PAREN, "Expected ')'")
74             return expression
75
76         raise self._error("Expected a number, name, or '('")
77
78     def _match(self, *kinds: TokenKind) -> bool:
79         for kind in kinds:
80             if self._check(kind):
81                 self.current += 1
82                 return True
83         return False
84
85     def _consume(self, kind: TokenKind, message: str) -> Token:
86         if self._check(kind):
87             self.current += 1
88             return self._previous()
89         raise self._error(message)
90
91     def _check(self, kind: TokenKind) -> bool:
92         return self._peek().kind is kind
93
94     def _peek(self) -> Token:

```

```

95         return self.tokens[self.current]
96
97     def _previous(self) -> Token:
98         return self.tokens[self.current - 1]
99
100    def _error(self, message: str) -> ParserError:
101        token = self._peek()
102        return ParserError(
103            f"{message} At position {token.position}, "
104            f"found {token.lexeme!r}."
105        )

```

Metoda `_consume` zároveň kontroluje povinný následující token. Chybí-li například středník, parser skončí s chybou obsahující pozici neočekávaného tokenu. Kvalitní praktický parser se navíc pokusí najít vhodné místo, od něhož může pokračovat a oznámit více chyb během jednoho překladu.

4.4 Sémantická analýza

Parser zaručuje, že program odpovídá gramatice. To ještě neznamená, že dává smysl. Příkaz `let x = ;` je syntakticky chybný, protože za rovnítkem chybí výraz. Příkaz `print y;` je naopak syntakticky správný, ale není možné jej vykonat, pokud proměnná `y` nebyla deklarována.

Sémantický analyzátor proto kontroluje podmínky, které nelze nebo není vhodné vyjadřovat samotnou gramatikou. Patří mezi ně zejména existence deklarací, viditelnost jmen, typová správnost výrazů a platnost jednotlivých operací.

4.4.1 Tabulka symbolů

Překladač si pro každý deklarovaný identifikátor uchovává informace v *tabulce symbolů*. U proměnné to může být jméno, typ, oblast platnosti a místo uložení; u funkce navíc typy parametrů a návratová hodnota.

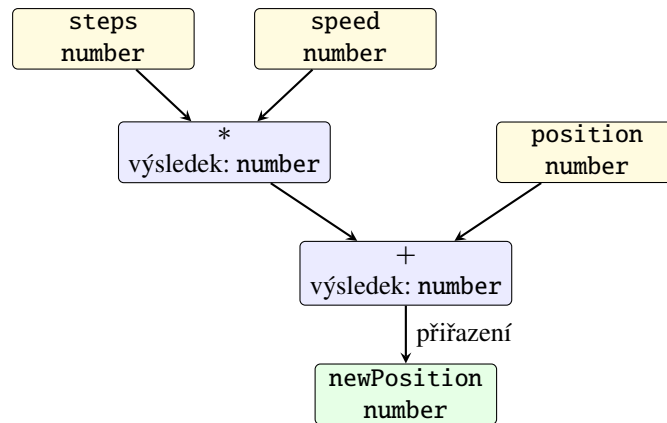
Při čtení deklarace překladač přidá nový záznam. Při každém použití identifikátoru hledá odpovídající záznam v aktuální a okolních oblastech platnosti. Díky tomu odhalí použití nedeklarované proměnné i nepovolenou opakovanou deklaraci.

4.4.2 Typová kontrola

Typová kontrola přiřazuje typy listům stromu a postupuje směrem ke kořeni. Je-li například `steps` číslo a `speed` číslo, výsledek jejich násobení je opět číslo. Součet s číselnou proměnnou `position` proto může být přiřazen do další číselné proměnné.

Jiný jazyk může některé kombinace automaticky převádět, například celé číslo na desetinné, zatímco další kombinace odmítne. Pravidla typového systému jsou proto součástí definice konkrétního jazyka, nikoliv univerzální vlastností všech překladačů.

Sémantická kontrola malého jazyka. Náš jazyk má jediný číselný typ. Nemusíme tedy porovnávat různé typy, ale stále musíme zkontrolovat deklarace. Analyzátor v programu 4.4.1 prochází příkazy v pořadí, ukládá deklarovaná jména do množiny a rekurzivně kontroluje výrazy.



Obrázek 4.7: Zjednodušená typová kontrola výrazu.

Program 4.4.1: Sémantická kontrola deklarací a použití proměnných.

```

1 from .ast_nodes import (
2     BinaryExpr,
3     Expr,
4     LetStmt,
5     NameExpr,
6     NumberExpr,
7     PrintStmt,
8     Program,
9 )
10
11
12 class SemanticError(ValueError):
13     pass
14
15
16 class SemanticAnalyzer:
17     def __init__(self) -> None:
18         self.declared_names: set[str] = set()
19
20     def analyze(self, program: Program) -> None:
21         for statement in program.statements:
22             if isinstance(statement, LetStmt):
23                 if statement.name in self.declared_names:
24                     raise SemanticError(
25                         f"Variable {statement.name!r} is already declared."
26                     )
27                 self._check_expr(statement.initializer)
28                 self.declared_names.add(statement.name)
29             elif isinstance(statement, PrintStmt):
30                 self._check_expr(statement.expression)
31
32     def _check_expr(self, expression: Expr) -> None:
33         if isinstance(expression, NumberExpr):
34             return
35         if isinstance(expression, NameExpr):
36             if expression.name not in self.declared_names:
37                 raise SemanticError(
38                     f"Variable {expression.name!r} is not declared."
39                 )
40         return

```

```

41     if isinstance(expression, BinaryExpr):
42         self._check_expr(expression.left)
43         self._check_expr(expression.right)
44         return
45     raise TypeError(f"Unknown expression: {expression!r}")

```

Inicializační výraz se kontroluje ještě před vložením nového jména do tabulky. Program `let x = x + 1`; proto nemůže použít právě deklarovanou proměnnou, dokud pro ni neexistuje předchozí hodnota. Po úspěšné kontrole v další fázi, že každé jméno odkazuje na existující proměnnou.



Syntaktický strom může být po sémantické analýze doplněn o typy, odkazy do tabulky symbolů a další atributy. Hovoříme pak o anotovaném stromu. Další fáze díky tomu nemusí stejné informace zjišťovat znovu.

4.5 Optimalizace

Po přední části máme ověřenou reprezentaci programu. Mohli bychom z ní rovnou vytvořit cílový kód, ale výsledek by často prováděl zbytečnou práci.

Optimalizace programu je úprava jeho vnitřní reprezentace, která zachová pozorovatelné chování programu a lepší zvolené vlastnosti výsledného kódu, například dobu běhu, velikost nebo spotřebu energie.

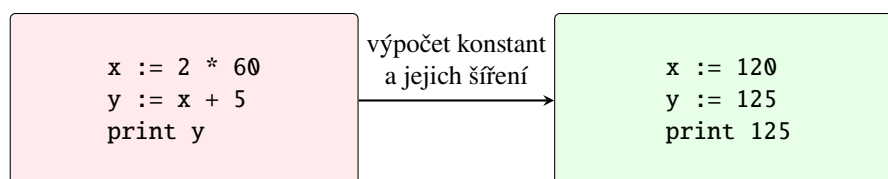
Slovo „optimalizace“ neznamena, že překladač vždy nalezne nejlepší možný program. Takový požadavek by byl příliš drahý a obecně i algoritmicky problematický. Překladač používá sadu transformací, které bývají výhodné a jejichž správnost umí zdůvodnit.

4.5.1 Lokální optimalizace

Základní blok je souvislá posloupnost instrukcí, do níž se vstupuje pouze na začátku a z níž se odchází pouze na konci. Uvnitř bloku se tedy tok řízení nevětví. *Lokální optimalizace* pracuje s jedním takovým blokem.

Mezi běžné lokální úpravy patří:

- *výpočet konstant* (anglicky *constant folding*) — výraz $2 \cdot 60$ lze spočítat už při překladu;
- *šíření konstant a kopií* — známe-li $x = 120$, můžeme následující použití x nahradit touto hodnotou;
- *odstranění mrtvého kódu* — výpočet, jehož výsledek se nikde nepoužije, může být zbytečný;
- *odstranění společných podvýrazů* — nezměnily-li se operandy, nemusíme stejný výraz počítat podruhé.



Obrázek 4.8: Příklad výpočtu a šíření konstant.



Transformace musí respektovat přesnou sémantiku jazyka. Nahrazení $0 \cdot E$ nulou například není bezpečné, pokud vyhodnocení E může mít vedlejší účinek nebo skončit chybou.

Výpočet konstant v AST. Optimizátor v programu 4.5.1 prochází strom zdola nahoru. Jsou-li oba operandy binární operace číselné konstanty, nahradí celý podstrom jediným číselným vrcholem. Dělení nulou záměrně ponechá beze změny, aby příslušná chyba vznikla až při vykonání programu.

Program 4.5.1: Optimalizace výpočtem konstant.

```

1 from .ast_nodes import (
2     BinaryExpr,
3     Expr,
4     LetStmt,
5     NumberExpr,
6     PrintStmt,
7     Program,
8 )
9 from .tokens import TokenKind
10
11
12 class ConstantFolder:
13     def optimize(self, program: Program) -> Program:
14         statements = []
15         for statement in program.statements:
16             if isinstance(statement, LetStmt):
17                 statements.append(
18                     LetStmt(
19                         statement.name,
20                         self._fold(statement.initializer),
21                     )
22                 )
23             elif isinstance(statement, PrintStmt):
24                 statements.append(
25                     PrintStmt(self._fold(statement.expression))
26                 )
27         return Program(statements)
28
29     def _fold(self, expression: Expr) -> Expr:
30         if not isinstance(expression, BinaryExpr):
31             return expression
32
33         left = self._fold(expression.left)
34         right = self._fold(expression.right)
35
36         if not isinstance(left, NumberExpr):
37             return BinaryExpr(left, expression.operator, right)
38         if not isinstance(right, NumberExpr):
39             return BinaryExpr(left, expression.operator, right)
40
41         a, b = left.value, right.value
42         if expression.operator is TokenKind.PLUS:
43             return NumberExpr(a + b)
44         if expression.operator is TokenKind.MINUS:
45             return NumberExpr(a - b)
46         if expression.operator is TokenKind.STAR:
47             return NumberExpr(a * b)
48         if expression.operator is TokenKind.SLASH and b != 0:
49             return NumberExpr(a / b)

```

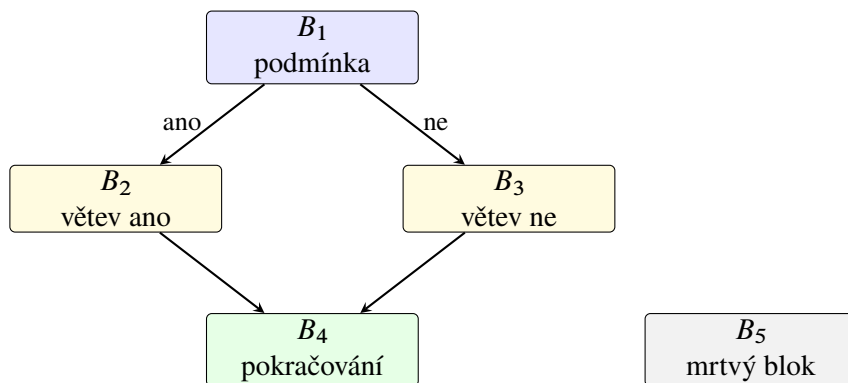
50

```
return BinaryExpr(left, expression.operator, right)
```

Ve výrazu $2 + 3 \cdot 4$ se nejprve podstrom $3 \cdot 4$ nahradí číslem 12 a potom celý součet číslem 14. Optimalizace tedy přirozeně využívá stromovou strukturu vytvořenou parserem.

4.5.2 Globální optimalizace a graf toku řízení

Optimalizace přes podmínky, cykly a více bloků potřebuje znát možné pořadí vykonávání programu. Základní bloky proto použijeme jako vrcholy *grafu toku řízení*. Hrana $B_i \rightarrow B_j$ znamená, že po bloku B_i může následovat blok B_j .



Obrázek 4.9: Příklad grafu toku řízení s nedosažitelným blokem B_5 .

Blok je dosažitelný, jestliže do něj vede cesta z počátečního bloku. Nedosažitelné bloky nelze za běhu navštívit a můžeme je odstranit. K jejich nalezení stačí BFS ze sekce 1.3 nebo DFS ze sekce 1.4.

Další důležitou informací je *živost proměnné*. Hodnota proměnné je za danou instrukcí živá, pokud může být později přečtena dříve, než ji přepíše jiná hodnota. Instrukce, která zapisuje mrtvou hodnotu a nemá jiný účinek, může být odstraněna.

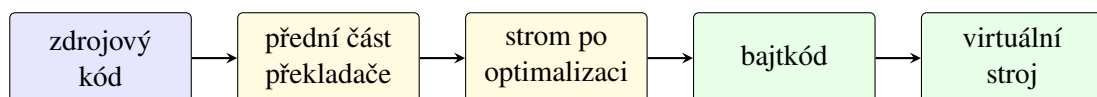
Ve zjednodušeném základním bloku lze živost spočítat průchodem instrukcí odzadu:

1. začneme množinou proměnných potřebných na konci bloku;
2. proměnnou, do které aktuální instrukce zapisuje, z množiny odstraníme;
3. proměnné, které instrukce čte, do množiny přidáme.

U celého grafu se informace mezi bloky předávají a výpočet se opakuje, dokud se množiny nepřestanou měnit.

4.5.3 Generování bajtkódu

Po optimalizaci převedeme AST do jednoduchého zásobníkového bajtkódu. Instrukce PUSH vloží konstantu na zásobník, LOAD načte proměnnou a instrukce ADD, SUB, MUL a DIV odeberou dva operandy a vloží výsledek. Příkazy završují instrukce STORE a PRINT.



Obrázek 4.10: Překlad malého jazyka do bajtkódu pro virtuální stroj.

Generátor v programu 4.5.2 prochází výraz v pořadí *levý podstrom, pravý podstrom, operace*. Tím automaticky vytvoří pořadí vhodné pro zásobníkový stroj.

Program 4.5.2: Generování jednoduchého zásobníkového bajtkódu.

```

1 from dataclasses import dataclass
2
3 from .ast_nodes import (
4     BinaryExpr,
5     Expr,
6     LetStmt,
7     NameExpr,
8     NumberExpr,
9     PrintStmt,
10    Program,
11 )
12 from .tokens import TokenKind
13
14
15 @dataclass(frozen=True)
16 class Instruction:
17     opcode: str
18     argument: float | str | None = None
19
20
21 class CodeGenerator:
22     OPERATORS = {
23         TokenKind.PLUS: "ADD",
24         TokenKind.MINUS: "SUB",
25         TokenKind.STAR: "MUL",
26         TokenKind.SLASH: "DIV",
27     }
28
29     def generate(self, program: Program) -> list[Instruction]:
30         code: list[Instruction] = []
31         for statement in program.statements:
32             if isinstance(statement, LetStmt):
33                 self._emit_expr(statement.initializer, code)
34                 code.append(Instruction("STORE", statement.name))
35             elif isinstance(statement, PrintStmt):
36                 self._emit_expr(statement.expression, code)
37                 code.append(Instruction("PRINT"))
38         code.append(Instruction("HALT"))
39         return code
40
41     def _emit_expr(
42         self, expression: Expr, code: list[Instruction]
43     ) -> None:
44         if isinstance(expression, NumberExpr):
45             code.append(Instruction("PUSH", expression.value))
46         elif isinstance(expression, NameExpr):
47             code.append(Instruction("LOAD", expression.name))
48         elif isinstance(expression, BinaryExpr):
49             self._emit_expr(expression.left, code)
50             self._emit_expr(expression.right, code)
51             code.append(Instruction(self.OPERATORS[expression.operator]))
52         else:
53             raise TypeError(f"Unknown expression: {expression!r}")

```

Po optimalizaci se příkaz `let x = 2 + 3 * 4;` přeloží jen na

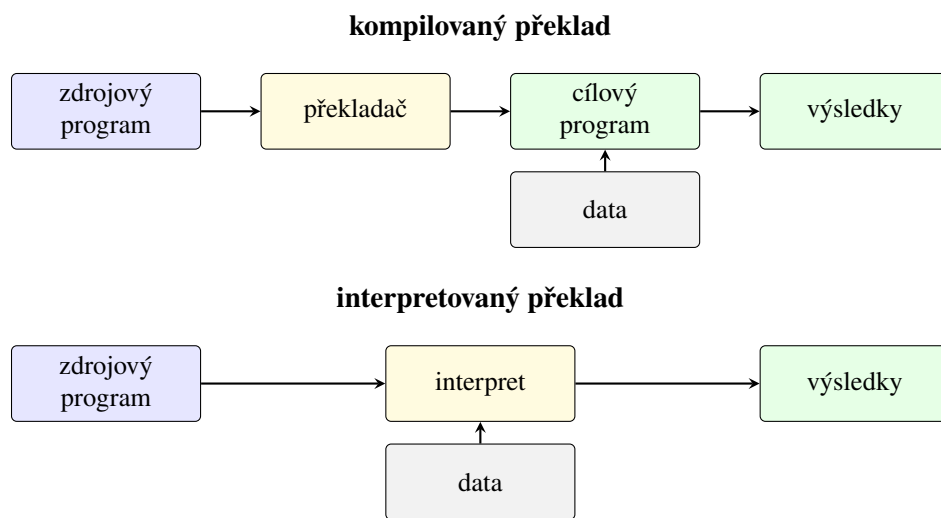
PUSH 14, STORE x.

Bez optimalizace by vznikly tři instrukce PUSH, násobení, sčítání a teprve potom uložení.

4.6 Interpretované jazyky

Dosud jsme předpokládali, že překladač vytvoří cílový program a ten se spustí až potom. Druhou základní možností je vykonávat program prostřednictvím jiného programu — interpretu.

Interpret načítá reprezentaci programu a vykonává její příkazy nebo instrukce. Nemusí předem vytvořit samostatný spustitelný soubor se strojovým kódem.



Obrázek 4.11: Zjednodušené porovnání kompilovaného a interpretovaného překladu.



Přesnější je mluvit o kompilované nebo interpretované *implementaci*, nikoliv o neměnné vlastnosti jazyka. Tentýž jazyk může mít klasický překladač, interpret i běhové prostředí kombinující více přístupů.

Výhody a nevýhody interpretace. Interpretované vykonávání usnadňuje přenos programu mezi platformami: stačí, aby na nich existoval odpovídající interpret. Často také zjednodušuje interaktivní práci, ladění a rychlé úpravy programu.

Za běhu se však opakuje práce, kterou by klasický překladač mohl provést předem. Interpret musí načítat instrukce své vnitřní reprezentace, rozhodovat o jejich významu a spravovat zásobník či prostředí proměnných. Čistá interpretace proto bývá pomalejší než přímé vykonávání dobře přeloženého strojového kódu. Navíc musí být při spuštění přítomno běhové prostředí.

4.6.1 Virtuální stroj

Náš překladač vytváří bajtkód. Program 4.6.1 jej vykonává pomocí zásobníku a slovníku proměnných. Jde tedy o malý *virtuální stroj*: instrukce nejsou instrukcemi skutečného procesoru, ale mají přesně definovaný účinek

v našem programu.

Program 4.6.1: Interpret zásobníkového bajtkódu.

```

1 from collections.abc import Callable
2
3 from .codegen import Instruction
4
5
6 class VirtualMachine:
7     def __init__(
8         self, output: Callable[[float], None] = print
9     ) -> None:
10         self.output = output
11         self.stack: list[float] = []
12         self.variables: dict[str, float] = {}
13
14     def run(self, code: list[Instruction]) -> None:
15         instruction_pointer = 0
16         while True:
17             instruction = code[instruction_pointer]
18             instruction_pointer += 1
19             opcode = instruction.opcode
20
21             if opcode == "HALT":
22                 return
23             if opcode == "PUSH":
24                 assert isinstance(instruction.argument, float)
25                 self.stack.append(instruction.argument)
26             elif opcode == "LOAD":
27                 assert isinstance(instruction.argument, str)
28                 self.stack.append(self.variables[instruction.argument])
29             elif opcode == "STORE":
30                 assert isinstance(instruction.argument, str)
31                 self.variables[instruction.argument] = self.stack.pop()
32             elif opcode == "PRINT":
33                 self.output(self.stack.pop())
34             else:
35                 self._binary_operation(opcode)
36
37     def _binary_operation(self, opcode: str) -> None:
38         right = self.stack.pop()
39         left = self.stack.pop()
40         operations = {
41             "ADD": lambda: left + right,
42             "SUB": lambda: left - right,
43             "MUL": lambda: left * right,
44             "DIV": lambda: left / right,
45         }
46         try:
47             result = operations[opcode]()
48         except KeyError as error:
49             raise ValueError(f"Unknown opcode {opcode!r}.") from error
50         self.stack.append(result)

```

U binární operace leží na vrcholu zásobníku pravý operand a pod ním levý. Virtuální stroj je odebere, spočítá výsledek a ten opět vloží na zásobník. Instrukce HALT ukončí interpretační smyčku.

4.6.2 JIT kompilace

JIT kompilace (z anglického *just-in-time compilation*) překládá mezikód do strojového kódu až za běhu programu. Běhové prostředí může nejprve bajtkód interpretovat, sledovat často používané části a jen ty později přeložit a optimalizovat.



Obrázek 4.12: Překlad přes mezikód s možnou JIT kompilací.

Tento přístup kombinuje přenositelnost mezikódu s rychlejším vykonáváním často používaných částí. JIT navíc zná skutečné chování programu za běhu, a může tedy využít informace, které statický překladač neměl. Platí za to časem a pamětí spotřebovanými na překlad během spuštění. Není proto automaticky nejlepší pro každý program. Známymi příklady běhových prostředí využívajících JIT jsou virtuální stroj Javy, prostředí .NET a moderní implementace JavaScriptu.

Celý překlad

Program 4.6.2 spojuje všechny vytvořené části. Zdrojový text projde lexerem, parserem, sémantickou kontrolou, výpočtem konstant a generátorem bajtkódu. Nakonec jej vykoná virtuální stroj.

Program 4.6.2: Sestavení a spuštění celého malého překladače.

```

1 from .codegen import CodeGenerator
2 from .lexer import Lexer
3 from .optimizer import ConstantFolder
4 from .parser import Parser
5 from .semantic import SemanticAnalyzer
6 from .virtual_machine import VirtualMachine
7
8
9 SOURCE = """
10 let x = 2 + 3 * 4;
11 print x;
12 let y = (x - 2) / 3;
13 print y;
14 """
15
16
17 def main() -> None:
18     tokens = Lexer(SOURCE).scan_tokens()
19     syntax_tree = Parser(tokens).parse()
20     SemanticAnalyzer().analyze(syntax_tree)
21     optimized_tree = ConstantFolder().optimize(syntax_tree)
22     bytecode = CodeGenerator().generate(optimized_tree)
23
24     for instruction in bytecode:
25         print(instruction)
26
27     print("Program output:")
28     VirtualMachine().run(bytecode)
  
```

```
29  
30  
31 if __name__ == "__main__":  
32     main()
```

Pro přiložený vstup optimalizátor nahradí výraz $2 + 3 \cdot 4$ číslem 14. Vygenerovaný bajtkód nejprve uloží hodnotu do proměnné `x`, vypíše ji, spočítá proměnnou `y` a vypíše také její hodnotu. Výstupem je

```
14.0  
4.0
```



Ukázkový překladač je malý, ale hranice mezi částmi odpovídají skutečným systémům: lexer neřeší gramatiku, parser neřeší deklarace, sémantická analýza nemění význam programu a generátor kódu už dostává ověřenou reprezentaci. Právě toto rozdělení umožňuje každou část samostatně testovat a později nahradit propracovanější variantou.

Hardware

V předchozí kapitole 4 jsme sledovali, jak se zdrojový program postupně převádí do podoby, kterou lze vykonat. Tím však příběh programu nekončí. Výsledné strojové instrukce musí načíst procesor, určit jejich význam, získat potřebná data a provést požadované operace. K tomu procesor potřebuje registry, různé druhy paměti a cesty, po nichž se mezi jednotlivými částmi počítače přenášejí data, adresy a řídicí signály.

V této kapitole se proto podíváme na základní části počítače z pohledu vykonávání programu. Nejde nám o podrobný elektrotechnický popis jednotlivých součástek. Chceme především pochopit, proč nestačí říci, že program „běží v procesoru“, proč počítač používá několik různých druhů paměti a jak konstrukce paměťového subsystému ovlivňuje rychlost programu. Začneme obecným schématem počítače, poté rozebereme procesor a nakonec se budeme věnovat pamětem od registrů až po dlouhodobá úložiště.

5.1 Základní části počítače

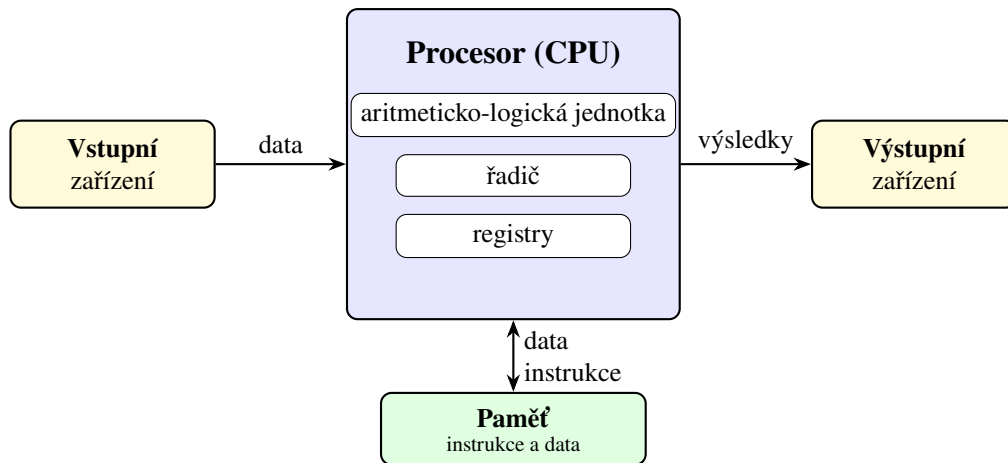
Slovem *hardware* označujeme fyzické součásti počítače: procesor, paměťové čipy, základní desku, úložiště, klávesnici, displej a další zařízení. Samotný hardware však neurčuje, jakou konkrétní úlohu má počítač řešit. Potřebný postup popisuje *software*, tedy programy a data, s nimiž tyto programy pracují. Stejný počítač proto může chvíli zobrazovat webovou stránku, poté překládat program a nakonec přehrávat video. Hardware zůstává stejný, mění se pouze instrukce, které právě vykonává.

Při velmi hrubém pohledu můžeme činnost počítače rozdělit mezi následující části:

- **datová část** provádí výpočty a zpracovává data;
- **řídicí část** určuje, co a v jakém pořadí se má stát;
- **paměť** uchovává programy, jejich instrukce, vstupní data a mezivýsledky;
- **vstupní zařízení** předávají počítači data z okolí;
- **výstupní zařízení** předávají výsledky uživateli nebo jinému zařízení.

Datová a řídicí část jsou v běžném počítači součástí *procesoru*, často označovaného zkratkou CPU z anglického *Central Processing Unit*. Procesor je sice místem, kde se instrukce skutečně provádějí, bez paměti a vstupně/výstupních zařízení by však neměl odkud instrukce ani data získat a kam uložit výsledky. Jednotlivé části proto nelze chápat jako izolované krabičky; při vykonávání programu spolu neustále komunikují.

Na obrázku 5.1 je zachycen typický tok informací. Vstupní zařízení dodá data, procesor je podle instrukcí programu zpracuje a výsledek může předat výstupnímu zařízení. Během tohoto procesu se instrukce i data přesouvají mezi procesorem a pamětí. Rozdělení na vstup a výstup přitom není vždy absolutní. Například síťová karta data přijímá



Obrázek 5.1: Zjednodušené schéma základních částí počítače.

i odesílá a dotyková obrazovka současně zobrazuje výstup a zaznamenává vstup uživatele.



V této kapitole se zaměříme hlavně na procesor a paměťový subsystém, tedy na části, které přímo souvisejí s vykonáváním programu. Vstupně/výstupní zařízení budeme uvažovat pouze na úrovni potřebné k pochopení celkového schématu.

Data, instrukce a jejich význam. Uvnitř počítače jsou informace reprezentovány pomocí bitů. Samotná posloupnost bitů však ještě neříká, co představuje. Stejný vzor může být podle okolností interpretován jako celé číslo, část obrázku, znak textu, adresa v paměti nebo strojová instrukce. Význam tedy nevychází pouze z uložených bitů, ale také z toho, jak s nimi právě pracuje procesor a jaký formát programu očekává.

Tento princip je důležitý i pro programování. Proměnná určitého datového typu určuje, jak má program uloženou hodnotu chápat, zatímco instrukce určuje, jaká operace se má s hodnotou provést. Na úrovni hardware jsou však obě informace stále reprezentovány bity a pro jejich přenos se používají stejné základní fyzikální prostředky.



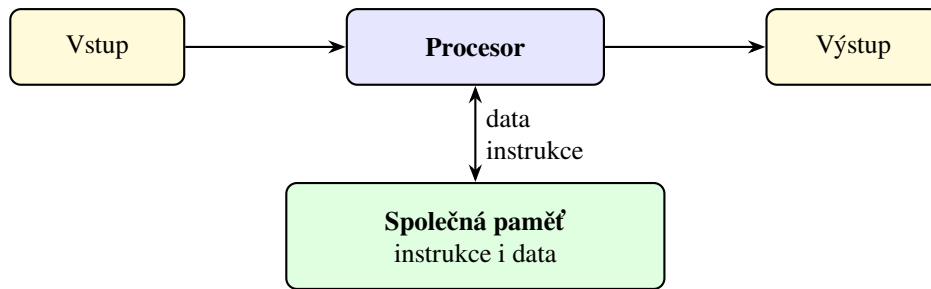
Při hledání chyby v programu je užitečné rozlišovat mezi uloženou hodnotou a její interpretací. Bity mohou být přeneseny nebo uloženy správně, ale program je může vyložit v nesprávném formátu.

5.2 Uložení programu a dat

5.2.1 Von Neumannova architektura

Základní představu o uspořádání počítače poskytuje *von Neumannova architektura*. Jejím podstatným rysem je, že program i data jsou uloženy ve společné paměti. Procesor z ní načítá strojové instrukce a stejným způsobem přistupuje také k hodnotám, které mají být zpracovány. Instrukce určují, jaká operace se má provést, odkud se mají získat operandy a kam se má uložit výsledek.

Tato myšlenka se někdy označuje jako *princip uloženého programu*. Program není pevně zabudován do zapojení počítače, ale je uložen v paměti podobně jako ostatní data. Změnou obsahu paměti tak můžeme změnit činnost počítače, aniž bychom museli přestavět jeho hardware. Právě tato univerzálnost je jedním z důvodů, proč se daný princip stal základem obecných počítačů.



Obrázek 5.2: Základní uspořádání von Neumannovy architektury.

Paměť v tomto modelu sama nerozlišuje, zda uložené bity představují data, nebo instrukce. Rozhoduje až způsob, jakým je procesor použije. Procesor opakovaně načte instrukci z paměti, zjistí její význam a provede ji. Pokud instrukce pracuje s daty uloženými v paměti, musí procesor obvykle provést další paměťové přenosy. Zjednodušeně tak dostáváme stále se opakující cyklus

načtení instrukce → dekódování → provedení.

Jeho podrobnější podobu rozebereme v podsekcí 5.3.2.

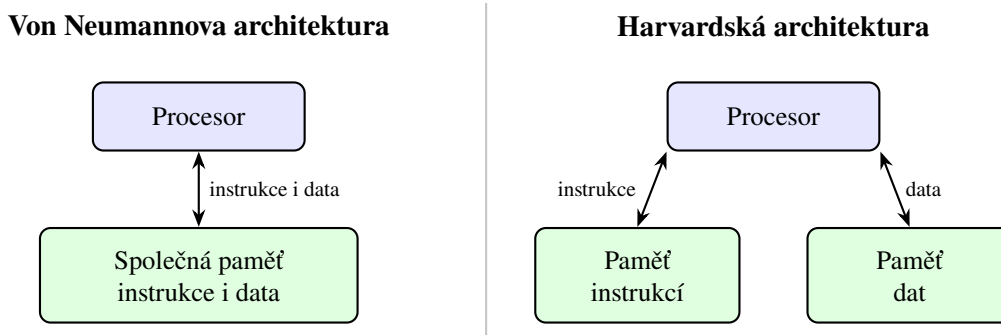
Společná cesta pro instrukce i data přináší jednoduché a univerzální uspořádání, ale současně může omezovat rychlost. Procesor musí s pamětí komunikovat při načítání instrukcí i při čtení a zápisu dat. Pokud je procesor schopen pracovat rychleji, než mu paměť dodává potřebné informace, část času pouze čeká. Toto omezení se označuje jako *von Neumannovo úzké hrdlo*.



Rychlost počítače neurčuje pouze počet operací, které dokáže procesor provést za sekundu. Stejně důležité je, zda procesor dostává instrukce a data dostatečně rychle. Právě proto bude později tak významná cache a celá paměťová hierarchie.

5.2.2 Harvardská architektura

Harvardská architektura používá oddělenou paměť pro instrukce a oddělenou paměť pro data. Obě paměti mohou mít vlastní adresový prostor i vlastní přenosové cesty. Procesor pak může současně načítat další instrukci a přenášet data potřebná pro právě prováděný výpočet. Paměť instrukcí navíc může mít jiné vlastnosti než paměť dat, protože se od ní může očekávat jiný způsob používání.



Obrázek 5.3: Porovnání von Neumannovy a harvardské architektury.

Rozdíl obou přístupů zachycuje obrázek 5.3. U von Neumannovy architektury vede mezi procesorem a společnou pamětí jedna logická cesta pro instrukce i data. V harvardském uspořádání má procesor k dispozici dvě oddělené cesty. To však neznamená, že každý reálný počítač musí přesně odpovídat jednomu z těchto dvou schémat.

V praxi se často používá *modifikovaná harvardská architektura*. Z pohledu programu může existovat jeden společný adresový prostor, zatímco uvnitř procesoru jsou oddělené například instrukční a datová cache. Počítač se tedy navenek chová podobně jako von Neumannův, ale v částech, kde je výhodné souběžné načítání, využívá prvky harvardského uspořádání.



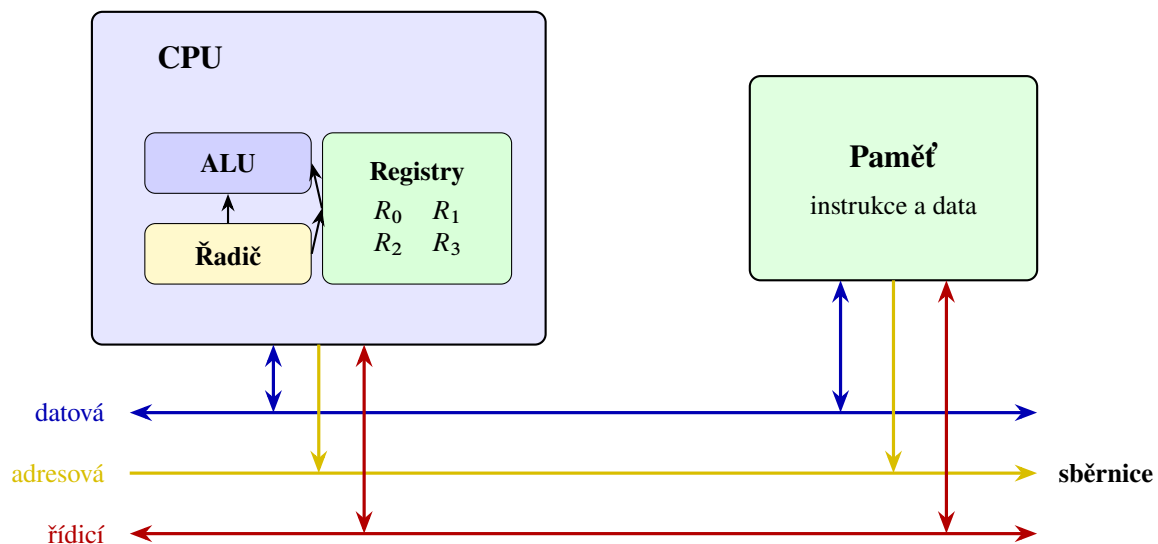
Von Neumannova a harvardská architektura jsou především užitečné modely. Skutečné procesory obsahují mnoho dalších mechanismů a jejich vnitřní uspořádání bývá kombinací více myšlenek.

5.3 Procesor

Procesor vykonává strojové instrukce programu. Nejde však o jedinou součástku provádějící všechny činnosti stejným způsobem. Uvnitř procesoru spolupracuje několik částí, z nichž každá má jinou úlohu. Pro náš zjednodušený model budou nejdůležitější aritmeticko-logická jednotka, řadič a registry.

- **Aritmeticko-logická jednotka (ALU)** provádí aritmetické a logické operace. Může například sčítat čísla, porovnávat hodnoty nebo provádět bitové operace.
- **Řadič** organizuje činnost procesoru. Na základě právě načtené instrukce vytváří řídicí signály, kterými určuje, odkud se mají získat operandy, jakou operaci má provést ALU a kam se má uložit výsledek.
- **Registry** uchovávají malé množství právě používaných dat přímo uvnitř procesoru. Přístup k nim je výrazně rychlejší než přístup do hlavní paměti.

Skutečný procesor obsahuje ještě řadu dalších částí, například jednotky pro práci s desetinnými čísly, mechanismy pro předvídání větvení, několik úrovní cache nebo jednotky schopné provádět jednu operaci nad více hodnotami současně. Základní vztah mezi řadičem, výpočetními jednotkami, registry a pamětí však zůstává užitečný i při popisu podstatně složitějších procesorů.



Obrázek 5.4: Zjednodušená vnitřní struktura procesoru a jeho spojení s pamětí.

5.3.1 Sběrnice

Procesor, paměť a další zařízení potřebují přenášet různé druhy informací. Soubor vodičů a pravidel, podle nichž se po nich informace přenášejí, nazýváme *sběrnicí*. Na obrázku 5.4 rozlišujeme tři logické skupiny signálů:

- **datová sběrnice** přenáší samotná data a instrukce;
- **adresová sběrnice** určuje paměťové místo nebo zařízení, s nímž chce procesor komunikovat;
- **řídící sběrnice** přenáší signály určující například směr přenosu, požadavek na čtení či zápis nebo informaci o dokončení operace.

Uvažujme například čtení hodnoty z paměti. Procesor nejprve předá na adresovou sběrnici adresu požadovaného místa a řídícím signálem oznámí, že chce číst. Paměť vyhledá odpovídající obsah a předá jej po datové sběrnici zpět procesoru. Při zápisu je postup podobný, pouze procesor zároveň odešle hodnotu a řídícím signálem určí, že se má obsah paměti změnit.

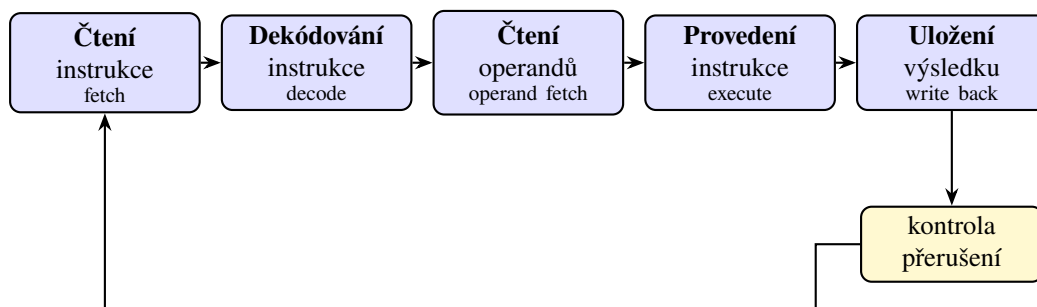


Rozdělení na datovou, adresovou a řídící sběrnici popisuje význam přenášených signálů. Ve skutečném zařízení nemusí každé skupině odpovídat samostatná sada vodičů; některé fyzické spoje mohou v různých okamžicích přenášet různé druhy informací.

5.3.2 Instrukční cyklus

Program je pro procesor posloupností strojových instrukcí uložených v paměti. Každá instrukce musí být načtena, rozpoznána a provedena. Jednotlivé procesory se v podrobnostech liší, ale základní průběh lze popsat následujícími kroky:

1. **Čtení instrukce (*fetch*)**. Procesor získá z paměti instrukci, na kterou ukazuje čítač instrukcí.
2. **Dekódování instrukce (*decode*)**. Řadič určí, jakou operaci instrukce požaduje a kde se nacházejí její operandy.
3. **Čtení operandů (*operand fetch*)**. Potřebné hodnoty se získají z registrů, případně z paměti.
4. **Provedení instrukce (*execute*)**. Příslušná výpočetní jednotka provede požadovanou operaci.
5. **Uložení výsledku (*write back*)**. Výsledek se zapíše do registru nebo do paměti.



Obrázek 5.5: Zjednodušený instrukční cyklus procesoru.

Představme si například instrukci, která má sečíst hodnoty registrů R_0 a R_1 a výsledek uložit do registru R_2 . Procesor instrukci nejprve načte a dekoduje. Poté přečte hodnoty obou zdrojových registrů, předá je ALU, nechá provést součet a výsledek zapíše do cílového registru. Následně může pokračovat další instrukcí.

Obrázek 5.5 vytváří dojem, že procesor dokončí všechny kroky jedné instrukce a teprve potom začne načítat další. Takový postup je možný, moderní procesory však jednotlivé fáze často překrývají. Zatímco jedna instrukce

se provádí, další se může dekodovat a ještě další načítat. Tento způsob zpracování se nazývá *zřetězení* neboli *pipeline*. Překrývání fází zvyšuje počet instrukcí, které lze zpracovat za určitou dobu, přestože doba průchodu jedné instrukce všemi fázemi se nemusí zkrátit.

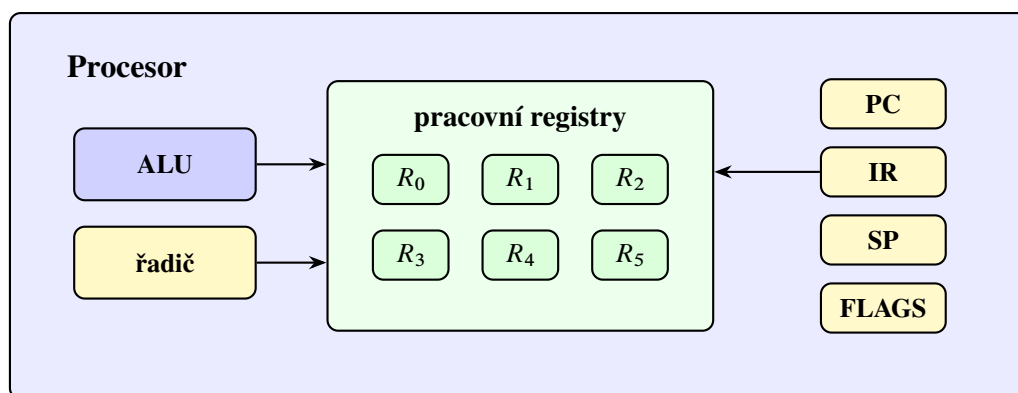
Činnost procesoru je řízena hodinovým signálem. Jeho frekvence udává počet taktů za sekundu, nikoli přímo počet dokončených instrukcí. Některá instrukce může vyžadovat více taktů, více instrukcí se může zpracovávat souběžně a procesor může čekat na data z paměti. Samotná hodnota frekvence proto nestačí k porovnání výkonu různých procesorů.

Přerušení. Běžící program není jediným zdrojem událostí, na které musí procesor reagovat. Klávesnice může oznámit stisk klávesy, úložiště dokončení přenosu nebo časovač uplynutí stanoveného intervalu. K takovým situacím slouží *přerušení*. Procesor po dokončení vhodného kroku uloží informace potřebné pro pozdější pokračování programu a přejde k obsluze vzniklé události. Po jejím vyřízení se může k původnímu programu vrátit.

Přerušení umožňuje, aby procesor nemusel každé zařízení neustále kontrolovat. Zařízení jej samo upozorní ve chvíli, kdy potřebuje obsluhu. Operační systém pak rozhodne, jaká část programu má událost zpracovat a kdy může pokračovat přerušovaný výpočet.

5.3.3 Registry

Registry jsou velmi malé a rychlé paměti přímo uvnitř procesoru. Uchovávají operandy právě prováděných instrukcí, adresy, mezivýsledky i informace o stavu výpočtu. Instrukce obvykle pracují s registry přímo, zatímco hodnotu z hlavní paměti je nejprve potřeba do registru načíst a výsledek případně později zapsat zpět.



Obrázek 5.6: Pracovní a vybrané speciální registry v procesoru.

Vedle pracovních registrů, jejichž význam určuje právě prováděný program, se setkáme také s registry se zvláštní úlohou:

- **PC (*program counter*)** obsahuje adresu následující instrukce, která se má načíst. Po běžné instrukci se posune vpřed, zatímco instrukce skoku do něj může uložit jinou adresu.
- **IR (*instruction register*)** uchovává právě načtenou nebo prováděnou instrukci.
- **SP (*stack pointer*)** ukazuje na vrchol zásobníku v paměti. Zásobník se využívá například při volání funkcí.
- **FLAGS** uchovává příznaky výsledku poslední operace, například zda byl výsledek nulový, záporný nebo zda došlo k přetečení.

Konkrétní názvy, počty a přesný význam registrů závisí na instrukční sadě procesoru. Obrázek 5.6 proto nepředstavuje schéma jednoho konkrétního modelu, ale obecnou ilustraci rolí, které registry mohou zastávat.



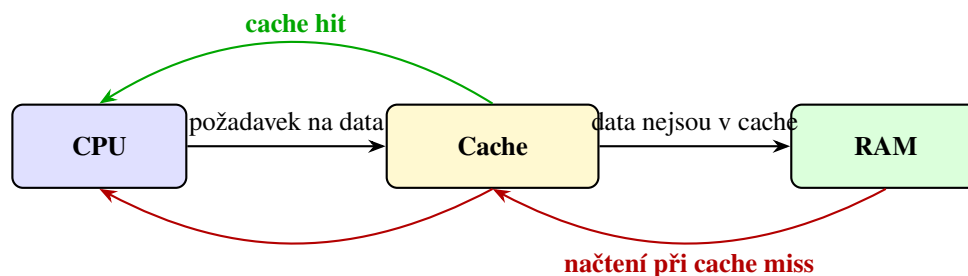
Registry nejsou určeny k dlouhodobému uložení velkého množství dat. Jejich hlavní výhodou je rychlost a přímá dostupnost pro instrukce procesoru; za tuto výhodu platíme velmi malou kapacitou.

5.4 Vnitřní paměti

Procesor dokáže provádět operace velmi rychle, ale pro většinu instrukcí potřebuje data. Kdyby musel při každé operaci čekat na hlavní paměť, zůstala by velká část jeho výkonu nevyužita. Počítač proto nepoužívá jediný druh paměti. Mezi procesorem a hlavní pamětí se nachází cache a přímo uvnitř procesoru registry, které jsme již popsali v podsekcí 5.3.3.

5.4.1 Cache

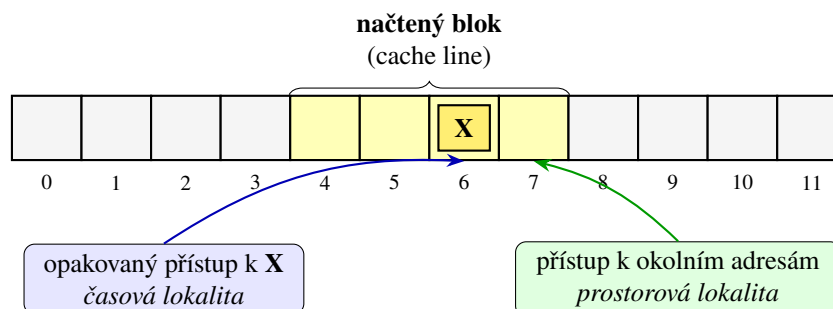
Cache je malá a velmi rychlá paměť, která uchovává kopie vybraných dat a instrukcí z pomalejší paměti. Když procesor požaduje určitou hodnotu, nejprve se ověří, zda její kopie není v cache. Pokud se hledaná informace v cache nachází, nastane *cache hit* a procesor ji získá rychle. Pokud se v cache nenachází, nastane *cache miss*; data je potřeba načíst z nižší, pomalejší úrovně paměti a současně se obvykle uloží do cache pro případ dalšího použití.



Obrázek 5.7: Obsluha požadavku procesoru pomocí cache.

Cache není pouhým seznamem jednotlivých naposledy použitých hodnot. Data se mezi paměťovými úrovněmi přenášejí po blocích, kterým se říká *cache line*. Jestliže procesor požádá o jeden bajt nebo jedno číslo, do cache se spolu s ním načte také okolní část paměti. Tento postup je výhodný díky dvěma často pozorovaným vlastnostem programů:

- **časová lokalita** znamená, že nedávno použitá data nebo instrukce budou pravděpodobně brzy použity znovu;
- **prostorová lokalita** znamená, že po přístupu k určitému místu budou pravděpodobně brzy potřeba také okolní místa.



Obrázek 5.8: Příklad prostorové a časové lokality při práci s pamětí.

Sekvenční průchod polem obvykle vykazuje dobrou prostorovou lokalitu, protože program postupně čte sousední

paměťová místa. Opakované používání stejné proměnné uvnitř cyklu zase vykazuje časovou lokalitu. Naopak přístupy k velkému množství vzdálených adres v nepravidelném pořadí mohou vést k častým výpadkům cache a výpočet se zpomalí, i když počet provedených aritmetických operací zůstane stejný.

Procesory obvykle používají několik úrovní cache. Cache L1 je nejmenší a nejbližší výpočetním jednotkám, další úrovně L2 a L3 bývají postupně větší, ale také pomalejší. Některé úrovně mohou být rozděleny na instrukční a datovou cache, jiné mohou být sdíleny několika jádry procesoru. Smyslem všech úrovní je zachytit co největší část požadavků dříve, než bude nutné přistoupit do výrazně pomalejší hlavní paměti.

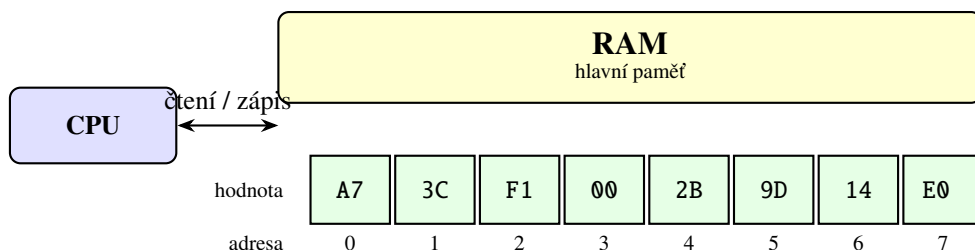


Dva programy se stejnou časovou složitostí mohou na skutečném počítači běžet různě rychle. Jedním z důvodů je odlišný způsob přístupu k paměti: algoritmus s lepší lokalitou dokáže využít cache účinněji.

5.4.2 Hlavní paměť RAM

RAM z anglického *Random Access Memory* je hlavní pracovní paměť počítače. Ukládají se do ní právě běžící programy, jejich data a mezivýsledky výpočtů. Je rychlejší než dlouhodobá úložiště, ale pomalejší než cache a registry. RAM je současně *volatilní*: bez napájení se její obsah ztratí.

Označení *random access* neznamena, že by paměť vracela náhodná data. Vyjadřuje, že k paměťovým místům lze přistupovat přímo podle jejich adresy a není potřeba postupně procházet všechna předchozí místa. Z pohledu programu se hlavní paměť chová jako dlouhá lineární posloupnost adresovatelných míst. V běžném počítači adresa obvykle označuje jeden bajt.

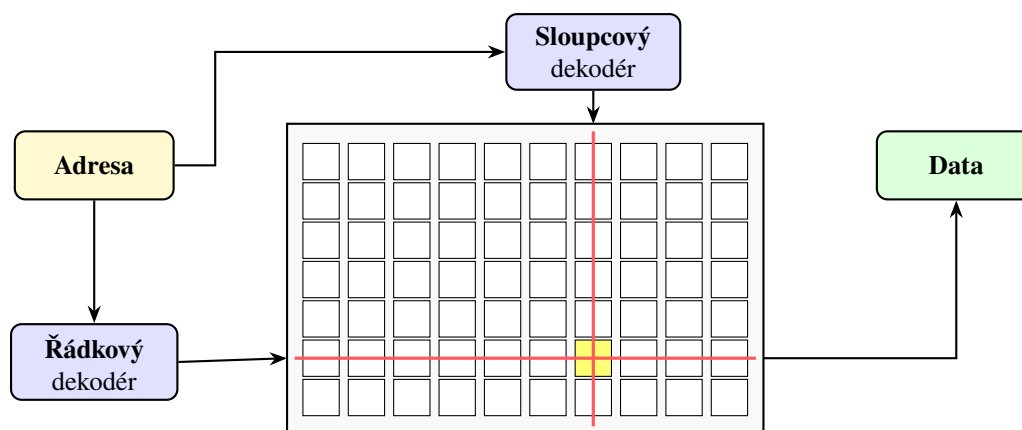


Obrázek 5.9: Lineární pohled programu na adresovanou paměť.

Na obrázku 5.9 jsou na adresách 0 až 7 uloženy osmibitové hodnoty zapsané v šestnáctkové soustavě. Chce-li procesor přečíst hodnotu z adresy 3, předá tuto adresu paměti a obdrží uloženou hodnotu 00. Větší datový objekt, například 32bitové celé číslo, obvykle zabírá několik sousedních bajtů. Instrukční sada a program pak určují, jak se mají tyto bajty společně interpretovat.

Fyzické uspořádání RAM. Lineární posloupnost adres je rozhraním určeným pro procesor a program. Uvnitř paměťového čipu nejsou buňky nutně uspořádány v jediné řadě, ale tvoří matice řádků a sloupců. Část adresy se použije k výběru řádku a další část k výběru sloupce. Řádkový a sloupcový dekodér tak určí buňku nebo skupinu buněk, s nimiž se má pracovat.

Hlavní paměť počítače je obvykle tvořena pamětí DRAM. Jedna její buňka uchovává bit pomocí elektrického náboje, který postupně slábne, a proto se musí pravidelně obnovovat. Cache se naproti tomu obvykle vytváří z paměti SRAM. Ta nepotřebuje pravidelné obnovování a je rychlejší, ale jedna buňka zabírá více místa a její výroba je dražší. Proto se SRAM hodí pro malé rychlé cache, zatímco DRAM umožňuje vyrobit hlavní paměť s mnohem větší kapacitou.



Obrázek 5.10: Zjednodušené fyzické uspořádání paměťových buněk do matice.

Vlastnost	SRAM	DRAM
Typické použití	cache	hlavní paměť
Rychlost	vyšší	nižší
Hustota buněk	nižší	vyšší
Potřeba obnovování	ne	ano
Uchování bez napájení	ne	ne

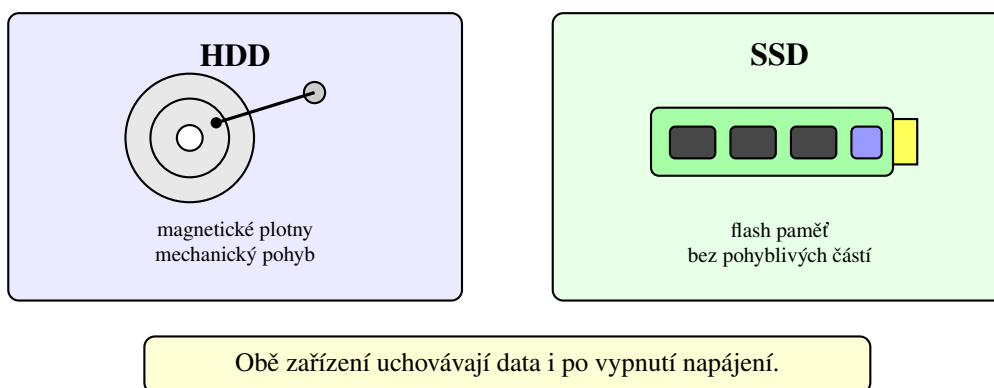
Tabulka 5.1: Základní porovnání pamětí SRAM a DRAM.



SRAM i DRAM jsou volatilní. Rozdíl v potřebě obnovování neznamena, že by SRAM udržela data po vypnutí počítače; pouze za běžného napájení nepotřebuje periodicky obnovovat náboj paměťových buněk.

5.5 Externí paměti a úložiště

Hlavní paměť uchovává data pouze po dobu, kdy je počítač napájen. Programy, dokumenty a další soubory proto potřebujeme ukládat do nevolatilní paměti, jejíž obsah zůstane zachován i po vypnutí. Mezi nejběžnější dlouhodobá úložiště patří pevné disky HDD a disky SSD. Obě zařízení poskytují operačnímu systému adresovatelný prostor rozdělený do bloků, ale fyzikální princip jejich fungování je odlišný.



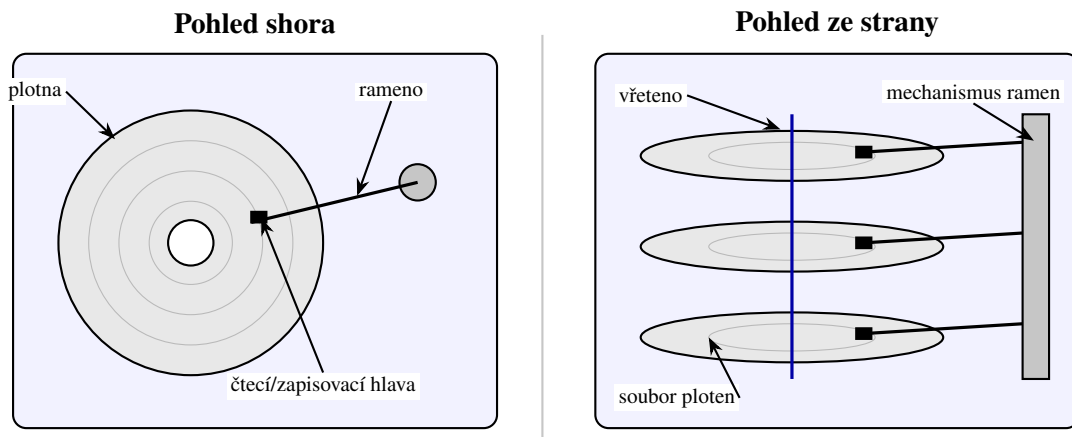
Obrázek 5.11: Základní konstrukční rozdíl mezi HDD a SSD.

HDD ukládá data magneticky na rotující plotny a používá pohyblivé čtecí a zapisovací hlavy. SSD ukládá data elektronicky do buněk flash paměti a neobsahuje části, které by se při běžném čtení mechanicky pohybovaly. Díky

tomu má SSD zpravidla podstatně rychlejší náhodný přístup, je odolnější vůči otřesům a pracuje tiše. HDD naopak často nabízí velkou kapacitu za nižší cenu.

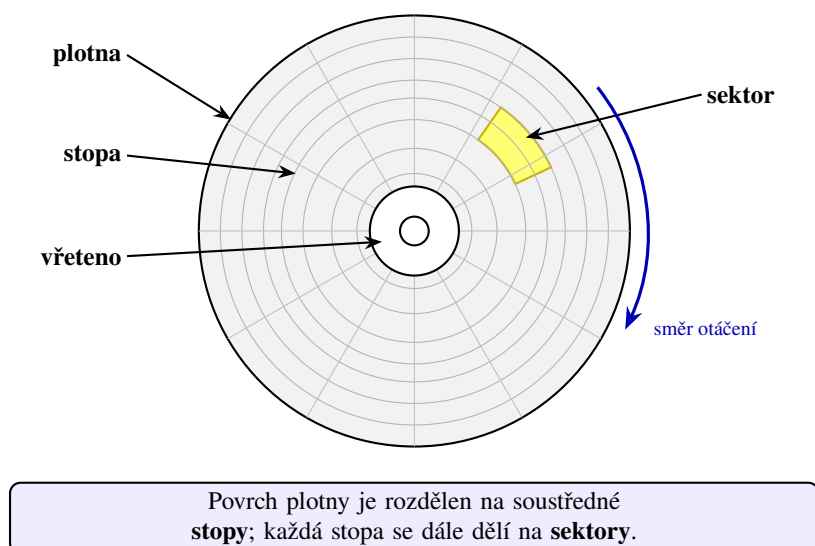
5.5.1 Pevný disk HDD

Uvnitř HDD se nachází jedna nebo více kruhových ploten upevněných na společném vřetenu. Povrch každé plotny je pokryt magnetickou vrstvou. Plotny se při provozu otáčejí a nad jejich povrchem se pohybují čtecí a zapisovací hlavy umístěné na společném rameni. Hlava se povrchu za běžného provozu nedotýká; změny magnetizace povrchu používá ke čtení a zápisu bitů.



Obrázek 5.12: Zjednodušený pohled na vnitřní strukturu HDD shora a ze strany.

Data jsou na povrchu plotny uspořádána do soustředných *stop*. Každá stopa se dále dělí na *sektory*. Stopy ve stejné vzdálenosti od středu na různých površích ploten tvoří *cylindr*. V moderním disku se operační systém nemusí zabývat přesnou fyzickou polohou dat; řadič disku převádí logická čísla bloků na konkrétní místa uvnitř zařízení. Geometrický model je však stále užitečný k pochopení výkonu HDD.



Obrázek 5.13: Stopy a sektory na jedné plotně HDD.

Při čtení bloku musí disk nejprve přesunout rameno nad správnou stopu. Doba tohoto pohybu se označuje jako *doba vystavení* neboli *seek time*. Poté disk čeká, než se požadovaný sektor otočí pod hlavu; vzniká *rotační zpoždění*. Teprve potom lze data skutečně přenést. Přesun hlavy a čekání na natočení plotny jsou mechanické operace, a proto

jsou ve srovnání s elektronickými operacemi pomalé.

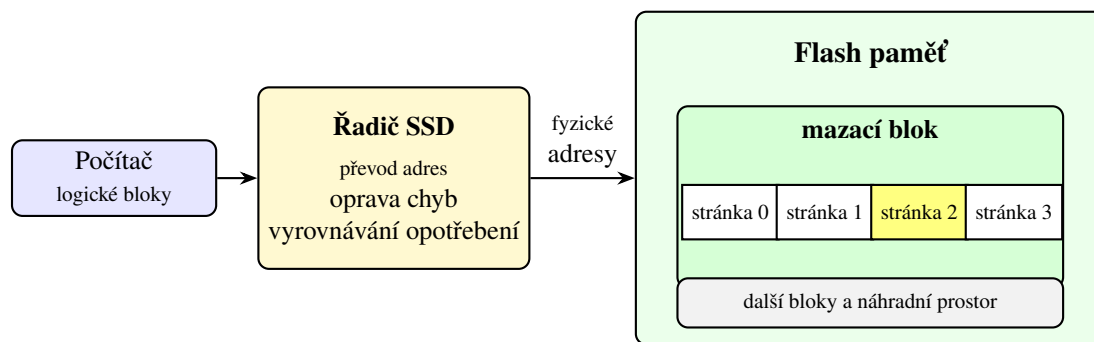
Sekvenční čtení velkého souboru je pro HDD výhodné, protože sousední sektory procházejí pod hlavou postupně a rameno se nemusí často přesouvat. Naopak mnoho malých požadavků na vzdálená místa disku vede k opakovanému pohybu hlav a výkon výrazně klesá. Rozdíl mezi sekvenčním a náhodným přístupem je proto u HDD zvlášť výrazný.



Při prudkém nárazu může dojít k poškození mechanických částí HDD nebo povrchu plotny. Běžná záloha proto nemá být uložena pouze na druhém oddílu téhož fyzického disku; porucha zařízení by mohla zasáhnout původní data i jejich „zálohu“.

5.5.2 Disk SSD

SSD neukládá data magneticky, ale do buněk nevolatilní flash paměti. Kromě paměťových čipů obsahuje řadič, který přijímá požadavky počítače, vyhledává fyzické umístění dat, opravuje některé chyby a rozděljuje zápisy mezi paměťové buňky. Operační systém přitom stejně jako u HDD pracuje s logickými bloky a nemusí znát konkrétní uspořádání buněk uvnitř zařízení.



Obrázek 5.14: Zjednodušené části SSD a organizace flash paměti.

Flash paměť čte a zapisuje data po *stránkách*, ale mazání probíhá po větších *blocích* obsahujících více stránek. Již zapsanou stránku proto nelze libovolně přepsat stejným způsobem jako bajt v RAM. Řadič obvykle zapíše novou verzi dat na jiné volné místo, změní svou převodní tabulku a později uvolní bloky obsahující již neplatná data.

Každá paměťová buňka snese pouze omezený počet mazacích cyklů. Řadič proto používá *vyrovnávání opotřebení*, kterým se snaží rozložit zápisy mezi různé fyzické bloky. Současně může mít SSD část kapacity vyhrazenou jako náhradní prostor. Tyto mechanismy prodlužují životnost zařízení a umožňují skrýt složitější vlastnosti flash paměti za jednoduché rozhraní logických bloků.



SSD nemá mechanické vyhledávání stopy ani rotační zpoždění, a proto je jeho náhodný přístup podstatně rychlejší než u HDD. Přesto ani SSD není stejně rychlé jako RAM a při zápisu musí jeho řadič řešit omezení flash paměti.

Porovnání HDD a SSD. Volba úložiště závisí na požadované kapacitě, rychlosti, ceně i způsobu použití. Pro systémový disk a často spouštěné aplikace je obvykle výhodné SSD, protože zkracuje čekání při mnoha menších náhodných přístupech. HDD se může hodit pro velké archivy nebo zálohy, u nichž je důležitá kapacita a převládá sekvenční přenos.

Tabulka 5.2 zachycuje obecné tendence, nikoliv záruku pro každý konkrétní model. Rychlé HDD může v některé

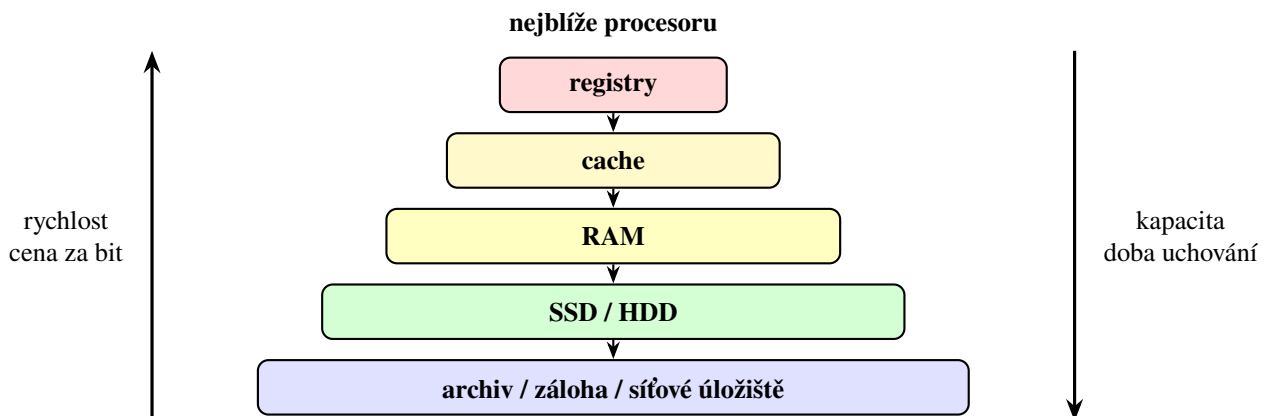
Vlastnost	HDD	SSD
Princip uložení	magnetický	elektronický
Pohyblivé části	ano	ne
Náhodný přístup	pomalejší	rychlejší
Hlučnost	slyšitelný pohyb	tiché
Typická přednost	kapacita a cena	rychlost a odolnost
Uchování bez napájení	ano	ano

Tabulka 5.2: Obecné porovnání vlastností HDD a SSD.

úloze překonat velmi pomalé SSD a výsledný výkon ovlivňuje také použité rozhraní, řadič, velikost požadavků nebo zaplnění zařízení. Při praktickém porovnávání je proto potřeba sledovat parametry a měření konkrétních výrobků.

5.6 Paměťová hierarchie

Žádný běžný druh paměti současně nenabízí nejvyšší rychlost, obrovskou kapacitu, nízkou cenu a trvalé uložení dat. Počítač proto kombinuje několik paměťových úrovní. Blízko procesoru se nacházejí malé a rychlé registry a cache, pod nimi větší hlavní paměť a ještě níže dlouhodobá úložiště. Pro zálohování a archivaci lze přidat síťová úložiště, magnetické pásky nebo jiné systémy s velkou kapacitou, u nichž nevádí delší doba přístupu.



Obrázek 5.15: Zjednodušená paměťová hierarchie počítače.

Směrem vzhůru v hierarchii na obrázku 5.15 roste rychlost a obvykle také cena za uložený bit, zatímco kapacita klesá. Směrem dolů získáváme větší a levnější úložný prostor, ale na data čekáme déle. Jednotlivé úrovně proto plní rozdílné role:

- **registry** uchovávají hodnoty, s nimiž právě pracují instrukce;
- **cache** uchovává kopie nedávno nebo blízce používaných částí paměti;
- **RAM** obsahuje běžící programy a jejich pracovní data;
- **SSD a HDD** dlouhodobě uchovávají programy a soubory;
- **archivní a záložní systémy** upřednostňují kapacitu, cenu a bezpečné uchování před okamžitým přístupem.

Paměťová hierarchie funguje dobře tehdy, když se na rychlejších úrovních nacházejí právě ta data, která budou brzy potřeba. Část přesunů zajišťuje hardware automaticky, jako v případě cache. Jiné přesuny řídí operační systém, například když načítá program z SSD do RAM. Další provádí přímo uživatel, když přesune starší data do archivu

Úroveň	Relativní rychlost	Relativní kapacita	Volatilní
Registry	nejvyšší	nejmenší	ano
Cache	velmi vysoká	malá	ano
RAM	střední	větší	ano
SSD/HDD	nižší	velká	ne
Archiv/záloha	nejnižší	velmi velká	ne

Tabulka 5.3: Kvalitativní porovnání úrovní paměťové hierarchie.

nebo vytvoří zálohu.

Z pohledu programu může být velká část těchto přesunů skrytá. Program pracuje s adresami v paměti a procesor se pokouší obsloužit požadavky co nejrychleji. Pokud jsou data v registru nebo cache, pokračuje výpočet rychle. Při výpadku cache je potřeba čekat na RAM; pokud se potřebná část programu nebo soubor teprve načítá z úložiště, čekání je ještě delší. Rozdíl mezi jednotlivými úrovněmi vysvětluje, proč má způsob uspořádání dat významný vliv na skutečný výkon.



Paměťová hierarchie vytváří dojem velké a rychlé paměti tím, že kombinuje malou rychlou paměť s většími pomalejšími úrovněmi. Tento dojem je nejpřesvědčivější tehdy, když program vykazuje dobrou časovou a prostorovou lokalitu.

Shrnutí kapitoly.

- Procesor vykonává instrukce, ale při své činnosti neustále komunikuje s pamětí, vstupními a výstupními zařízeními.
- Ve von Neumannově architektuře sdílejí instrukce a data společnou paměť; harvardská architektura jejich paměti a přenosové cesty odděluje.
- Základními částmi procesoru jsou výpočetní jednotky, řadič a registry. Instrukce procházejí fázemi načtení, dekódování, získání operandů, provedení a uložení výsledku.
- Cache zkracuje průměrnou dobu přístupu k datům tím, že využívá časovou a prostorovou lokalitu programů.
- Hlavní paměť poskytuje programu lineární adresový prostor, přestože jsou fyzické buňky v čípech uspořádány do matic.
- HDD ukládá data magneticky na rotující plotny, zatímco SSD používá flash paměť bez pohyblivých částí.
- Registry, cache, RAM, úložiště a archivní systémy společně vytvářejí paměťovou hierarchii, která vyvažuje rychlost, kapacitu a cenu.

Kryptografie

V kapitole 5 jsme sledovali, jak počítač ukládá a zpracovává bity. Samotná bitová reprezentace však neříká nic o tom, kdo smí uložená data přečíst, zda je někdo cestou změnil ani od koho skutečně pocházejí. Jakmile zprávu posíláme přes síť nebo ji ukládáme na zařízení, ke kterému mohou získat přístup další lidé, potřebujeme tyto otázky řešit výpočetními prostředky.

Právě zde nastupuje kryptografie. Nejprve si ujasníme její cíle a doplníme základy pravděpodobnosti, bez nichž nelze moderní pojem bezpečnosti přesně formulovat. Poté začneme historickými substitučními šiframi a přejdeme k symetrickému a asymetrickému šifrování. Na algoritmu RSA uvidíme, jak lze bezpečnost spojit s obtížností matematického problému. Závěr kapitoly bude patřit pseudonáhodným generátorům, jednosměrným a hešovacím funkcím, standardu SHA a elektronickému podpisu.

6.1 Co je to kryptografie?

Název *kryptologie* vznikl spojením řeckých slov označujících něco skrytého a nauku. Kryptologie je zastřešující obor zabývající se ochranou informací i překonáváním této ochrany. *Kryptografie* zkoumá návrh a vlastnosti metod, které mají informace chránit, zatímco *kryptanalýza* hledá způsoby, jak takové metody prolomit. Tyto dvě oblasti se navzájem doplňují: šifru nelze považovat za bezpečnou jen proto, že její autor zatím žádný útok nenašel.

Základní situaci budeme popisovat pomocí tradičních postav Alice a Boba. Alice chce Bobovi poslat zprávu

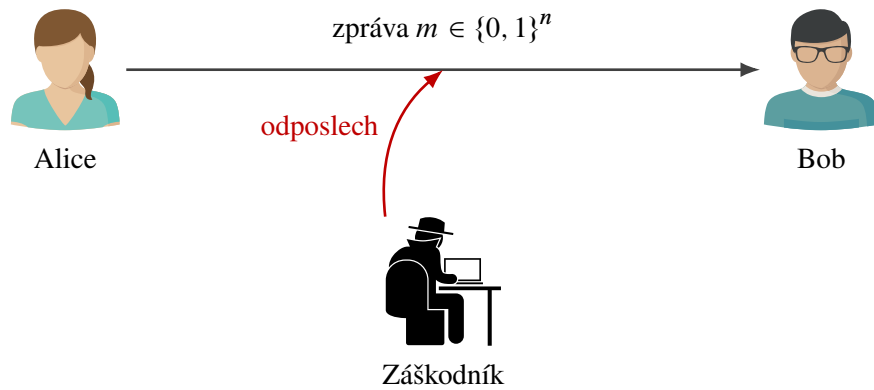
$$m \in \{0, 1\}^n.$$

Komunikační kanál však sleduje záškodník. Ten může přenášená data odposlechnout, pozměnit, zadržet nebo nahradit vlastní zprávou. Předpokládáme přitom, že záškodník zná používané algoritmy. Tajemstvím nemá být způsob šifrování, ale pouze vhodně zvolený klíč.

Kryptografie neřeší pouze utajení obsahu. V praxi se obvykle snažíme dosáhnout několika odlišných cílů:

- **důvěrnost** – obsah zprávy má být čitelný pouze oprávněným příjemcům;
- **integrita** – příjemce má poznat, zda byla zpráva změněna;
- **autenticita** – příjemce má umět ověřit, od koho zpráva pochází;
- **doložitelnost původu** – odesílatel nemá moci snadno popřít, že zprávu vytvořil; přesný právní význam však závisí na použitém typu podpisu a okolnostech.

Šifrování se zaměřuje především na důvěrnost, samo o sobě však automaticky nezaručuje integritu ani autenticitu. Hešovací funkce pomáhají odhalit změnu dat, ale neříkají, kdo otisk vytvořil. Elektronický podpis propojí obsah



Obrázek 6.1: Alice posílá Bobovi zprávu přes kanál sledovaný záškodníkem.

zprávy se soukromým klíčem podepisující osoby. Jednotlivé nástroje proto budeme posuzovat podle toho, jaký bezpečnostní problém skutečně řeší.



Bezpečnost moderního kryptografického systému nemá stát na utajení algoritmu. Algoritmus může být veřejný a podrobně zkoumaný; tajný zůstává klíč. Díky tomu lze systém analyzovat a případné slabiny odhalit dříve, než budou zneužity.

6.1.1 Šifrování a dešifrování

Zprávu před zašifrováním nazýváme *otevřený text*. Šifrovací algoritmus **E** ji spolu s klíčem převede na *šifrový text* c . Dešifrovací algoritmus **D** má oprávněnému příjemci umožnit získat původní zprávu:

$$c = \mathbf{E}(m, k), \quad m = \mathbf{D}(c, k).$$

Podle toho, zda se pro obě operace používá tentýž tajný klíč, nebo dvojice veřejného a soukromého klíče, budeme později rozlišovat symetrickou a asymetrickou kryptografii.

Požadavek na správné dešifrování nazýváme *korektnost*. V nejjednodušším symetrickém případě má pro každou dovolenou zprávu m a klíč k platit

$$\mathbf{D}(\mathbf{E}(m, k), k) = m.$$

Korektnost ale ještě neznamena bezpečnost. I velmi snadno prolomitelná historická šifra může být korektní, pokud Bob se správným klíčem vždy získá původní zprávu.



V této kapitole pracujeme převážně s bitovými řetězci. Text, obrázek i jiný soubor lze před šifrováním převést na posloupnost bitů, takže model $m \in \{0, 1\}^n$ není omezen pouze na krátké textové zprávy.

6.2 Odbočka k pravděpodobnosti

V historických šifrách se často ptáme, kolik klíčů musí záškodník vyzkoušet. Moderní kryptografie však používá také náhodně generované klíče a algoritmy, které při běhu provádějí náhodné volby. Bezpečnost potom nepopisujeme pouze větou „útok je těžký“, ale pravděpodobností, s jakou může uspět. Proto si nejprve vyložíme základní pojmy potřebné v dalších sekcích.

6.2.1 Náhodný pokus, výsledky a jevy

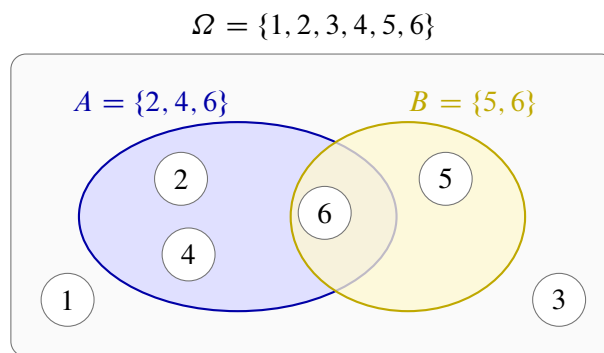
Náhodný pokus je postup, jehož konkrétní výsledek předem neznáme, přestože známe množinu všech možných výsledků. Příkladem je hod hrací kostkou, hod mincí nebo rovnoměrný výběr bitového řetězce.

Definice 6.2.1. Výběrový prostor Ω je množina všech možných výsledků náhodného pokusu. Náhodný jev je libovolná podmnožina $A \subseteq \Omega$. Je nastane právě tehdy, když výsledek pokusu patří do množiny A .

Při hodu šestistěnnou kostkou máme

$$\Omega = \{1, 2, 3, 4, 5, 6\}.$$

Jev „padne sudé číslo“ odpovídá množině $A = \{2, 4, 6\}$ a jev „padne číslo větší než čtyři“ množině $B = \{5, 6\}$. Průnik $A \cap B = \{6\}$ nastane, když nastanou oba jevy současně. Sjednocení $A \cup B = \{2, 4, 5, 6\}$ nastane, když nastane alespoň jeden z nich.



Obrázek 6.2: Výběrový prostor hodu kostkou a dva náhodné jevy.

Definice 6.2.2. Jsou-li všechny výsledky konečného výběrového prostoru Ω stejně pravděpodobné, definujeme pravděpodobnost jevu $A \subseteq \Omega$ vztahem

$$\Pr(A) = \frac{|A|}{|\Omega|}.$$

Pro sudé číslo na kostce tedy dostáváme

$$\Pr(A) = \frac{3}{6} = \frac{1}{2}.$$

Pravděpodobnost nemožného jevu je $\Pr(\emptyset) = 0$ a pravděpodobnost jistého jevu je $\Pr(\Omega) = 1$. Často ji zapisujeme také v procentech, například $\frac{1}{2} = 50\%$.

Jev, že A nenastane, značíme $A^c = \Omega \setminus A$. Platí

$$\Pr(A^c) = 1 - \Pr(A).$$

Tento jednoduchý vztah bude později velmi užitečný u narozeninového paradoxu: místo přímého počítání pravděpodobnosti, že vznikne alespoň jedna shoda, nejprve spočítáme pravděpodobnost, že nevznikne žádná.

Více kroků náhodného pokusu. Při dvou hodech mincí je potřeba rozlišovat pořadí výsledků:

$$\Omega = \{00, 01, 10, 11\},$$

kde například 01 znamená, že v prvním hodu padla nula a ve druhém jednička. Jsou-li oba hody nezávislé a obě hodnoty stejně pravděpodobné, má každý výsledek pravděpodobnost $\frac{1}{4}$.

Obecně existuje právě 2^n bitových řetězců délky n . Vybereme-li řetězec x rovnoměrně náhodně z $\{0, 1\}^n$, potom pro každý předem zvolený řetězec $a \in \{0, 1\}^n$ platí

$$\Pr_{x \in \{0,1\}^n}[x = a] = \frac{1}{2^n}.$$

Dolní index zde říká, odkud a jak náhodnou hodnotu vybíráme; výraz v hranatých závorkách popisuje jev, jehož pravděpodobnost měříme.

Příklad 6.2.3. Vyberme rovnoměrně náhodně $x \in \{0, 1\}^4$. Celkem existuje $2^4 = 16$ možností. Jev, že x začíná jedničkou, obsahuje osm řetězců, a proto má pravděpodobnost $\frac{8}{16} = \frac{1}{2}$. Jev $x = 1011$ obsahuje jediný výsledek a má pravděpodobnost $\frac{1}{16}$.

6.2.2 Podmíněná pravděpodobnost a nezávislost

Někdy už o výsledku něco víme a ptáme se, jak tato informace změní pravděpodobnost jiného jevu. Jestliže například víme, že na kostce padlo sudé číslo, zůstávají možné jen výsledky $\{2, 4, 6\}$.

Definice 6.2.4. Nechť $A, B \subseteq \Omega$ a $\Pr(B) > 0$. Podmíněnou pravděpodobnost jevu A za podmínky B definujeme jako

$$\Pr(A | B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

Pro jevy $A = \{2, 4, 6\}$ a $B = \{4, 5, 6\}$ při hodu kostkou je $A \cap B = \{4, 6\}$. Proto

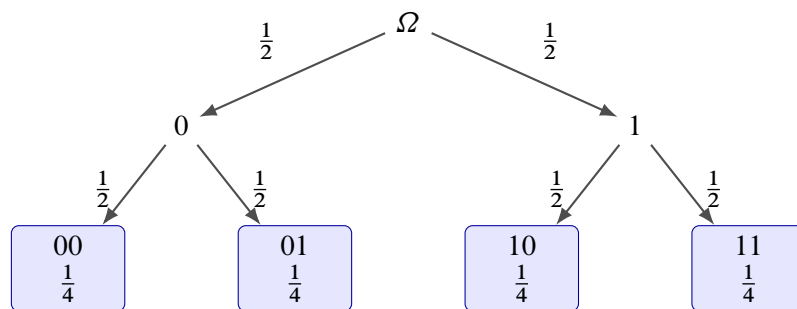
$$\Pr(A | B) = \frac{\Pr(\{4, 6\})}{\Pr(\{4, 5, 6\})} = \frac{2/6}{3/6} = \frac{2}{3}.$$

Definice 6.2.5. Jevy A a B jsou *nezávislé*, jestliže informace o nastání jednoho nemění pravděpodobnost druhého, tedy

$$\Pr(A | B) = \Pr(A).$$

Je-li $\Pr(B) > 0$, je tato podmínka ekvivalentní rovnosti

$$\Pr(A \cap B) = \Pr(A) \Pr(B).$$



Obrázek 6.3: Dva nezávislé hody mincí a pravděpodobnosti výsledných řetězců.

V kryptografii je nezávislost klíče na zprávě zásadní. Pokud by volba klíče závisela na obsahu zprávy, mohl by způsob jeho generování sám prozrazovat informace. U jednorázové šifry budeme dokonce požadovat, aby znalost šifrovaného textu nezměnila pravděpodobnost žádné původní zprávy:

$$\Pr(M = m | C = c) = \Pr(M = m).$$

Náhodné veličiny.

Definice 6.2.6. *Náhodná veličina* přiřazuje každému výsledku náhodného pokusu nějakou hodnotu. Formálně jde o zobrazení $X : \Omega \rightarrow S$, kde S je množina možných hodnot veličiny.

Písmena M , K a C budeme používat pro náhodné veličiny představující postupně zprávu, klíč a šifrový text. Zápis $M = m$ označuje jev, že náhodná veličina M nabyla konkrétní hodnoty m . Rozdíl mezi M a m je tedy podobný rozdílu mezi proměnnou a její právě zvolenou hodnotou.



Pravděpodobnostní algoritmus může při běhu používat vlastní náhodné bity. Pokud píšeme pravděpodobnost jeho úspěchu, počítáme nejen s náhodným vstupem, ale také s těmito vnitřními volbami. Stejný algoritmus proto může na stejném vstupu v různých bězích postupovat odlišně.

6.3 Historické metody

Nejstarší šifry pracovaly přímo se znaky textu. Zprávu bylo možné šifrovat ručně bez počítače, jejich jednoduchost je však zároveň slabinou. Na historických metodách dobře uvidíme, proč velký počet klíčů sám o sobě ještě nezaručuje bezpečnost.

6.3.1 Substituční šifra s posunem

Mějme abecedu

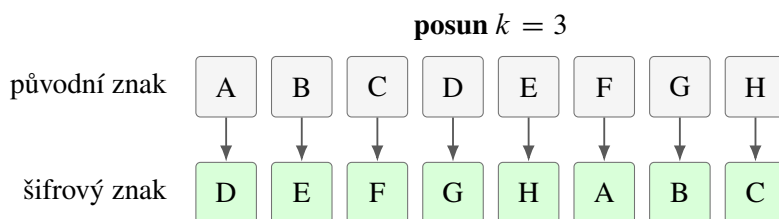
$$\Sigma = \{a_0, a_1, \dots, a_{q-1}\}$$

a tajný posun k , kde $0 < k < q$. Každý znak nahradíme znakem posunutým o k míst:

$$a_i \mapsto a_{(i+k) \bmod q}.$$

Operace modulo q způsobí, že se po dosažení konce abecedy vrátíme na její začátek. Dešifrování používá opačný posun:

$$a_j \mapsto a_{(j-k) \bmod q}.$$



Obrázek 6.4: Substituční šifra s posunem $k = 3$ na zkrácené abecedě.

Pro standardní latinskou abecedu a $k = 3$ například dostaneme

$$\text{TAJNE} \mapsto \text{WDMQH}.$$

Jestliže záškodník nezná klíč, může jednoduše vyzkoušet všech $q - 1$ nenulových posunů. Pro $q = 26$ jde pouze o 25 možností. U dostatečně dlouhé zprávy navíc snadno pozná, který výsledek připomíná smysluplný text.

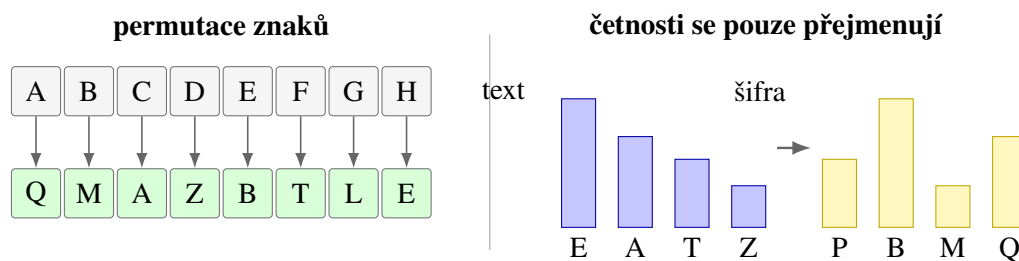
6.3.2 Obecná substituční šifra

Místo jednoho posunu můžeme zvolit libovolnou permutaci abecedy

$$\pi : \Sigma \rightarrow \Sigma.$$

Permutace je bijekce, takže každý znak má právě jeden zašifrovaný obraz a každý znak šifrové abecedy má právě jeden vzor. Šifrování nahradí každý znak a znakem $\pi(a)$, zatímco dešifrování používá inverzní permutaci π^{-1} .

Pro abecedu o q znacích existuje $q!$ různých permutací. U 26 písmen je to přibližně $4 \cdot 10^{26}$ klíčů, takže prosté vyzkoušení všech možností už není praktické. Obecná substituce však stále zachovává strukturu původního textu: stejné znaky se vždy mění na stejné znaky, opakovaná slova vytvářejí stejné vzory a četnosti písmen se pouze přejmenují.



Obrázek 6.5: Obecná substituce a zachování četností znaků.

Uvažujme například klíč, jehož prvních několik přiřazení je

$$A \mapsto Q, \quad B \mapsto W, \quad C \mapsto E, \quad D \mapsto R, \quad E \mapsto T, \quad \dots$$

Pro permutaci použitou na obrázku 6.5 dostáváme

$$\text{TAJEMSTVI} \mapsto \text{ZQPTDLZCO}.$$



Velký počet klíčů neznamená automaticky bezpečnou šifru. Obecná substituce má obrovský prostor klíčů, ale frekvenční analýza využívá vlastnosti jazyka a nemusí jednotlivé klíče zkoušet jeden po druhém.

Frekvenční analýza. V delším českém textu se některá písmena objevují výrazně častěji než jiná. Jestliže se v šifrovaném textu mimořádně často vyskytuje znak Q, může odpovídat některému běžnému písmenu otevřeného textu. Záškodník dále sleduje dvojice a trojice znaků, délky slov nebo opakované vzory. Každý takový poznatek zmenšuje množinu možných permutací.

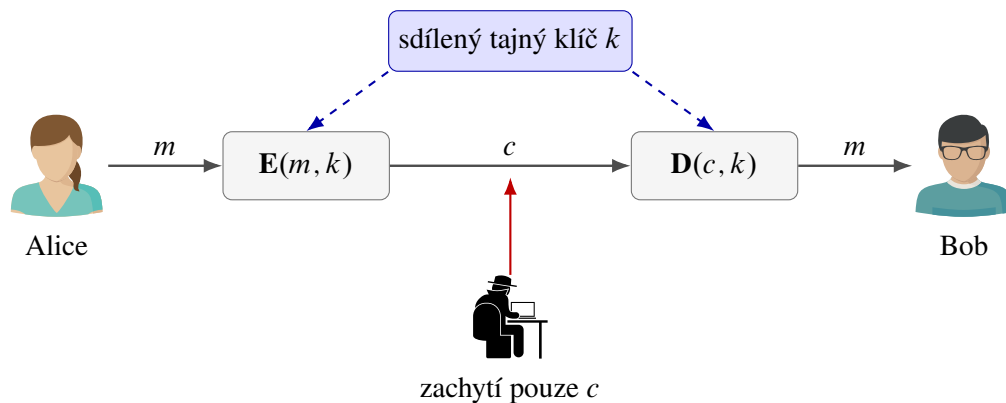
Historické metody nám proto ukazují dvě důležité zásady. Bezpečnost nelze odvozovat jen z toho, že klíčů je mnoho, a šifra by neměla zbytečně zachovávat statistickou strukturu otevřeného textu. Moderní šifry se snaží, aby bez znalosti klíče vypadal šifrový text jako náhodná data.

6.4 Symetrická kryptografie

V symetrické kryptografii používají Alice i Bob tentýž tajný klíč

$$k \in \{0, 1\}^n.$$

Alice vytvoří šifrový text $c = \mathbf{E}(m, k)$ a Bob obnoví zprávu výpočtem $m = \mathbf{D}(c, k)$. Klíč proto musí znát obě strany, ale nikdo další.



Obrázek 6.6: Základní schéma symetrického šifrování.

Symetrické algoritmy jsou obvykle velmi rychlé a hodí se pro šifrování velkého množství dat. Hlavní organizační problém spočívá v bezpečném předání klíče. Pokud Alice dokáže Bobovi tajně poslat dlouhý klíč, nabízí se otázka, proč stejným bezpečným kanálem neposlat přímo zprávu. Tento problém bude zvlášť dobře vidět u jednorázové šifry.

6.4.1 Operace XOR

Jednorázová šifra používá operaci XOR, značenou symbolem $\oplus$. Pro jednotlivé bity platí

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

Operaci na stejně dlouhých bitových řetězcích provádíme po jednotlivých pozicích. Důležitá je rovnost

$$(a \oplus b) \oplus b = a,$$

protože $b \oplus b = 0$ a $a \oplus 0 = a$.

6.4.2 Jednorázová šifra

Alice a Bob sdílejí rovnoměrně náhodný tajný klíč

$$k \in \{0, 1\}^n,$$

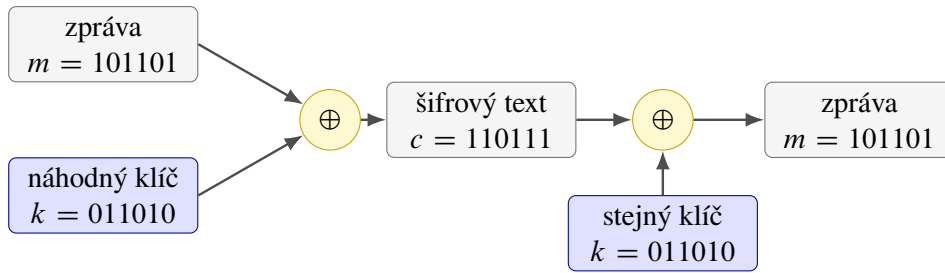
který je stejně dlouhý jako zpráva $m \in \{0, 1\}^n$. Alice šifruje

$$c = m \oplus k.$$

Bob použije stejný klíč a spočítá

$$c \oplus k = (m \oplus k) \oplus k = m.$$

Anglicky se tato metoda nazývá *one-time pad*; český název zdůrazňuje, že každý klíč smí být použit právě jednou.



Obrázek 6.7: Šifrování a dešifrování jednorázovou šifrou.

Tvrzení 6.4.1. Nechť je K rovnoměrně náhodný na $\{0, 1\}^n$ a nezávislý na náhodné veličině M . Položme $C = M \oplus K$. Potom pro každé $m, c \in \{0, 1\}^n$ s $\Pr(M = m) > 0$ platí

$$\Pr(M = m \mid C = c) = \Pr(M = m).$$

Důkaz. Pro pevně zvolené hodnoty m a c existuje právě jeden klíč, který zprávu m převede na c , totiž

$$k = m \oplus c.$$

Protože je K rovnoměrně náhodný a nezávislý na M , dostáváme

$$\Pr(M = m \text{ a } C = c) = \Pr(M = m) \Pr(K = m \oplus c) = \frac{\Pr(M = m)}{2^n}.$$

Dále sečteme přes všechny možné zprávy m' :

$$\Pr(C = c) = \sum_{m' \in \{0, 1\}^n} \frac{\Pr(M = m')}{2^n} = \frac{1}{2^n}.$$

Z definice podmíněné pravděpodobnosti tedy plyne

$$\Pr(M = m \mid C = c) = \frac{\Pr(M = m \text{ a } C = c)}{\Pr(C = c)} = \Pr(M = m).$$

□

Zachycený šifrový text tedy nezmění pravděpodobnost žádné původní zprávy. Jednorázová šifra poskytuje *perfektní utajení*: její bezpečnost není založena na omezeném výpočetním výkonu záškodníka. Platí dokonce proti protivníkovi s libovolně výkonným počítačem.



Perfektní utajení platí pouze tehdy, když je klíč skutečně náhodný, stejně dlouhý jako zpráva, nezávislý na zprávě, bezpečně sdílený a použitý právě jednou.

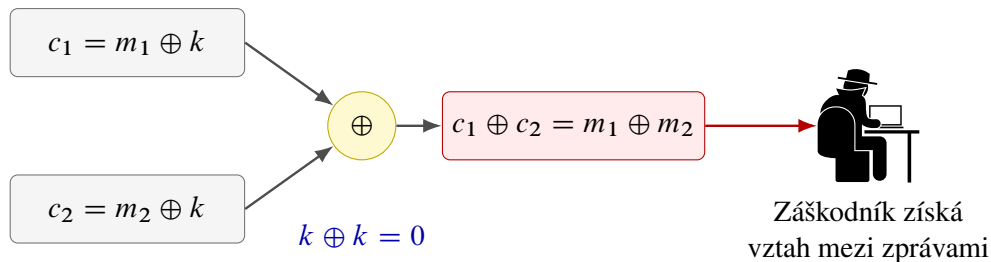
6.4.3 Proč se klíč nesmí opakovat?

Zašifruje-li Alice dvě zprávy tímž klíčem k , dostaneme

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k.$$

Záškodník může oba zachycené texty spojit:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2.$$

Obrázek 6.8: Opakované použití klíče odstraní z výrazu neznámé k .

Klíč se vyruší. Záškodník sice nemusí okamžitě znát obě zprávy, získá však informaci o jejich vzájemném vztahu. Pokud jednu z nich zná nebo správně odhadne, dopočítá druhou.

Nevýhodou jednorázové šifry je nutnost bezpečně sdílet náhodný klíč délky celé zprávy. V sekci 6.7 uvidíme, jak lze perfektní bezpečnost nahradit bezpečností výpočetní a dlouhý náhodný klíč krátkým seedem.

6.5 Asymetrická kryptografie

Symetrické šifrování předpokládá, že Alice a Bob už sdílejí tajný klíč. Asymetrická kryptografie tento požadavek mění. Každý uživatel si vytvoří dvojici vzájemně souvisejících klíčů:

veřejný klíč k_V a soukromý klíč k_S .

Veřejný klíč smí znát kdokoli, soukromý klíč si jeho vlastník musí ponechat pouze pro sebe.

Chce-li Alice poslat zprávu Bobovi, získá Bobův veřejný klíč k_V a spočítá

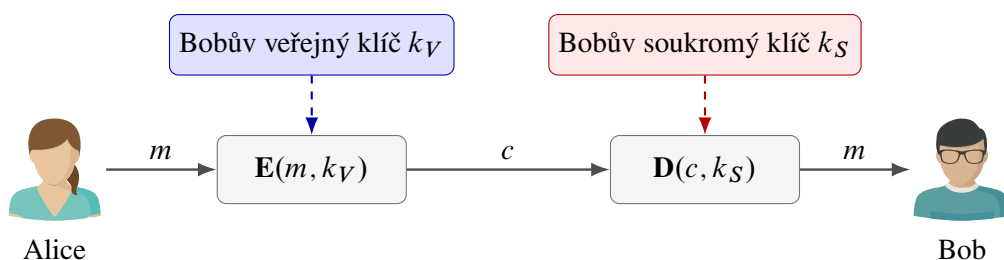
$$c = \mathbf{E}(m, k_V).$$

Bob použije svůj soukromý klíč:

$$m = \mathbf{D}(c, k_S).$$

Korektnost požaduje

$$\mathbf{D}(\mathbf{E}(m, k_V), k_S) = m.$$



Obrázek 6.9: Šifrování Bobovým veřejným klíčem a dešifrování jeho soukromým klíčem.

Znalost veřejného klíče umožňuje zprávy pro Boba šifrovat, nesmí však umožnit je efektivně dešifrovat ani odvodit k_S . Asymetrie tedy není v tom, že by jedna operace byla matematicky nemožná, ale v rozdílu výpočetní obtížnosti: s tajnou informací je dešifrování snadné, bez ní má být prakticky neproveditelné.



Veřejný klíč musí být spolehlivě spojen s identitou svého vlastníka. Pokud záškodník podstrčí Alici svůj klíč a vydává jej za Bobův, může zprávu přečíst a teprve potom ji znovu zašifrovat skutečným Bobovým klíčem. K ověřování vazby mezi identitou a veřejným klíčem se používají certifikáty.

Asymetrické operace bývají výrazně pomalejší než symetrické. Reálné systémy proto často používají oba přístupy společně: asymetrická kryptografie bezpečně ustaví krátký dočasný klíč a samotná data se potom šifrují rychlou symetrickou šifrou. V této kapitole se soustředíme na princip; podrobnosti konkrétních síťových protokolů ponecháme stranou.

6.5.1 Výpočetní bezpečnost

U jednorázové šifry jsme dokázali, že šifrový text neposkytuje žádnou informaci ani neomezeně silnému záškodníkovi. U asymetrické kryptografie tak silný požadavek splnit nemůžeme: veřejný klíč i algoritmus šifrování jsou dostupné, takže záškodník může kandidátní zprávy sám šifrovat a porovnávat.

Bezpečnost proto formulujeme vůči algoritmům s omezenou dobou výpočtu. Pravděpodobnost úspěchu každého pravděpodobnostního polynomiálního útoku má být zanedbatelná. Tato myšlenka se naplno projeví v sekci 6.7.



Jednoduchý zápis $c = E(m, k_V)$ nezobrazuje všechny praktické podrobnosti. Bezpečné asymetrické šifrování obvykle používá také náhodnost a předepsané kódování zprávy, aby stejné zprávy nevytvářely pokaždé stejný šifrový text.

6.6 Algoritmus RSA

RSA je nejznámější příklad asymetrického kryptografického systému. Jeho název vznikl z příjmení autorů Rona Rivesta, Adiho Shamira a Leonarda Adlemana. Základní operací je umocňování modulo součin dvou velkých prvočísel.

6.6.1 Vytvoření klíčů

Bob postupuje následovně:

1. zvolí dvě různá velká prvočísla p a q a spočítá

$$n = pq;$$

2. spočítá

$$\varphi(n) = (p - 1)(q - 1);$$

3. zvolí číslo s nesoudělné s $\varphi(n)$;
4. nalezne číslo v takové, že

$$vs \bmod \varphi(n) = 1;$$

5. zveřejní klíč $k_V = (v, n)$ a uchová v tajnosti klíč $k_S = (s, n)$.

Číslo v je multiplikativní inverze čísla s modulo $\varphi(n)$. Existuje právě proto, že $\gcd(s, \varphi(n)) = 1$, a lze je efektivně nalézt rozšířeným Eukleidovým algoritmem.



Číslo s je soukromý exponent, nikoli třetí prvočíslo. Musí být nesoudělné s $(p - 1)(q - 1)$, ale samo prvočíslem být nemusí.

6.6.2 Šifrování a dešifrování

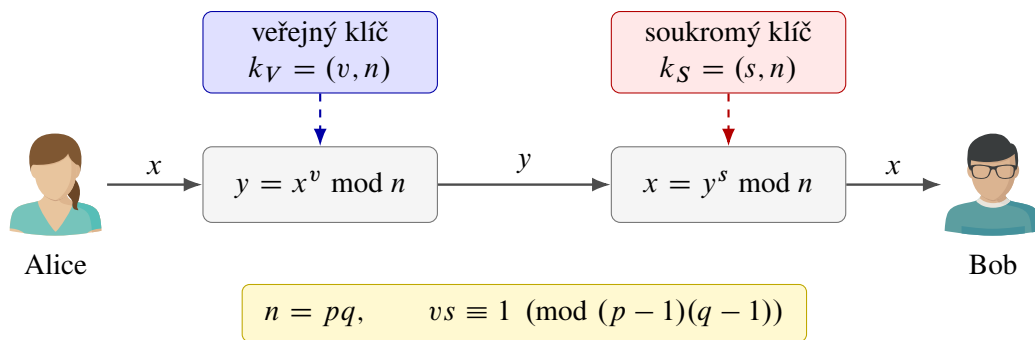
Zprávu reprezentujeme číslem x splňujícím $0 \leq x < n$. Alice zašifruje

$$y = \mathbf{E}(x, k_V) = x^v \bmod n.$$

Bob dešifruje

$$\mathbf{D}(y, k_S) = y^s \bmod n.$$

Mocniny mohou být obrovské, v programu je však nepočítáme nejprve jako běžná celá čísla. Opakovaným umocňováním a průběžným výpočtem zbytku lze modulární mocninu určit v čase polynomiálním vzhledem k počtu bitů vstupu.



Obrázek 6.10: Schéma šifrování a dešifrování v RSA.

6.6.3 Malá Fermatova věta a čínská věta o zbytcích

Korektnost RSA odvodíme ze dvou vět elementární teorie čísel.

Věta 6.6.1 (Malá Fermatova věta). Nechť p je prvočíslo a $a \in \mathbb{Z}$ není dělitelné číslem p . Potom

$$a^{p-1} \bmod p = 1.$$

Ekvivalentně pro každé $a \in \mathbb{Z}$ platí $a^p \bmod p = a \bmod p$.

Věta 6.6.2 (Čínská věta o zbytcích). Nechť $m_1, \dots, m_k \in \mathbb{N}$ jsou po dvou nesoudělná. Pro libovolná $a_1, \dots, a_k \in \mathbb{Z}$ existuje řešení soustavy

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \dots, k,$$

a všechna její řešení jsou navzájem shodná modulo $M = m_1 \cdots m_k$.

Důsledek 6.6.3. Jsou-li p a q různá prvočísla a platí

$$a \equiv b \pmod{p} \quad \text{a} \quad a \equiv b \pmod{q},$$

potom $a \equiv b \pmod{pq}$.

6.6.4 Korektnost RSA

Věta 6.6.4. Pro klíče vytvořené uvedeným postupem a každé celé číslo x splňující $0 \leq x < n$ platí

$$\mathbf{D}(\mathbf{E}(x, k_V), k_S) = x.$$

Důkaz. Z rovnosti $vs \bmod (p-1)(q-1) = 1$ plyne, že pro nějaké $t \in \mathbb{N}_0$ platí

$$vs = 1 + t(p-1)(q-1).$$

Nejprve ukážeme $x^{vs} \equiv x \pmod{p}$. Pokud p dělí x , jsou obě strany kongruence nulové modulo p . Pokud p číslo x nedělí, použijeme větu 6.6.1:

$$\begin{aligned} x^{vs} &= x^{1+t(p-1)(q-1)} \\ &= x \left(x^{p-1}\right)^{t(q-1)} \equiv x \cdot 1^{t(q-1)} \equiv x \pmod{p}. \end{aligned}$$

Stejným rozlišením případů dostaneme

$$x^{vs} \equiv x \pmod{q}.$$

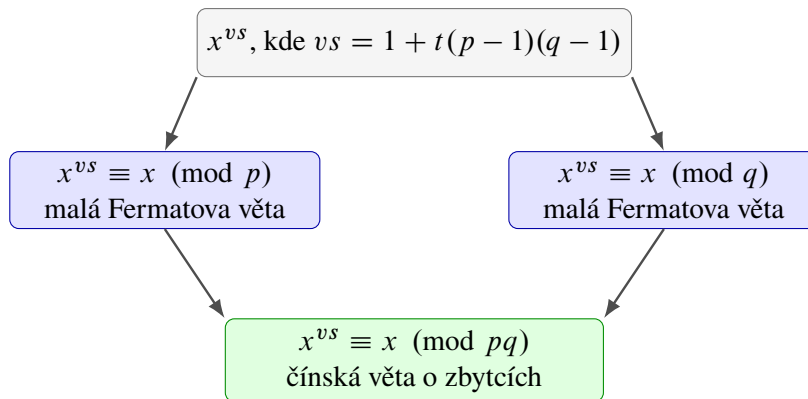
Podle důsledku 6.6.3 tedy

$$x^{vs} \equiv x \pmod{pq}.$$

Protože $n = pq$ a $0 \leq x < n$, uzavíráme

$$\mathbf{D}(\mathbf{E}(x, k_V), k_S) = (x^v \bmod n)^s \bmod n = x^{vs} \bmod n = x.$$

□



Obrázek 6.11: Myšlenka důkazu korektnosti RSA přes moduly p , q a pq .

Příklad s malými čísly.

Příklad 6.6.5. Zvolme $p = 5$, $q = 11$ a $s = 27$. Potom

$$n = 55, \quad \varphi(n) = (5-1)(11-1) = 40.$$

Protože $\gcd(27, 40) = 1$, můžeme nalézt v splňující

$$27v \bmod 40 = 1.$$

Vyhovuje $v = 3$, neboť $27 \cdot 3 = 81 \equiv 1 \pmod{40}$. Veřejný klíč je $k_V = (3, 55)$ a soukromý klíč $k_S = (27, 55)$.

Alice zašifruje $x = 7$:

$$y = 7^3 \bmod 55 = 343 \bmod 55 = 13.$$

Bob poté získá

$$13^{27} \bmod 55 = 7.$$

Malá čísla slouží pouze k ručnímu ověření postupu. V reálné kryptografii by tak malý modul záškodník rozložil okamžitě.

6.6.5 Bezpečnost a faktorizace

Veřejný klíč obsahuje $n = pq$, ale nikoli samotná prvočísla p a q . Pokud záškodník rozklad zjistí, spočítá $\varphi(n) = (p-1)(q-1)$ a z veřejného exponentu v odvodí soukromý exponent s . Efektivní faktorizace by tedy RSA prolomila.

Rozhodovací verzi problému můžeme zapsat takto:

FACTOR : pro daná $n, b \in \mathbb{N}$ rozhodni, zda existuje d takové, že $1 < d < b$ a $d \mid n$.

Jak jsme viděli v sekci 2.6, kladnou odpověď stačí doložit dělitelem d . Ověření nerovností i rovnosti $n \bmod d = 0$ proběhne v polynomiálním čase, a proto FACTOR $\in$ NP.

Naivní zkoušení dělitelů je vzhledem k bitové délce $|n|_B$ exponenciální. Jsou známy podstatně rychlejší klasické algoritmy, žádný obecný polynomiální klasický algoritmus pro faktorizaci však znám není.



Znalost efektivní faktorizace stačí k prolomení popsaného RSA. Není však dokázáno, že každý způsob, jak invertovat RSA, musí zároveň faktorizovat n . Bezpečnost RSA proto nelze jednoduše ztotožnit s větou „faktorizace je těžká“.



Uvedené „učebnicové RSA“ je deterministické a bez doplňujícího kódování není bezpečným praktickým šifrovacím schématem. Skutečné systémy používají standardizované náhodné vycpávání a oddělené postupy, například OAEP pro šifrování a PSS pro podpis.

6.7 Pseudonáhodné generátory a jednosměrné funkce

Jednorázová šifra ukázala, že dlouhý rovnoměrně náhodný klíč dokáže skrýt zprávu dokonale. Zároveň jsme narazili na její zásadní omezení: pro zprávu délky n potřebujeme předem sdílet n náhodných bitů. Moderní kryptografie se proto vzdává absolutního požadavku, aby šifrový text neobsahoval vůbec žádnou informaci, a nahrazuje jej požadavkem výpočetním. Připouštíme, že informace může být v principu zjistitelná, ale žádný dostatečně rychlý algoritmus ji nemá umět prakticky využít.

Tento posun je zásadní. Výpočetní složitost zde není překážkou, kterou se snažíme odstranit, nýbrž prostředkem ochrany. Oprávněný uživatel zná krátký klíč nebo zvláštní tajnou informaci a výpočet provede rychle. Záškodník stejnou informaci nemá a stojí před úlohou, pro kterou neznáme efektivní postup.

Technické definice v této sekci vyjadřují tři myšlenky:

1. oprávněný výpočet musí probíhat v polynomiálním čase;
2. záškodník smí používat libovolný pravděpodobnostní polynomiální algoritmus;
3. pravděpodobnost jeho úspěchu musí s rostoucí délkou vstupu klesat rychleji než převrácená hodnota libovolného polynomu.

Poslední bod zachytíme pojmem zanedbatelné funkce.

6.7.1 Zanedbatelná funkce

Definice 6.7.1. Funkce $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ je *zanedbatelná*, jestliže pro každé $k \in \mathbb{N}$ existuje $n_k \in \mathbb{N}$ takové, že pro každé $n > n_k$ platí

$$|\varepsilon(n)| < \frac{1}{n^k}.$$

Zanedbatelná funkce tedy nakonec klesne pod $\frac{1}{n}$, pod $\frac{1}{n^2}$, pod $\frac{1}{n^{100}}$ i pod převrácenou hodnotu jakéhokoliv jiného pevně zvoleného polynomu. Typickým příkladem je 2^{-n} . Naopak funkce $\frac{1}{n^{10}}$ zanedbatelná není, protože pro $k = 11$ nikdy dlouhodobě neklesne pod $\frac{1}{n^{11}}$.

Číslo n zde hraje roli *bezpečnostního parametru*. Jeho zvětšením prodlužujeme klíče a zvyšujeme množství práce potřebné k útoku. Tvrzení o zanedbatelné pravděpodobnosti neznamená, že útok nemůže nikdy uspět; říká, že jeho šanci lze volbou dostatečně velkého parametru stlačit pod každou prakticky významnou inverzní polynomiální mez.

6.7.2 Pseudonáhodný generátor

Deterministický program nedokáže vytvořit skutečnou náhodnost z ničeho. Jakmile známe jeho vstup, je výstup pevně určen. Může však z krátkého náhodného vstupu vytvořit delší řetězec, který žádný efektivní algoritmus neumí od skutečně náhodného řetězce rozlišit.

Definice 6.7.2. Funkce

$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$$

je *pseudonáhodný generátor* (PRG), jestliže splňuje následující podmínky:

1. G je vyčíslitelná deterministickým polynomiálním algoritmem;
2. $\ell(n) > n$ pro každé $n \in \mathbb{N}$;
3. pro každý pravděpodobnostní polynomiální algoritmus $\mathcal{A}$ existuje zanedbatelná funkce $\varepsilon(n)$ taková, že

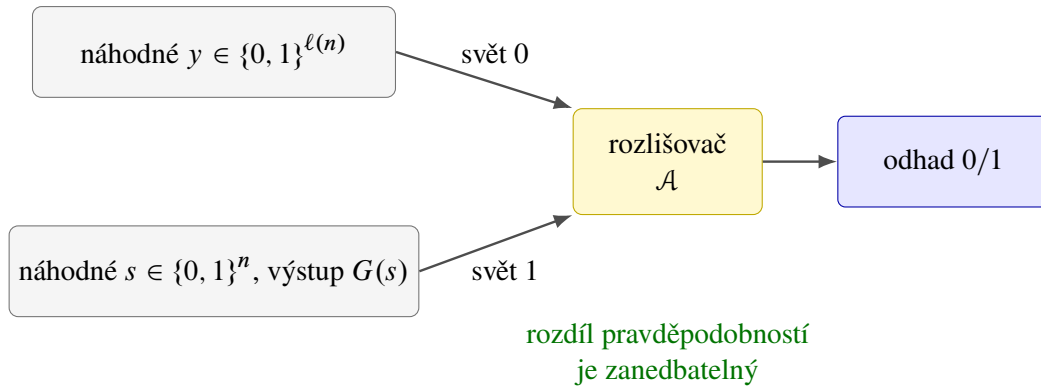
$$\left| \Pr_{y \in \{0,1\}^{\ell(n)}} [\mathcal{A}(y) = 1] - \Pr_{y \in \{0,1\}^n} [\mathcal{A}(G(y)) = 1] \right| \leq \varepsilon(n).$$

Náhodný vstup $s \in \{0, 1\}^n$ nazýváme *seed*. Druhá podmínka zajišťuje *prodloužení* (*stretch*): výstup $G(s)$ obsahuje více bitů než seed. Nejdůležitější je třetí podmínka. Algoritmus $\mathcal{A}$ obdrží řetězec délky $\ell(n)$ a má rozhodnout, zda byl vybrán rovnoměrně náhodně, nebo zda vznikl jako $G(s)$ z náhodného seedu. Rozdíl jeho pravděpodobností přijetí v obou případech se nazývá rozlišovací výhoda.

Podmínka neříká, že jsou obě rozdělení stejná. Výstupů G může být nejvýše 2^n , zatímco všech řetězců délky $\ell(n)$ je $2^{\ell(n)}$. Většina dlouhých řetězců proto vůbec nemusí být výstupem generátoru. Rozdíl je informačně zjistitelný, avšak efektivní algoritmus nemá umět poznat, do které skupiny předložený řetězec patří.



Pseudonáhodnost není vlastnost jediného konkrétního řetězce. Je to vlastnost způsobu generování a říká, že celé rozdělení výstupů je pro efektivního pozorovatele nerozlišitelné od rovnoměrného rozdělení.



Obrázek 6.12: Experiment rozlišující skutečně náhodný řetězec od výstupu PRG.

6.7.3 Žádné pseudonáhodné generátory při $P = NP$

Tvrzení 6.7.3. Jestliže $P = NP$, pak neexistuje žádný pseudonáhodný generátor.

Důkaz. Předpokládejme, že $P = NP$, a vezměme libovolnou polynomiálně vyčíslitelnou funkci

$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}, \quad \ell(n) > n.$$

Pro bezpečnostní parametr n označme množinu všech jejích možných výstupů

$$L_{G,n} = \{z \in \{0, 1\}^{\ell(n)} \mid \text{existuje } s \in \{0, 1\}^n \text{ takové, že } G(s) = z\}.$$

Ověřit, že $z \in L_{G,n}$, lze v nedeterministickém polynomiálním čase: certifikátem je seed $s \in \{0, 1\}^n$ a ověřovatel pouze spočítá $G(s)$ a porovná výsledek se z . Při předpokladu $P = NP$ proto existuje deterministický polynomiální algoritmus $\mathcal{D}$, který rozhodne, zda $z \in L_{G,n}$:

$$\mathcal{D}(z) = 1 \iff z \in L_{G,n}.$$

Pokud rozlišovač dostane řetězec $G(s)$ vytvořený z náhodného seedu, přijme jej vždy:

$$\Pr_{s \in \{0,1\}^n} [\mathcal{D}(G(s)) = 1] = 1.$$

Množina $L_{G,n}$ obsahuje nejvýše 2^n řetězců, protože existuje pouze 2^n seedů. Pro rovnoměrně náhodný řetězec $z \in \{0, 1\}^{\ell(n)}$ tedy platí

$$\Pr_{z \in \{0,1\}^{\ell(n)}} [\mathcal{D}(z) = 1] = \frac{|L_{G,n}|}{2^{\ell(n)}} \leq \frac{2^n}{2^{\ell(n)}} = 2^{n-\ell(n)}.$$

Celá rozlišovací výhoda algoritmu $\mathcal{D}$ je proto

$$\begin{aligned} & \left| \Pr_{z \in \{0,1\}^{\ell(n)}} [\mathcal{D}(z) = 1] - \Pr_{s \in \{0,1\}^n} [\mathcal{D}(G(s)) = 1] \right| \\ &= \left| \frac{|L_{G,n}|}{2^{\ell(n)}} - 1 \right| = 1 - \frac{|L_{G,n}|}{2^{\ell(n)}} \\ &\geq 1 - \frac{2^n}{2^{\ell(n)}} = 1 - 2^{n-\ell(n)} \geq 1 - \frac{1}{2} = \frac{1}{2}. \end{aligned}$$

Poslední nerovnost plyne z $\ell(n) > n$, a tedy $\ell(n) \geq n + 1$. Konstantní funkce $\frac{1}{2}$ není zanedbatelná, takže G nesplňuje podmínku nerozlišitelnosti z definice PRG. Protože jsme takto vyloučili libovolného kandidáta G , při rovnosti $P = NP$ žádný pseudonáhodný generátor neexistuje. $\square$



Jde o obměněnou implikaci tvrzení „existence PRG implikuje $P \neq NP$ “. Samotná nerovnost $P \neq NP$ však existenci pseudonáhodných generátorů nezaručuje.

6.7.4 Jednosměrná funkce

Pseudonáhodný generátor předpokládá existenci nějakého zdroje výpočetní obtížnosti. Tento zdroj formálně zachycují jednosměrné funkce: dopředný výpočet je snadný, ale návrat k libovolnému vzoru typického výstupu je pro efektivního záškodníka neproveditelný.

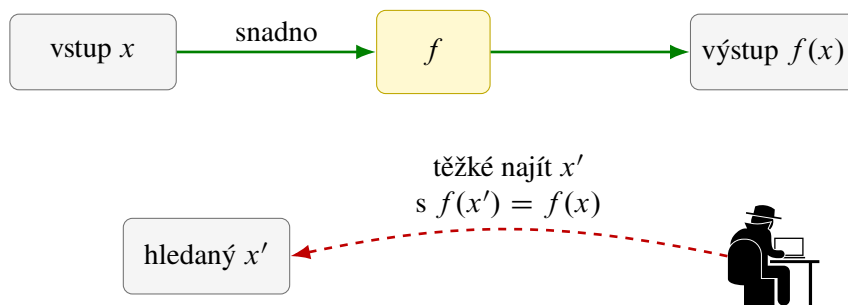
Definice 6.7.4. Zobrazení

$$f : \{0, 1\}^* \rightarrow \{0, 1\}^*$$

je *jednosměrná funkce* (OWF), jestliže:

1. hodnotu $f(x)$ lze pro každé $x \in \{0, 1\}^*$ spočítat v polynomiálním čase;
2. pro každý pravděpodobnostní polynomiální algoritmus $\mathcal{A}$ existuje zanedbatelná funkce $\varepsilon(n)$ taková, že pro každé $n \in \mathbb{N}$ platí

$$\Pr_{x \in \{0, 1\}^n} [f(\mathcal{A}(f(x))) = f(x)] \leq \varepsilon(n).$$



Obrázek 6.13: Dopředný výpočet jednosměrné funkce a obtížné hledání vzoru.

V experimentu nejprve rovnoměrně vybereme $x \in \{0, 1\}^n$, spočítáme $f(x)$ a výsledek předáme algoritmu $\mathcal{A}$. Algoritmus nemusí nalézt původní x . Uspěje i tehdy, když vrátí jiné x' se stejným obrazem, tedy $f(x') = f(x)$. Definice proto skutečně měří schopnost nalézt libovolný vzor, nikoli schopnost uhodnout jednu předem určenou reprezentaci.

První podmínka je nutná pro oprávněné použití: funkci musíme umět rychle vyčíslit. Druhá podmínka nepožaduje obtížnost pouze na jednom pečlivě vybraném vstupu. Vstup x je náhodný a úspěch musí být zanedbatelný v průměru nad všemi takovými volbami i nad náhodnými kroky algoritmu $\mathcal{A}$. To je pro kryptografii podstatné. Funkce, která je těžká pouze na několika výjimečných vstupech, by nechránila běžně generované klíče.



Nejhorší případ známý z teorie složitosti pro kryptografii nestačí. Systém není bezpečný, pokud záškodník snadno prolomí třeba polovinu náhodně generovaných klíčů, i kdyby zbývající případy byly mimořádně obtížné.

Kandidáti a padací vrátka. Není dokázáno, že jednosměrné funkce existují. Známe však přirozené kandidáty:

- násobení dvou velkých prvočísel $f(p, q) = pq$; dopředný výpočet je snadný, zatímco zpětný rozklad součinu je považován za obtížný;
- varianty problému součtu podmnožiny;

- modulární umocňování, jehož inverze souvisí s problémem diskretního logaritmu.

U žádného z těchto kandidátů dnes neumíme bez dalších předpokladů dokázat požadovanou průměrnou obtížnost.

Pro asymetrickou kryptografii potřebujeme ještě něco navíc. Vlastník soukromého klíče musí mít tajnou informaci, která obtížnou inverzi zjednoduší. Takové informaci se říká *padací vrátka* (*trapdoor*). RSA lze didakticky chápat právě tímto způsobem: bez znalosti p a q vypadá inverze modulárního umocňování obtížně, zatímco vlastník soukromého exponentu s dešifruje efektivně.

6.7.5 Souvislost OWF, PRG a šifrování

Mezi oběma technickými pojmy platí hluboká věta:

Věta 6.7.5. Jednosměrné funkce existují právě tehdy, když existují pseudonáhodné generátory.

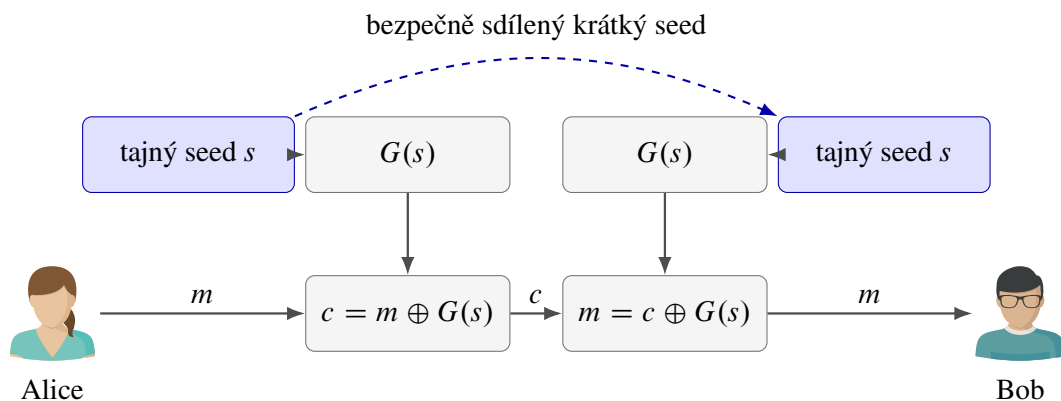
Úplný důkaz je výrazně nad rámec tohoto textu. Důležitý je jeho význam: schopnost provést snadný dopředný, ale průměrně obtížně invertovatelný výpočet stačí k postupné konstrukci bitů, které vypadají náhodně. Pseudonáhodné generátory tedy nejsou nezávislý zázrak; jejich existence je založena na téměř druhu výpočetní obtížnosti jako velká část moderní kryptografie.

V praxi můžeme krátký tajný seed $s \in \{0, 1\}^n$ rozvinout na dlouhý řetězec $G(s) \in \{0, 1\}^{\ell(n)}$ a šifrovat podobně jako jednorázovou šifrou:

$$c = m \oplus G(s).$$

Bob se stejným seedem vygeneruje tentýž proud a spočítá

$$c \oplus G(s) = m.$$



Obrázek 6.14: Použití krátkého seedu k vytvoření dlouhého šifrovacího proudu.

Oproti jednorázové šifře už výstup $G(s)$ není skutečně náhodný, a proto nezískáváme perfektní utajení. Pokud jej však žádný efektivní algoritmus nerozliší od náhodného řetězce, získáváme bezpečnost výpočetní. Tato výměna dramaticky zmenší množství tajných dat, která musí Alice a Bob sdílet.



Samotný deterministický proud $G(s)$ se nesmí bez dalšího opakovat pro více zpráv. Praktické proudové šifry kombinují klíč s jedinečnou veřejnou hodnotou, například nonce, aby pro každé šifrování vznikl jiný proud bitů.

6.8 Hešovací funkce

Šifrování převádí zprávu na šifrový text, který má oprávněný příjemce znovu dešifrovat. Hešovací funkce má jiný účel. Z libovolně dlouhých dat vytvoří krátký otisk pevné délky. Z otisku se původní data běžně neobnovují; používá se především k rychlému porovnání dat, kontrole integrity a jako součást dalších kryptografických konstrukcí.

Definice 6.8.1. Hešovací funkce je zobrazení

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

Pro $x \in \{0, 1\}^*$ nazýváme hodnotu $\mathcal{H}_n(x)$ *hash* nebo *otisk* řetězce x .

Index n připomíná délku výstupu. Ať je vstupem krátké heslo, rozsáhlý dokument nebo obraz disku, výsledkem je vždy právě n bitů. Kryptografická hešovací funkce má být rychle vyčíslitelná a malá změna vstupu má typicky změnit velkou část výstupních bitů. Tato citlivost se často označuje jako lavinový efekt.



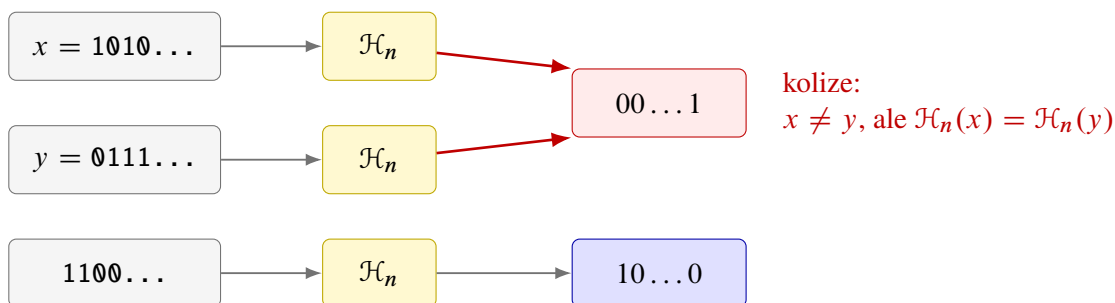
Lavinový efekt je užitečná konstrukční vlastnost, sám o sobě však nedokazuje bezpečnost. Funkce může na pohled měnit výstup chaoticky a přesto umožňovat efektivní hledání vzorů nebo kolizí.

6.8.1 Kolize jsou nevyhnutelné

Množina $\{0, 1\}^*$ je nekonečná, zatímco množina $\{0, 1\}^n$ obsahuje pouze 2^n prvků. Z Dirichletova principu proto plyne, že existují různá data $x \neq y$ se stejným otiskem:

$$\mathcal{H}_n(x) = \mathcal{H}_n(y).$$

Takovou dvojici nazýváme *kolize*.



Obrázek 6.15: Více vstupů se musí zobrazit na tentýž n -bitový otisk.

Kolizím tedy nelze zabránit matematicky. Bezpečnostní požadavek musí znít jinak: kolize existují, ale žádný efektivní algoritmus je nemá umět nalézt s nezanedbatelnou pravděpodobností.

6.8.2 Narozeninový paradox

Nejprve uvažujme skupinu k lidí a zjednodušující předpoklad, že každý z 365 dnů narození je stejně pravděpodobný a výběry jsou nezávislé. Pravděpodobnost, že všech k narozenin připadne na různé dny, je

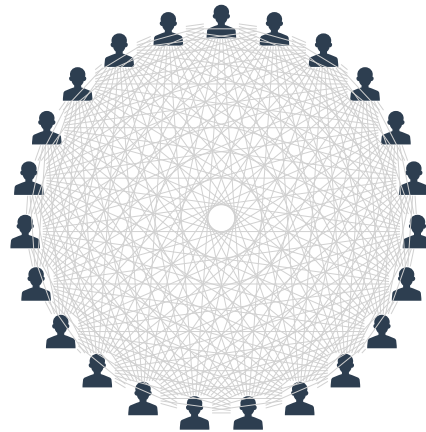
$$\Pr(C = 0) = \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - k + 1}{365}.$$

Pravděpodobnost alespoň jedné shody proto získáme doplnkem:

$$\Pr(C > 0) = 1 - \prod_{i=0}^{k-1} \frac{365 - i}{365}.$$

Už pro $k = 23$ vychází přibližně 50,7 %. Důvodem je počet dvojic: ve skupině 23 lidí jich existuje

$$\binom{23}{2} = 253.$$



23 lidí vytváří $\binom{23}{2} = 253$ dvojic

Obrázek 6.16: U 23 lidí porovnáváme 253 různých dvojic.

Stejný výpočet použijeme na r -bitové otisky. Předpokládejme pro hrubý odhad, že se otisky chovají jako nezávislé rovnoměrně náhodné hodnoty z množiny o 2^r prvcích. Pro k různých zpráv je pravděpodobnost alespoň jedné kolize

$$\begin{aligned} \Pr(C > 0) &= 1 - \prod_{j=1}^{k-1} \left(1 - \frac{j}{2^r}\right) \\ &\approx 1 - \exp\left(-\frac{k(k-1)}{2^{r+1}}\right). \end{aligned}$$

Použili jsme přibližnou rovnost $1 - x \approx e^{-x}$ pro malé x a součet

$$\sum_{j=1}^{k-1} j = \frac{k(k-1)}{2}.$$

Pro $k \approx 2^{r/2}$ dostáváme

$$\Pr(C > 0) \approx 1 - e^{-1/2} \approx 0,3935.$$

K pravděpodobnosti blízké jedné polovině stačí jen o konstantní faktor více pokusů. Hrubou silou lze proto kolizi r -bitové hešovací funkce hledat řádově v čase $2^{r/2}$, nikoli až 2^r .



Výstup délky r bitů poskytuje proti obecnému hledání kolizí přibližně jen $r/2$ bitů bezpečnosti. Právě proto kryptografické hešovací funkce používají poměrně dlouhé otisky.

6.8.3 Bezpečnostní vlastnosti

Uvažujme rodinu hešovacích funkcí

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

a pravděpodobnostní polynomiální algoritmus $\mathcal{A}$, který představuje záškodníka.

Odolnost vůči nalezení vzoru. Záškodník dostane otisk náhodně zvoleného vstupu a snaží se najít libovolná data se stejným otiskem. Formálně požadujeme, aby pro nějakou zanedbatelnou funkci ε platilo

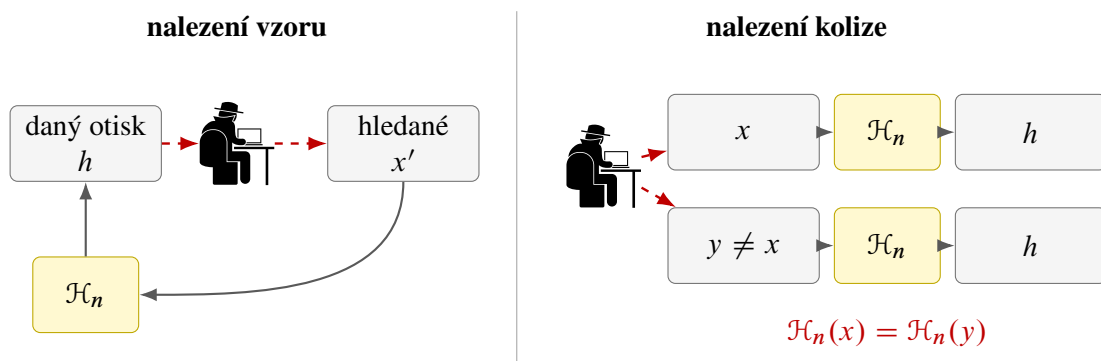
$$\Pr_{x \in \{0, 1\}^n} [\mathcal{H}_n(\mathcal{A}(\mathcal{H}_n(x))) = \mathcal{H}_n(x)] \leq \varepsilon(n).$$

Algoritmus nemusí vrátit původní x ; úspěchem je jakýkoliv vzor daného otisku.

Odolnost vůči kolizím. Záškodník se snaží vytvořit libovolné dva různé vstupy se stejným otiskem. Požadujeme

$$\Pr \left[\begin{array}{l} \mathcal{A} \text{ vypíše } (x, y), \\ x \neq y \text{ a } \mathcal{H}_n(x) = \mathcal{H}_n(y) \end{array} \right] \leq \varepsilon(n),$$

kde pravděpodobnost počítáme přes náhodné volby algoritmu $\mathcal{A}$.



Obrázek 6.17: Rozdíl mezi hledáním vzoru daného otisku a hledáním libovolné kolize.

Tyto požadavky nelze zaměňovat s pouhou malou pravděpodobností, že se srazí dva náhodně vybrané vstupy. Odolnost vůči kolizím se ptá na aktivního záškodníka, který vstupy volí záměrně a může využít strukturu algoritmu $\mathcal{H}_n$.

Vztah k jednosměrným funkcím. Odolnost vůči nalezení vzoru připomíná definici jednosměrné funkce: dopředný výpočet $\mathcal{H}_n(x)$ je rychlý, ale z výstupu má být těžké získat libovolný odpovídající vstup. Bezpečná hešovací funkce navíc požaduje obtížné hledání kolizí. Ne každá jednosměrná funkce proto musí být vhodnou hešovací funkcí a ne každá rychlá funkce s krátkým výstupem splňuje kryptografické požadavky.

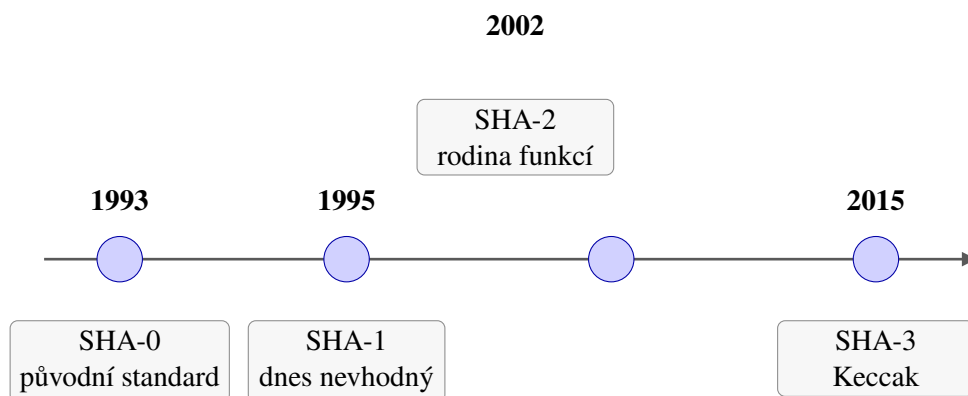


Hešovací funkce není šifrování bez klíče. Šifrování je navrženo tak, aby oprávněný příjemce mohl data obnovit. U hešování se žádná dešifrovací operace nepředpokládá a ztráta informace je záměrná.

6.9 Secure Hash Algorithm

Zkratka SHA pochází z anglického názvu *Secure Hash Algorithm*. Neoznačuje jedinou funkci, ale několik rodin standardizovaných kryptografických hešovacích funkcí.

- **SHA-0** byl zveřejněn v roce 1993 jako původní Secure Hash Standard.
- **SHA-1** jej v roce 1995 nahradil. Později byly nalezeny praktické kolize, a proto se pro nové bezpečnostní použití nepoužívá.
- **SHA-2** je rodina funkcí zahrnující například SHA-224, SHA-256, SHA-384 a SHA-512.
- **SHA-3** byl standardizován v roce 2015 a vychází z algoritmu Keccak. Má odlišnou vnitřní konstrukci než SHA-2.



Obrázek 6.18: Zjednodušená časová osa rodin SHA.

Pro nové aplikace se podle konkrétního standardu a protokolu používají zejména funkce z rodin SHA-2 a SHA-3. Skutečnost, že obě rodiny zůstávají v použití, není rozpor: mají jinou konstrukci a mohou představovat nezávislé možnosti pro různé systémy.

6.9.1 Sponge function

SHA-3 používá konstrukci nazývanou *sponge function*, doslova „houbová funkce“. Název připomíná dvě fáze: v první funkce postupně „nasává“ bloky zprávy a ve druhé ze svého vnitřního stavu „vymačkává“ výstupní bity.

Vnitřní stav má délku

$$b = r + c.$$

Prvních r bitů tvoří část *rate*, která komunikuje se vstupem a výstupem. Zbývajících c bitů tvoří *capacity*; nejsou přímo vydávány jako výstup a významně ovlivňují bezpečnost konstrukce. Celý stav zpracovává veřejná kryptografická permutace

$$f : \{0, 1\}^b \rightarrow \{0, 1\}^b.$$

Fáze absorbing. Zprávu m nejprve jednoznačně doplníme předepsaným paddingem a rozdělíme na r -bitové bloky

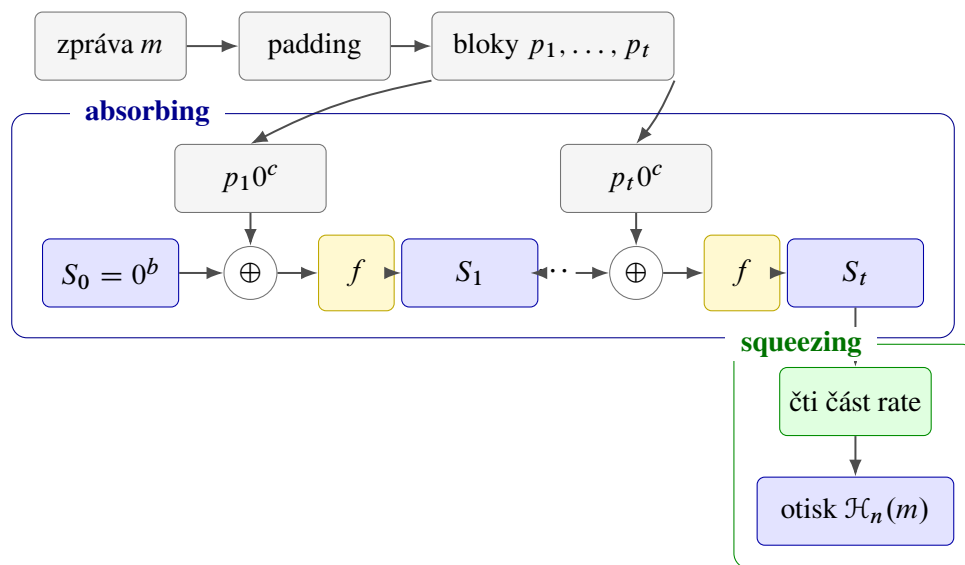
$$p_1, p_2, \dots, p_t.$$

Začneme nulovým stavem $S_0 = 0^b$. Každý blok prodloužíme o c nulových bitů, zkombinujeme s předchozím stavem operací XOR a na výsledek použijeme permutaci f :

$$S_i = f(S_{i-1} \oplus (p_i 0^c)), \quad i = 1, \dots, t.$$

Řetězec $p_i 0^c$ má délku $r + c = b$, takže jej lze XORovat s celým stavem. Vstupní blok ovlivní po aplikaci permutace obě části stavu.

Fáze squeezing. Po zpracování všech bloků čteme bity z části rate výsledného stavu S_t . Potřebujeme-li více než r výstupních bitů, znovu použijeme permutaci f a z nového stavu odebereme další část rate. Proces opakujeme, dokud nemáme požadovanou délku otisku.



Obrázek 6.19: Fáze absorbing a squeezing konstrukce sponge function.



Permutace f není tajný klíč. Je veřejnou a pevně určenou součástí algoritmu. Bezpečnost má vycházet z její konstrukce a ze způsobu, jakým se celý vnitřní stav používá, nikoli z utajení postupu.

6.9.2 SHA-3 jako konkrétní sponge function

SHA-3 používá stav délky

$$b = 1600.$$

Pro variantu SHA3- d s d -bitovým otiskem se volí

$$c = 2d, \quad r = 1600 - c.$$

Například SHA3-256 má

$$d = 256, \quad c = 512, \quad r = 1088.$$

Další běžné varianty jsou SHA3-224, SHA3-384 a SHA3-512. Liší se délkou otisku, a tím i rozdělením stavu na rate a capacity.

SHA-3 je navržen jako bezpečná kryptografická hešovací funkce, formální důkaz, že konkrétní SHA3- d splňuje definici jednosměrné funkce z definice 6.7.4, však znám není. To není u praktických kryptografických funkcí

neobvyklé: bezpečnost se opírá o rozsáhlou veřejnou analýzu, známé útoky a přesně popsané bezpečnostní předpoklady.



Délka otisku a vnitřní konstrukce řeší různé otázky. Delší otisk zvětšuje prostor možných hodnot, ale bezpečnost stále závisí také na tom, zda ve funkci neexistuje strukturální zkratka rychlejší než obecný útok hrubou silou.

6.10 Elektronický podpis

Šifrování chrání důvěrnost, ale samo o sobě nemusí prokazovat původ zprávy. Bob může obdržet šifrový text, který dokáže otevřít svým soukromým klíčem, a přesto neví, zda jej skutečně vytvořila Alice. Elektronický podpis má propojit konkrétní obsah zprávy se soukromým klíčem podepisující osoby.

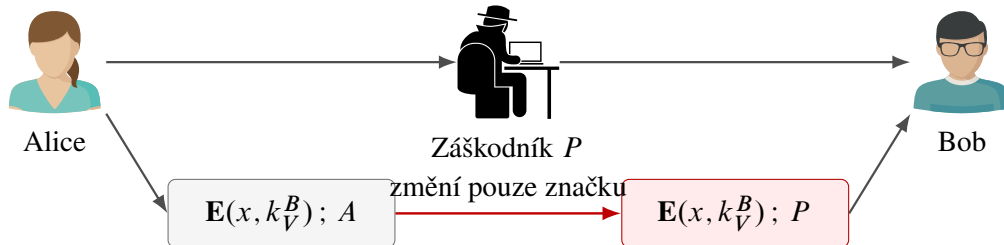
Proč nestačí připsat jméno? Představme si, že Alice pošle

$$\mathbf{E}(x, k_V^B)$$

a vedle šifrovaného textu pouze připojí značku „Alice“. Záškodník P může zprávu zachytit, značku nahradit a odeslat dál

$$\mathbf{E}(x, k_V^B); P.$$

Bob zprávu správně dešifruje, ale mylně ji přisoudí záškodníkovi. Obyčejný textový popis není se zprávou kryptograficky svázán.



Obrázek 6.20: Záškodník změnil nepodepsanou značku odesílatele.

Podpis musí záviset na obsahu zprávy i na tajné informaci podepisující osoby. Změní-li se zpráva, starý podpis už nesmí projít ověřením. Zároveň jej nesmí umět vytvořit nikdo, kdo nezná Alicin soukromý klíč k_S^A .

Komutativita klíčů jako didaktická představa. U učebnicového RSA platí nejen

$$\mathbf{D}(\mathbf{E}(x, k_V), k_S) = x,$$

ale díky násobení exponentů také obrácené pořadí

$$\mathbf{E}(\mathbf{D}(x, k_S), k_V) = x.$$

To nabízí jednoduchou představu podpisu. Alice použije na hodnotu x svůj soukromý klíč:

$$\sigma = \mathbf{D}(x, k_S^A).$$

Bob použije Alicin veřejný klíč a zkontroluje

$$\mathbf{E}(\sigma, k_V^A) \stackrel{?}{=} x.$$

Protože k_V^A je veřejný, podpis může ověřit kdokoliv. Vytvořit odpovídající σ však má umět pouze Alice se svým k_S^A .



Obecný elektronický podpis není „dešifrování soukromým klíčem“. Jde pouze o názornou intuici založenou na algebraické vlastnosti učebnicového RSA. Praktické podpisové schéma má samostatný algoritmus podepsání, algoritmus ověření a bezpečné kódování dat.

6.10.1 Podpis otisku

Podepisovat přímo dlouhou zprávu by bylo pomalé a nepraktické. Proto použijeme hešovací funkci

$$\mathcal{H}_n : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

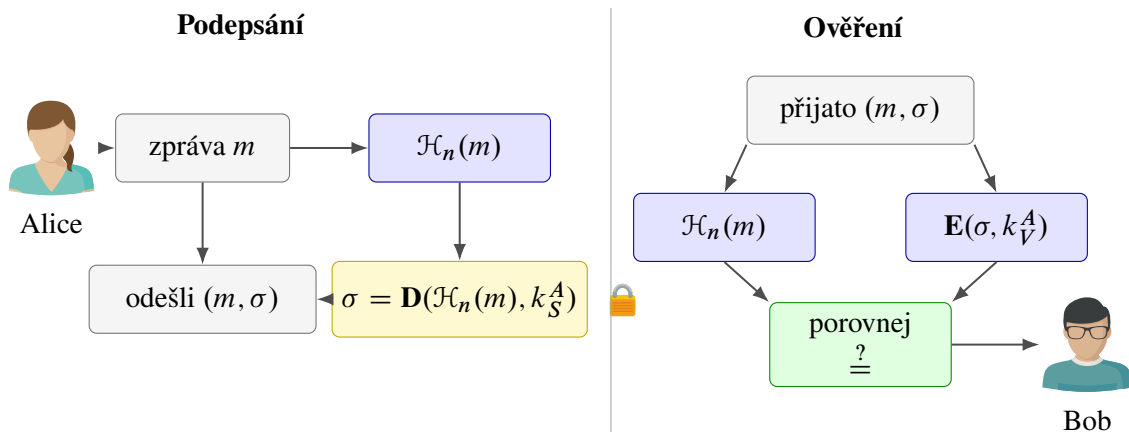
Alice nejprve spočítá krátký otisk $h = \mathcal{H}_n(m)$ a podepíše až jej:

$$\sigma = \mathbf{D}(\mathcal{H}_n(m), k_S^A).$$

Bob z přijaté zprávy sám vypočítá $\mathcal{H}_n(m)$ a ověří

$$\mathbf{E}(\sigma, k_V^A) \stackrel{?}{=} \mathcal{H}_n(m).$$

Pokud někdo zprávu pozmění, její nový otisk se s hodnotou získanou z podpisu neshoduje.



Obrázek 6.21: Podepsání otisku zprávy a následné ověření podpisu.

Odolnost hešovací funkce vůči kolizím je zde zásadní. Pokud by záškodník dokázal najít dvě různé zprávy $m \neq m'$ se stejným otiskem, mohl by se pokusit získat podpis neškodné zprávy m a později jej připojit k nebezpečné zprávě m' .

6.10.2 Certifikát a ověření veřejného klíče

Bob potřebuje vědět, že klíč k_V^A opravdu patří Alici. *Digitální certifikát* spojuje identitu držitele s jeho veřejným klíčem a toto spojení potvrzuje další důvěryhodná autorita svým podpisem. Ověření certifikátu proto není pouhým přečtením jména v souboru; příjemce kontroluje podpis vydavatele a navazující řetězec důvěry.

Certifikát neukládá Alicin soukromý klíč. Ten musí zůstat pod její kontrolou. Obsahuje zejména identifikační údaje, veřejný klíč, dobu platnosti a podpis vydavatele. Při ověřování je dále potřeba zkontrolovat, zda certifikát nevypršel nebo nebyl zneplatněn.



Obrázek 6.22: Certifikát kryptograficky spojuje identitu Alice s jejím veřejným klíčem.



Platný podpis dokládá, že podpis vytvořil někdo s přístupem k odpovídajícímu soukromému klíči a že se podepsaná data nezměnila. Pokud byl soukromý klíč odcizen nebo s ním bylo neoprávněně nakládáno, samotná matematická kontrola tuto okolnost neodhalí.

6.10.3 Elektronický podpis v právním rámci

Technický pojem elektronického podpisu je potřeba odlišovat od jeho právních kategorií. V Evropské unii oblast upravuje zejména nařízení eIDAS č. 910/2014 v aktuálním konsolidovaném znění. Rozlišuje elektronický podpis, zaručený elektronický podpis a kvalifikovaný elektronický podpis. Kvalifikovanému elektronickému podpisu přiznává právní účinek rovnocenný vlastnoručnímu podpisu; jinému elektronickému podpisu nelze právní účinek a přípustnost jako důkazu odmítnout jen proto, že má elektronickou podobu nebo nesplňuje požadavky kvalifikovaného podpisu.

V České republice na evropskou úpravu navazuje zejména zákon č. 297/2016 Sb., o službách vytvářejících důvěru pro elektronické transakce. Informace o kvalifikovaných poskytovatelích a službách zveřejňuje Digitální a informační agentura.



Právní předpisy a seznamy poskytovatelů se mohou měnit. Při skutečném právním jednání je proto potřeba ověřit aktuální znění předpisů, požadovaný druh podpisu a platnost použitého certifikátu.

Co podpis zajišťuje a co nikoli. Správně navržený a ověřený elektronický podpis pomáhá prokázat integritu dat a vazbu na soukromý klíč podepisující osoby. Nezajišťuje však důvěrnost: podepsaná zpráva může zůstat veřejně čitelná. Potřebujeme-li obsah zároveň utajit, kombinujeme podpis se šifrováním.

Pořadí a konkrétní způsob této kombinace musí určovat bezpečný protokol. Nestačí libovolně spojit dvě samostatně bezpečné operace. Moderní knihovny proto nabízejí hotová standardizovaná podpisová a šifrovací schémata; vytvářet vlastní kryptografický protokol bez odborné analýzy je velmi riskantní.



Při ověřování podpisu kontrolujeme současně tři vazby: otisk odpovídá přijatým datům, podpis odpovídá veřejnému klíči a certifikát důvěryhodně spojuje tento veřejný klíč s identitou podepisující osoby.

KAPITOLA 7

Umělá inteligence